

GravelKing Pro – Fixes Raw Source Audit

Complete current source implementations grouped by requested audit area

Purpose

This PDF contains the complete UTF-8 source contents of every file selected for the requested routing, responsive stage, duet ingress, video ingestion, library ordering, credit ledger, and Stripe webhook audit areas.

Raw-source guarantees

- Source files are embedded in full; no file content is truncated.
- No source placeholders, ellipses, or generated/minified bundles are included.
- File boundaries, byte counts, line counts, and SHA-256 hashes are listed below.
- Code pages use a monospaced font and preserve source line order and indentation.

Audit observation

The current snapshot contains email-gate components, but the inspected WebRTC ingress paths do not contain a combined email-plus-display-name preflight before getUserMedia. Those implementations are included exactly as present so the gap can be audited.

Generated (UTC): 2026-09-21 17:59:31

Files: 19 | Source lines: 9,591 | Source bytes: 383,347

Included source manifest

Each listed file appears in full in the following source sections

01. ROUTING & REWRITES
artifacts/gravelkingpro/.replit-artifact/artifact.toml
175 lines | 3,250 bytes | sha256 64d2bbbfaa75d330b963e649fc9f309337b73a2cdec4a7a8485ea93a024fa7ae
02. ROUTING & REWRITES
artifacts/gravelkingpro/src/App.tsx
319 lines | 13,225 bytes | sha256 0419517a399c8c6e4d3e2733a5e2b249965f9c6baecf7d73730fb389aeedac46
03. MAIN STAGE & RESPONSIVE TRANSPORT DOCK
artifacts/gravelkingpro/src/components/live-performance-stage.tsx
402 lines | 16,848 bytes | sha256 ac2a8b5fdaa357aba3cadb62f02d9e1e6b553c842208883bd3212fc0a607ad50
04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK
artifacts/gravelkingpro/src/pages/main-stage.tsx
1,687 lines | 76,283 bytes | sha256 4de666993601a06fbe95f0ccb04c7ff77048beb232d159eb017a3e0b345c78b1
05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK
artifacts/gravelkingpro/src/index.css
1,324 lines | 37,254 bytes | sha256 19d43db9c51c1fa3af2fa5e6e6ffe8ec6449014e3383c4820eddf0269556a73c
06. DUET INGRESS & LEAD CAPTURE
artifacts/gravelkingpro/src/pages/duet.tsx
972 lines | 46,650 bytes | sha256 d6f2058bf26620ba4b167c3544b1ccd6073d07e84b025157929f02276e92e5fc
07. DUET INGRESS & LEAD CAPTURE
artifacts/gravelkingpro/src/components/email-gate.tsx
108 lines | 3,919 bytes | sha256 95e84510cd293deed94fd82302a8eda2dd64ca930fa0cfa5e08b35b9edfb425d
08. DUET INGRESS & LEAD CAPTURE
artifacts/gravelkingpro/src/components/lead-capture-modal.tsx
221 lines | 9,786 bytes | sha256 f80e321d935ccd34e766675e33df6c820e0a3de3ec7af183c0ac8ab0d4a8505d
09. DUET INGRESS & LEAD CAPTURE
artifacts/api-server/src/routes/email-capture.ts
251 lines | 8,739 bytes | sha256 25d2e40344b9e3b548bc09b210e6e03485684bf7b227fa5f4ca7cedd4719cf35
10. DUET INGRESS & LEAD CAPTURE
artifacts/api-server/src/routes/lead-capture.ts
139 lines | 6,856 bytes | sha256 562b38a36309f89da3ecd1d3f94731c90412fb86f2318e8e18260a676f653740
11. VIDEO EXTRACTION & INGESTION
artifacts/api-server/src/routes/audio.ts
898 lines | 37,134 bytes | sha256 a0719f7317fbfe48f629ba1600bf54de630a23e0547d19019fe207f7fee34f19
12. VIDEO EXTRACTION & INGESTION
artifacts/api-server/src/routes/convert.ts
167 lines | 5,755 bytes | sha256 4b95149a495e214b7e23941f7370a42c82bd8bf3513fddfbaafa118016684c4a
13. VIDEO EXTRACTION & INGESTION
artifacts/api-server/src/middlewares/validateAssetIngestion.ts
366 lines | 14,600 bytes | sha256 c294a43c3978b39455216ea9d8dd2e6b49e0262fce472028a5bf6b4bebf3954
14. VAULT / LIBRARY ORDERING
artifacts/api-server/src/routes/tracks.ts
961 lines | 35,657 bytes | sha256 fedd8723d60237e1596b88ad19c3eee21efb99bf2ae7ad74506bee90aaeb7f7c
15. VAULT / LIBRARY ORDERING
artifacts/gravelkingpro/src/pages/library.tsx
684 lines | 29,872 bytes | sha256 d8622e836c9e883d1650977ce8719df9dd66924f78a5d6a3dcb81f71e2da4eae
16. VAULT / LIBRARY ORDERING
artifacts/gravelkingpro/src/lib/stage-catalog.ts
149 lines | 5,407 bytes | sha256 f1e500183bef5f653583e7c4cc8bc940fe5562ff571f40228620e7a49063082f
17. CREDIT LEDGER & STRIPE HOOKS
artifacts/api-server/src/lib/credits.ts
195 lines | 7,502 bytes | sha256 db597d33f9f2ec9a18c397745aecb7a370405eeb121f42c88fd4bdf5619be233
18. CREDIT LEDGER & STRIPE HOOKS
artifacts/api-server/src/webhookHandlers.ts
417 lines | 18,438 bytes | sha256 d4c340c8f70fe613d2e03834a15b90881f7c35fb8533029831bbf2beebd9ae88
19. CREDIT LEDGER & STRIPE HOOKS
artifacts/api-server/src/app.ts
156 lines | 6,172 bytes | sha256 4bf94a5f6ff8181beda4c566ff00eda4efb0a7906cf0e48f7430eac4a3aecda3

01. ROUTING & REWRITES

artifacts/gravelkingpro/.replit-artifact/artifact.toml | 175 lines | 3,250 bytes | sha256 64d2bbbfaa75d330b963e649fc9f309337b73a2cdec4a7a8485ea93a024fa7ae

```
kind = "web"
previewPath = "/"
title = "GravelKing Pro"
version = "1.0.0"
id = "artifacts/gravelkingpro"
router = "path"

[[integratedSkills]]
name = "react-vite"
version = "1.0.0"

[[services]]
name = "web"
paths = [ "/" ]
localPort = 19390

[services.development]
run = "pnpm --filter @workspace/gravelkingpro run dev"

[services.production]
build = [ "pnpm", "--filter", "@workspace/gravelkingpro", "run", "build" ]
publicDir = "artifacts/gravelkingpro/dist/public"
serve = "static"

[[services.production.rewrites]]
from = "/main-stage"
to = "/index.html"

[[services.production.rewrites]]
from = "/duet"
to = "/index.html"

[[services.production.rewrites]]
from = "/stage/duet/*"
to = "/index.html"

[[services.production.rewrites]]
from = "/pricing"
to = "/pricing.html"

[[services.production.rewrites]]
from = "/voice-removal"
to = "/mastering.html"

[[services.production.rewrites]]
from = "/mastering"
to = "/mastering.html"

[[services.production.rewrites]]
from = "/studio"
to = "/studio.html"

[[services.production.rewrites]]
from = "/songwriting"
to = "/songwriting.html"

[[services.production.rewrites]]
from = "/vocal-booth"
to = "/vocal-booth.html"

[[services.production.rewrites]]
from = "/download"
to = "/download.html"

[[services.production.rewrites]]
from = "/verify"
to = "/verify.html"

[[services.production.rewrites]]
from = "/whitepaper"
to = "/whitepaper.html"

[[services.production.rewrites]]
from = "/privacy"
to = "/privacy.html"

[[services.production.rewrites]]
from = "/data-deletion"
to = "/data-deletion.html"

[[services.production.rewrites]]
from = "/contact"
to = "/contact.html"

[[services.production.rewrites]]
from = "/label"
to = "/label.html"

[[services.production.rewrites]]
from = "/submit"
to = "/submit.html"

[[services.production.rewrites]]
from = "/optimizer"
to = "/optimizer.html"

[[services.production.rewrites]]
from = "/convert"
to = "/convert.html"

[[services.production.rewrites]]
from = "/report"
to = "/index.html"

[[services.production.rewrites]]
from = "/kernel"
to = "/index.html"

[[services.production.rewrites]]
from = "/account"
to = "/index.html"

[[services.production.rewrites]]
```

01. ROUTING & REWRITES (continued)

artifacts/gravelkingpro/.replit-artifact/artifact.toml | source continues without omission

```
from = "/library"
to = "/index.html"

[[services.production.rewrites]]
from = "/protected-lyrics"
to = "/index.html"

[[services.production.rewrites]]
from = "/weekend-special"
to = "/index.html"

[[services.production.rewrites]]
from = "/pitch"
to = "/index.html"

[[services.production.rewrites]]
from = "/demo"
to = "/index.html"

[[services.production.rewrites]]
from = "/sign-in/*"
to = "/index.html"

[[services.production.rewrites]]
from = "/sign-up/*"
to = "/index.html"

[[services.production.rewrites]]
from = "/login"
to = "/index.html"

[[services.production.rewrites]]
from = "/demo-login"
to = "/index.html"

[[services.production.rewrites]]
from = "/help"
to = "/index.html"

[[services.production.rewrites]]
from = "/help/*"
to = "/index.html"

[[services.production.rewrites]]
from = "/promoters"
to = "/index.html"

[[services.production.rewrites]]
from = "/admin"
to = "/index.html"

[[services.production.rewrites]]
from = "/admin/*"
to = "/index.html"

[[services.production.rewrites]]
from = "/label/*"
to = "/index.html"

[services.env]
PORT = "19390"
BASE_PATH = "/"
```

02. ROUTING & REWRITES

artifacts/gravelkingpro/src/App.tsx | 319 lines | 13,225 bytes | sha256 0419517a399c8c6e4d3e2733a5e2b249965f9c6baecf7d73730fb389aeedac46

```
import { useEffect, useRef } from "react";
import { ClerkProvider, SignIn, SignUp, useClerk } from "@clerk/react";
import { publishableKeyFromHost } from "@clerk/react/internal";
import { shadcn } from "@clerk/themes";
import { Switch, Route, Router as WouterRouter, useLocation } from "wouter";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { Toaster } from "@components/ui/toaster";
import { TooltipProvider } from "@components/ui/tooltip";
import { AppProvider } from "@lib/context";
import NotFound from "@pages/not-found";
import { Navigation } from "@components/Navigation";
import ArtistPage from "@pages/artist";
import Pricing from "@pages/pricing";
import Report from "@pages/report";
import Mastering from "@pages/mastering";
import KernelDashboard from "@pages/kernel";
import Contact from "@pages/contact";
import DownloadPage from "@pages/download";
import Account from "@pages/account";
import AdminAnalytics from "@pages/admin-analytics";
import AdminWaitlist from "@pages/admin-waitlist";
import AdminTracks from "@pages/admin-tracks";
import LabelPage from "@pages/label";
import LabelArtistPage from "@pages/label-artist";
import LibraryPage from "@pages/library";
import SubmitTrackPage from "@pages/submit-track";
import ConvertPage from "@pages/convert";
import SongwritingStudio from "@pages/songwriting";
import ProtectedLyrics from "@pages/protected-lyrics";
import VocalBooth from "@pages/vocal-booth";
import TrackPrepPage from "@pages/workshop";
import MainStage from "@pages/main-stage";
import AdminLabel from "@pages/admin-label";
import AdminOps from "@pages/admin-ops";
import AdminOrders from "@pages/admin-orders";
import AdminEmails from "@pages/admin-emails";
import AdminIntegrationDemo from "@pages/admin-integration-demo";
import PitchPage from "@pages/pitch";
import VerifyPage from "@pages/verify";
import WhitepaperPage from "@pages/whitepaper";
import PrivacyPolicy from "@pages/privacy";
import DataDeletionPage from "@pages/data-deletion";
import WeekendSpecial from "@pages/weekend-special";
import { PageTracker } from "@lib/useAnalytics";
import { RouteSeo } from "@lib/seo";
import Demologin from "@pages/demo-login";
import HelpPage from "@pages/help";
import AdminPublishChecklist from "@pages/admin-publish-checklist";
import PromotersPage from "@pages/promoters";
import AdminPromoters from "@pages/admin-promoters";
import AdminDecks from "@pages/admin-decks";
import AdminInvestors from "@pages/admin-investors";
import DuetRoom from "@pages/duet";
import { Layout } from "@components/layout";
import { Archive, Audiolines, Mic2, Sparkles, UserRound } from "lucide-react";
import { Link } from "wouter";

/**
 * Referral link capture: if the URL carries ?ref=CODE, report it to the server
 * once so it can set the httpOnly attribution cookie (survives the Stripe
 * checkout redirect). Best-effort; never blocks rendering.
 */
function RefCapture() {
  useEffect(() => {
    const params = new URLSearchParams(window.location.search);
    const code = params.get("ref");
    if (!code) return;
    void fetch("/api/referral/click", {
      method: "POST",
      credentials: "include",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ code }),
    }).catch(() => {});
  }, []);
  return null;
}

const queryClient = new QueryClient();

const clerkPubKey = publishableKeyFromHost(
  window.location.hostname,
  import.meta.env.VITE_CLERK_PUBLISHABLE_KEY,
);
const clerkProxyUrl = import.meta.env.VITE_CLERK_PROXY_URL;
const basePath = import.meta.env.BASE_URL.replace(/\/$/, "");

function stripBase(path: string): string {
  return basePath && path.startsWith(basePath)
    ? path.slice(basePath.length) || "/"
    : path;
}

const clerkAppearance = {
  theme: shadcn,
  cssLayerName: "clerk",
  options: {
    logoPlacement: "inside" as const,
    logoLinkUrl: basePath || "/",
    logoImageUrl: `${window.location.origin}${basePath}/logo.svg`,
  },
  variables: {
    colorPrimary: "#5a9bb",
    colorForeground: "#fafafa",
    colorMutedForeground: "#a1a1aa",
    colorDanger: "#fb7185",
    colorBackground: "#18181b",
    colorInput: "#27272a",
    colorInputForeground: "#fafafa",
    colorNeutral: "#3f3f46",
    fontFamily: "Inter, sans-serif",
    borderRadius: "0.75rem",
  },
  elements: {
```

02. ROUTING & REWRITES (continued)

artifacts/gravelkingpro/src/App.tsx | source continues without omission

```
    rootBox: "w-full flex justify-center",
    cardBox: "bg-zinc-900 rounded-2xl w-[440px] max-w-full overflow-hidden",
    card: "!shadow-none !border-0 !bg-transparent !rounded-none",
    footer: "!shadow-none !border-0 !bg-transparent !rounded-none",
    headerTitle: "text-zinc-50",
    headerSubtitle: "text-zinc-400",
    socialButtonsBlockButtonText: "text-zinc-100",
    formFieldLabel: "text-zinc-300",
    footerActionLink: "text-amber-400 hover:text-amber-300",
    footerActionText: "text-zinc-400",
    dividerText: "text-zinc-400",
    identityPreviewEditButton: "text-amber-400",
    formFieldSuccessText: "text-emerald-400",
    alertText: "text-rose-300",
    logoBox: "rounded-xl overflow-hidden",
    logoImage: "rounded-xl",
    socialButtonsBlockButton: "border-zinc-700 bg-zinc-800 hover:bg-zinc-700",
    formButtonPrimary: "bg-amber-500 text-zinc-950 hover:bg-amber-400",
    formFieldInput: "bg-zinc-800 border-zinc-700 text-zinc-50",
    footerAction: "text-zinc-400",
    dividerLine: "bg-zinc-700",
    alert: "bg-rose-950/40 border-rose-800",
    otpCodeFieldInput: "bg-zinc-800 border-zinc-700 text-zinc-50",
    formFieldRow: "text-zinc-300",
    main: "bg-transparent",
  },
};

function SignInPage() {
  // After sign-in, send the user to the pricing page with Studio pre-selected
  // so they land on the offer rather than the homepage. The pricing page's
  // auto-resume logic picks up ?plan=monthly and opens Stripe checkout
  // automatically – plan-specific redirects set by handleCheckout take priority.
  const afterUrl = `${basePath}/pricing?plan=monthly`;
  return (
    <div className="flex min-h-[100dvh] items-center justify-center bg-background px-4">
      <SignIn routing="path" path={` ${basePath}/sign-in`} signInUrl={` ${basePath}/sign-up`} fallbackRedirectUrl={afterUrl} signUpFallbackRedirectUrl={afterUrl} />
    </div>
  );
}

function SignUpPage() {
  // Same intent: new users land on the pricing page, not the homepage.
  const afterUrl = `${basePath}/pricing?plan=monthly`;
  return (
    <div className="flex min-h-[100dvh] items-center justify-center bg-background px-4">
      <SignUp routing="path" path={` ${basePath}/sign-up`} signInUrl={` ${basePath}/sign-in`} fallbackRedirectUrl={afterUrl} signInFallbackRedirectUrl={afterUrl} />
    </div>
  );
}

function ClerkQueryClientCacheInvalidator() {
  const { addListener } = useClerk();
  const previousUserId = useRef<string | null | undefined>(undefined);

  useEffect(() => {
    const unsubscribe = addListener(({ user }) => {
      const userId = user?.id ?? null;
      if (previousUserId.current !== undefined && previousUserId.current !== userId) {
        queryClient.clear();
      }
      previousUserId.current = userId;
    });
    return unsubscribe;
  }, [addListener]);

  return null;
}

function Router() {
  return (
    <Switch>
      <Route path="/" component={CoreLanding} />
      <Route path="/mastering" component={Mastering} />
      <Route path="/pricing" component={Pricing} />
      <Route path="/report" component={Report} />
      <Route path="/kernel" component={KernelDashboard} />
      <Route path="/contact" component={Contact} />
      <Route path="/download" component={DownloadPage} />
      <Route path="/account" component={Account} />
      <Route path="/artist" component={ArtistPage} />
      <Route path="/admin" component={AdminAnalytics} />
      <Route path="/admin/waitlist" component={AdminWaitlist} />
      <Route path="/admin/tracks" component={AdminTracks} />
      <Route path="/admin/label" component={AdminLabel} />
      <Route path="/admin/ops" component={AdminOps} />
      <Route path="/admin/orders" component={AdminOrders} />
      <Route path="/admin/emails" component={AdminEmails} />
      <Route path="/admin/integration-demo" component={AdminIntegrationDemo} />
      <Route path="/label" component={LabelPage} />
      <Route path="/label/:artist" component={LabelArtistPage} />
      <Route path="/library" component={LibraryPage} />
      <Route path="/submit" component={SubmitTrackPage} />
      <Route path="/convert" component={ConvertPage} />
      <Route path="/songwriting" component={SongwritingStudio} />
      <Route path="/protected-lyrics" component={ProtectedLyrics} />
      <Route path="/vocal-booth" component={VocalBooth} />
      <Route path="/studio/track-prep" component={TrackPrepPage} />
      <Route path="/stage/duet/:roomId" component={MainStage} />
      <Route path="/main-stage" component={MainStage} />
      <Route path="/duet" component={DuetRoom} />
      <Route path="/pitch" component={PitchPage} />
      <Route path="/verify" component={VerifyPage} />
      <Route path="/whitepaper" component={WhitepaperPage} />
      <Route path="/privacy" component={PrivacyPolicy} />
      /* Public by design: users and store reviewers must not need an account to request deletion. */
      <Route path="/data-deletion" component={DataDeletionPage} />
      <Route path="/weekend-special" component={WeekendSpecial} />
      <Route path="/demo" component={VerifyPage} />
      <Route path="/sign-in/*?" component={SignInPage} />
      <Route path="/sign-up/*?" component={SignUpPage} />
      <Route path="/login" component={SignInPage} />
    </Switch>
  );
}
```

02. ROUTING & REWRITES (continued)

artifacts/gravelkingpro/src/App.tsx | source continues without omission

```

    <Route path="/demo-login" component={DemoLogin} />
    <Route path="/help/:slug" component={HelpPage} />
    <Route path="/help" component={HelpPage} />
    <Route path="/admin/publish-checklist" component={AdminPublishChecklist} />
    <Route path="/promoters" component={PromotersPage} />
    <Route path="/admin/promoters" component={AdminPromoters} />
    <Route path="/admin/investors" component={AdminInvestors} />
    <Route path="/admin/decks" component={AdminDecks} />
    <Route component={NotFound} />
  </Switch>
);
}

function CoreLanding() {
  const destinations = [
    { href: "/mastering", label: "Studio", detail: "Build and finish", icon: Audiolines },
    { href: "/main-stage", label: "Main Stage", detail: "Perform and monitor", icon: Mic2 },
    { href: "/songwriting", label: "Jax", detail: "Write and revise", icon: Sparkles },
    { href: "/library", label: "Vault", detail: "Keep your work", icon: Archive },
    { href: "/artist", label: "Artist", detail: "Profile and pass", icon: UserRound },
  ];

  return (
    <Layout>
      <main className="mx-auto flex min-h-[calc(100vh-10rem)] max-w-5xl flex-col justify-center py-10">
        <p className="text-[10px] font-bold uppercase tracking-[0.3em] text-amber-300/80">Studio control surface</p>
        <h1 className="mt-4 max-w-2xl text-4xl font-black tracking-tight text-zinc-100 sm:text-6xl">
          GravelKing <span className="text-amber-300">Pro</span>
        </h1>
        <p className="mt-4 text-sm text-zinc-500">Choose a workspace.</p>
        <div className="mt-10 grid gap-3 sm:grid-cols-2 lg:grid-cols-3">
          {destinations.map(({ href, label, detail, icon: Icon }) => (
            <Link key={href} href={href} className="studio-card group rounded-2xl border border-white/10 p-5 transition-colors hover:border-amber-300/40 hover:bg-amber-300/5">
              <Icon className="h-5 w-5 text-amber-300" />
              <p className="mt-7 text-lg font-bold text-zinc-100">{label}</p>
              <p className="mt-1 text-xs uppercase tracking-[0.14em] text-zinc-600 group-hover:text-zinc-400">{detail}</p>
            </Link>
          ))}
        </div>
      </main>
    </Layout>
  );
}

function App() {
  return (
    <WouterRouter base={basePath}>
      <ClerkProviderWithRoutes />
    </WouterRouter>
  );
}

function ClerkProviderWithRoutes() {
  const [, setLocation] = useLocation();

  if (!clerkPubKey) {
    throw new Error("Missing VITE_CLERK_PUBLISHABLE_KEY in .env file");
  }

  return (
    <ClerkProvider
      publishableKey={clerkPubKey}
      proxyUrl={clerkProxyUrl}
      appearance={clerkAppearance}
      signInUrl={` ${basePath}/sign-in`}
      signUpUrl={` ${basePath}/sign-up`}
      localization={{
        signIn: { start: { title: "Welcome back", subtitle: "Sign in to GravelKing Pro" } },
        signUp: { start: { title: "Create your account", subtitle: "Start building with GravelKing Pro" } },
      }}
      routerPush={(to) => setLocation(stripBase(to))}
      routerReplace={(to) => setLocation(stripBase(to), { replace: true })}
    >
      <QueryClientProvider client={queryClient}>
        <ClerkQueryClientCacheInvalidator />
        <TooltipProvider>
          <AppProvider>
            <RouteSee />
            <PageTracker />
            <RefCapture />
            <div className="phase-one-app">
              <Router />
              <Navigation />
            </div>
            <Toaster />
          </AppProvider>
        </TooltipProvider>
      </QueryClientProvider>
    </ClerkProvider>
  );
}

export default App;
```

03. MAIN STAGE & RESPONSIVE TRANSPORT DOCK

artifacts/gravelkingpro/src/components/live-performance-stage.tsx | 402 lines | 16,848 bytes | sha256 ac2a8b5fdaa357aba3cadb62f02d9e1e6b553c842208883bd3212fc0a607ad50

```
import { useEffect, useMemo, useRef, type CSSProperties, type ReactNode } from "react";
import {
  ChevronLeft,
  Maximize2,
  Mic2,
  Pause,
  Play,
  Radio,
  Square,
  UserRound,
  Volume2,
} from "lucide-react";
import { Button } from "@/components/ui/button";
import { RollingPrompter } from "@/components/prompter/RollingPrompter";

export type StageLyric = {
  id?: string;
  text: string;
  startTimeMs: number;
  endTimeMs: number;
};

export type StagePerformer = {
  id: string;
  label: string;
  name: string;
  avatarUrl?: string | null;
  accent: "gold" | "violet";
  analyser?: AnalyserNode | null;
  status?: string;
  onInvite?: () => void;
};

interface LivePerformanceStageProps {
  title: string;
  subtitle?: string;
  trackName?: string;
  isPlaying: boolean;
  currentTimeSec: number;
  durationSec: number;
  lyrics?: StageLyric[];
  performers: StagePerformer[];
  backingAnalyser?: AnalyserNode | null;
  onExit: () => void;
  onTogglePlayback?: () => void;
  onStop?: () => void;
  onSeek?: (fraction: number) => void;
  onRecord?: () => void;
  isRecording?: boolean;
  recordDisabled?: boolean;
  controlSlot?: ReactNode;
  headerSlot?: ReactNode;
  /** Rendered inside the jumbotron when no lyrics/track are loaded (primary CTA card). */
  emptySlot?: ReactNode;
  stageMode?: "solo" | "duet";
  audioContextTimeSec?: number;
  playbackStartContextTimeSec?: number | null;
}

function formatTime(seconds: number): string {
  if (!Number.isFinite(seconds) || seconds < 0) return "0:00";
  const minutes = Math.floor(seconds / 60);
  const remaining = Math.floor(seconds % 60);
  return `${minutes}:${remaining.toString().padStart(2, "0")}`;
}

function sampleAnalyser(analyser: AnalyserNode, buffer: Uint8Array<ArrayBuffer>): number {
  analyser.getByteFrequencyData(buffer);
  if (buffer.length === 0) return 0;

  // Ignore the very lowest bins: mic rumble should not make the riser look live.
  const start = Math.floor(buffer.length * 0.04);
  const end = Math.max(start + 1, Math.floor(buffer.length * 0.72));
  let sum = 0;
  for (let i = start; i < end; i += 1) sum += buffer[i] ?? 0;
  return Math.min(1, (sum / (end - start)) / 255 * 1.85);
}

export function LivePerformanceStage({
  title,
  subtitle = "Live performance mode",
  trackName,
  isPlaying,
  currentTimeSec,
  durationSec,
  lyrics = [],
  performers,
  backingAnalyser = null,
  onExit,
  onTogglePlayback,
  onStop,
  onSeek,
  onRecord,
  isRecording = false,
  recordDisabled = false,
  controlSlot,
  emptySlot,
  headerSlot,
  stageMode = "duet",
  audioContextTimeSec,
  playbackStartContextTimeSec,
}: LivePerformanceStageProps) {
  const stageRef = useRef<HTMLDivElement | null>(null);
  const performerRefs = useRef<Record<string, HTMLDivElement | null>>({});
  const performersRef = useRef(performers);
  performersRef.current = performers;

  const analyserKey = performers.map((performer) => `${performer.id}:${performer.analyser ? "live" : "idle"}`).join("|");
  const timelineEndMs = useMemo(() => {
    const lyricEnd = lyrics.reduce((max, lyric) => Math.max(max, lyric.endTimeMs), 0);
    return Math.max(1, durationSec * 1000, lyricEnd);
  }, [durationSec, lyrics]);
  const currentTimeMs = Math.max(0, currentTimeSec * 1000);
```

03. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/components/live-performance-stage.tsx | source continues without omission

```
const activeIndex = lyrics.findIndex(
  (lyric) => currentTimeMs >= lyric.startTimeMs && currentTimeMs < lyric.endTimeMs,
);
const activeLyric = activeIndex >= 0 ? lyrics[activeIndex] : null;
const cursorLeft = Math.max(0, Math.min(100, (currentTimeMs / timelineEndMs) * 100));
const progress = durationSec > 0 ? Math.max(0, Math.min(1, currentTimeSec / durationSec)) : 0;

useEffect(() => {
  const root = stageRef.current;
  if (!root) return;

  const buffers = new Map<string, Uint8Array<ArrayBuffer>>>();
  let frame = 0;
  const tick = () => {
    let backingLevel = 0;
    for (const performer of performersRef.current) {
      const riser = performerRefs.current[performer.id];
      const analyser = performer.analyser;
      let level = 0;
      if (analyser) {
        const existing = buffers.get(performer.id);
        const data = existing && existing.length === analyser.frequencyBinCount
          ? existing
          : new Uint8Array(analyser.frequencyBinCount) as Uint8Array<ArrayBuffer>;
        buffers.set(performer.id, data);
        level = sampleAnalyser(analyser, data);
      }
      if (riser) {
        riser.style.setProperty("--mic-level", level.toFixed(3));
        riser.style.setProperty("--mic-scale", (level * 0.16).toFixed(3));
        riser.style.setProperty("--mic-glow", (0.18 + level * 0.72).toFixed(3));
        riser.style.setProperty("--mic-blur", `${10 + Math.round(level * 22)}px`);
        riser.dataset.micLive = level > 0.025 ? "true" : "false";
      }
      backingLevel = Math.max(backingLevel, level);
    }

    if (backingAnalyser) {
      const existing = buffers.get("_backing_");
      const data = existing && existing.length === backingAnalyser.frequencyBinCount
        ? existing
        : new Uint8Array(backingAnalyser.frequencyBinCount) as Uint8Array<ArrayBuffer>;
      buffers.set("_backing_", data);
      backingLevel = Math.max(backingLevel, sampleAnalyser(backingAnalyser, data));
    }

    root.style.setProperty("--stage-audio-level", backingLevel.toFixed(3));
    root.style.setProperty("--stage-beat-scale", (backingLevel * 0.045).toFixed(3));
    frame = requestAnimationFrame(tick);
  };

  frame = requestAnimationFrame(tick);
  return () => {
    cancelAnimationFrame(frame);
    buffers.clear();
  };
}, [analyserKey, backingAnalyser, isPlaying]);

const sharedVars = {
  "--stage-progress": `${progress * 100}%`,
} as CSSProperties;

return (
  <div
    ref={stageRef}
    className="live-performance-stage fixed inset-0 z-50 flex min-h-0 flex-col overflow-hidden"
    data-playing={isPlaying}
    data-stage-mode={stageMode}
    data-testid="live-performance-stage"
    style={sharedVars}
  >
    <div className="stage-ambient-glow stage-ambient-glow-left" />
    <div className="stage-ambient-glow stage-ambient-glow-right" />

    <header className="stage-command-bar relative flex shrink-0 items-center justify-between gap-3 px-4 py-3 sm:px-6">
      <div className="flex min-w-0 items-center gap-3">
        <button
          type="button"
          onClick={onExit}
          className="inline-flex h-11 min-h-11 min-w-11 shrink-0 items-center justify-center gap-1.5 rounded-full border border-white/15 bg-white/5 px-3 text-[10px] font-bold uppercase tracking-[0.12em] text-white/70 transition-colors hover:border-amber-300/45 hover:bg-amber-300/10 hover:text-amber-200"
          title="Exit live performance stage"
          aria-label="Exit live performance stage"
        >
          <ChevronLeft className="h-4 w-4" />
          <span className="hidden sm:inline">Exit Stage</span>
        </button>
        <div className="min-w-0">
          <div className="flex items-center gap-2">
            <Radio className="h-3.5 w-3.5 shrink-0 text-amber-300" />
            <span className="stage-eyebrow">{subtitle}</span>
          </div>
          <h1 className="truncate text-sm font-semibold text-white sm:text-base">{title}</h1>
        </div>
      </div>
      {headerSlot && (
        <div className="absolute left-1/2 top-1/2 z-10 -translate-x-1/2 -translate-y-1/2">
          {headerSlot}
        </div>
      )}
      <div className="hidden min-w-0 items-center gap-2 text-right sm:flex">
        <div className="min-w-0">
          <div className="stage-eyebrow">Now on stage</div>
          <div className="truncate text-xs text-white/80">{trackName || "Untitled performance"}</div>
        </div>
        <Maximize2 className="h-4 w-4 text-violet-300/70" />
      </div>
    </header>

    <main className="stage-viewport min-h-0 flex-1 overflow-hidden px-3 pb-2 pt-3 sm:px-8 sm:pt-5">
      <div className="stage-truss" aria-label="Arena lighting truss">
        <div className="stage-truss-beam stage-truss-beam-back" />
        <div className="stage-truss-beam stage-truss-beam-front" />
        <svg className="stage-spotlight-svg" viewBox="0 0 1000 420" preserveAspectRatio="none" aria-hidden="true">
```

03. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/components/live-performance-stage.tsx | source continues without omission

```
<defs>
  <linearGradient id="stage-spot-gold" x1="0" x2="0" y1="0" y2="1">
    <stop offset="0" stopColor="#f8d477" stopOpacity="0.58" />
    <stop offset="1" stopColor="#f8b84e" stopOpacity="0" />
  </linearGradient>
  <linearGradient id="stage-spot-violet" x1="0" x2="0" y1="0" y2="1">
    <stop offset="0" stopColor="#b89bff" stopOpacity="0.48" />
    <stop offset="1" stopColor="#7c5cff" stopOpacity="0" />
  </linearGradient>
</defs>
<polygon className="stage-spotlight stage-spotlight-left" points="115,28 190,28 410,390 235,390" fill="url(#stage-spot-gold)" />
<polygon className="stage-spotlight stage-spotlight-center" points="468,26 532,26 630,390 370,390" fill="url(#stage-spot-violet)" />
<polygon className="stage-spotlight stage-spotlight-right" points="810,28 885,28 765,390 590,390" fill="url(#stage-spot-gold)" />
</svg>
</div>

<section className="stage-jumbotron" aria-label="Live lyric display">
  <div className="stage-jumbotron-bezel">
    <div className="stage-jumbotron-mesh" />
    <div className="stage-jumbotron-content">
      <span className="stage-jumbotron-label">
        {isPlaying ? "Live teleprompter" : "Stand by"}
      </span>
      {lyrics.length > 0 ? (
        <RollingPrompter
          lines={lyrics}
          currentTimeSec={currentTimeSec}
          isPlaying={isPlaying}
          audioContextTimeSec={audioContextTimeSec}
          playbackStartContextTimeSec={playbackStartContextTimeSec}
          className="stage-lyric-stack"
        />
      ) : emptySlot ? (
        emptySlot
      ) : (
        <div className="flex flex-col items-center gap-2 text-center">
          {trackName ? (
            <
              <p className="stage-lyric-line stage-lyric-line-active truncate">{trackName}</p>
              <span className="rounded-full border border-sky-300/25 bg-sky-300/10 px-3 py-1 text-[11px] font-bold uppercase tracking-widest text-sky-200">
                □ Audio Only Mode – Free Performance
              </span>
            </>
          ) : (
            <p className="stage-lyric-line stage-lyric-line-active">Instrumental passage</p>
          )}
        </div>
      )}
    </div>
  </div>
</div>
<div className="stage-cadence-ribbon" aria-label="Lyric cadence timeline">
  <span className="stage-strike-cursor" style={{ left: `${cursorLeft}%` }} />
  {lyrics.map((lyric, index) => {
    const left = `${(lyric.startTimeMs / timelineEndMs) * 100}%`;
    const width = `${((lyric.endTimeMs - lyric.startTimeMs) / timelineEndMs) * 100}%`;
    return (
      <span
        key={lyric.id ?? `${lyric.startTimeMs}-${index}`}
        className={`stage-cadence-block ${index === activeIndex ? "stage-cadence-block-active" : ""}`}
        style={{ left, width }}
        title={`${formatTime(lyric.startTimeMs / 1000)} · ${lyric.text}`}
      />
    );
  })}
</div>
</section>

<section className="stage-deck-wrap" aria-label="Concert stage floor">
  <div className="stage-deck">
    <div className="stage-deck-grid" aria-hidden="true">
      {Array.from({ length: 48 }, (_, index) => <span key={index} className="stage-deck-tile" />)}
    </div>
    <div className="stage-performer-row">
      {performers.map((performer) => (
        <div
          key={performer.id}
          ref={node} => { performerRefs.current[performer.id] = node; }}
          className={`stage-performer-riser stage-performer-riser-${performer.accent}`}
          data-mic-live="false"
          data-testid={`stage-performer-${performer.id}`}
        >
          <div className="stage-riser-light" />
          {performer.onInvite ? (
            <button
              type="button"
              onClick={performer.onInvite}
              className="stage-invite-partner-button"
              aria-label="Invite duet partner"
            >
              <span className="stage-invite-plus">+</span>
              <span>Invite Duet Partner</span>
            </button>
          ) : (
            <
              <div className="stage-performer-avatar">
                {performer.avatarUrl ? (
                  <img src={performer.avatarUrl} alt="" />
                ) : (
                  <UserRound className="h-8 w-8" />
                )}
              </div>
              <div className="stage-performer-name">{performer.name}</div>
              <div className="stage-performer-status">
                <Mic2 className="h-3 w-3" />
                {performer.status || performer.label}
              </div>
            </>
          )}
          <div className="stage-riser-top" />
          <div className="stage-riser-front" />
        </div>
      )}
    </div>
  </div>
</div>
```

03. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

```

<div className="stage-floor-front-led" />
</div>
</section>
</main>

<footer className="stage-control-rail pb-safe shrink-0 px-3 pt-2 sm:px-6">
  <div className="stage-control-inner flex-col items-stretch gap-2 sm:flex-row sm:items-center">
    <div className="stage-control-timeline min-w-0 flex-1">
      <div className="flex items-center justify-between gap-2 text-[10px] text-white/55">
        <span className="font-mono">{formatTime(currentTimeSec)}</span>
        <span className="hidden truncate sm:block">{trackName || "No backing track"}</span>
        <span className="font-mono">{formatTime(durationSec)}</span>
      </div>
      <input
        type="range"
        min={0}
        max={1}
        step={0.001}
        value={progress}
        onChange={(event) => onSeek?.(Number(event.target.value))}
        disabled={!onSeek || durationSec <= 0}
        className="stage-progress-slider w-full"
        aria-label="Playback position"
      />
    </div>
    <div className="stage-control-actions flex min-w-0 flex-wrap items-center justify-between gap-1.5 sm:justify-end">
      <button
        size="sm"
        onClick={onTogglePlayback}
        className="studio-button-primary h-11 min-h-11 w-11 min-w-11 rounded-full bg-amber-400 p-0 text-black hover:bg-amber-300"
        aria-label={isPlaying ? "Pause playback" : "Play playback"}
      >
        {isPlaying ? <Pause className="h-4 w-4" /> : <Play className="ml-0.5 h-4 w-4" />}
      </button>
    </div>
    <div>
      <button type="button" className="stage-transport-button min-h-11 min-w-11" onClick={onStop} title="Stop playback" aria-label="Stop playback">
        <Square className="h-3.5 w-3.5" />
      </button>
    </div>
    <div>
      <button
        type="button"
        onClick={onRecord}
        disabled={recordDisabled}
        className={`stage-record-button min-h-11 min-w-11 ${isRecording ? "stage-record-button-active" : ""}`}
        title={isRecording ? "Stop recording" : "Start recording"}
        aria-label={isRecording ? "Stop recording" : "Start recording"}
      >
        {isRecording ? <Square className="h-4 w-4 fill-current" /> : <Mic2 className="h-4 w-4" />}
      </button>
    </div>
    <div>
      <span className="hidden items-center gap-1 text-[10px] uppercase tracking-widest text-white/45 md:flex">
        <Volume2 className="h-3 w-3" /> Live
      </span>
    </div>
  </div>
</footer>
</div>
);

```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK

artifacts/gravelkingpro/src/pages/main-stage.tsx | 1,687 lines | 76,283 bytes | sha256 4de666993601a06fbe95f0ccb04c7ff77048beb232d159eb017a3e0b345c78b1

```
/**
 * Main Stage – Karaoke performance surface with live mic monitoring.
 *
 * Layout:
 * - Full-screen local performer: rolling lyrics prompter + transport at full width
 * - Floating partner HUD: audio-reactive corner bubble when a duet peer connects
 * - FX rack overlay panel (right-side drawer, opens when monitor starts)
 * - Global bottom nav is suppressed by Navigation.tsx on this route
 *
 * Audio graph (direct sidetone, no inserted delay):
 * mic → gainNode → ctx.destination          (dry, latencyHint: interactive)
 *   gainNode → analyser                      (meter tap, parallel)
 *   gainNode → hpFilter → warmthFilter → reverbSend → convolver → reverbWet → ctx.destination
 *
 * Duet signaling: same in-memory SSE relay as /duet, reused under a GK-STAGE-XXXX room code.
 */

import { useCallback, useEffect, useRef, useState } from "react";
import { Layout } from "@components/layout";
import { LivePerformanceStage } from "@components/live-performance-stage";
import { DuetReviewCard, encodePcm16Wav } from "@components/stage/DuetRoom";
import { Slider } from "@components/ui/slider";
import { useAuth } from "@clerk/react";
import {
  decodeStageCatalogAudio,
  listStageCatalogEntries,
  saveStageCatalogEntry,
  stageCatalogLinesToPrompterLines,
  type StageCatalogEntry,
} from "@lib/stage-catalog";
import { downloadBlob } from "@lib/download";
import { Check, Copy, FolderOpen, Headphones, ListMusic, Mic, MicOff, Share2, SkipForward, Square, UserRound, Volume2, VolumeX, X } from "lucide-react";
import { useLocation } from "wouter";

type StageQueueItem = {
  id: string;
  trackId: string;
  title: string;
  durationSeconds: number | null;
  hasSyncedLyrics: boolean;
  queuedByPeerId: string;
  queuedByRole: "host" | "partner";
  serverTimestamp: number;
};

type StageQueueSnapshot = {
  nowPlaying: StageQueueItem | null;
  upNext: StageQueueItem[];
};

// — Audio graph types & builder —

interface MonitorGraph {
  ctx:      AudioContext;
  stream:   MediaStream;
  source:   MediaStreamAudioSourceNode;
  gainNode: GainNode;
  hpFilter: BiquadFilterNode;
  warmthFilter: BiquadFilterNode;
  reverbSend: GainNode;
  convolver: ConvolverNode;
  reverbWet: GainNode;
  analyser: AnalyserNode;
}

function buildIR(ctx: AudioContext): AudioBuffer {
  const len = Math.floor(ctx.sampleRate * 2.2);
  const pre = Math.floor(ctx.sampleRate * 0.018);
  const ir = ctx.createBuffer(2, len, ctx.sampleRate);
  for (let ch = 0; ch < 2; ch++) {
    const d = ir.getChannelData(ch);
    for (let i = 0; i < len; i++) {
      if (i < pre) { d[i] = 0; continue; }
      const pos = i - pre;
      const tail = len - pre;
      const n = (Math.random() + Math.random() + Math.random()) / 3 * 2 - 1;
      d[i] = n * Math.pow(1 - pos / tail, 2.2);
    }
  }
  return ir;
}

function buildMonitorGraph(
  stream: MediaStream,
  micGain: number,
  hpfn: boolean,
  warmth: number,
  reverbSend: number,
): MonitorGraph {
  const ctx = new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });

  const source = ctx.createMediaStreamSource(stream);
  const gainNode = ctx.createGain();
  gainNode.gain.value = micGain;

  const hpFilter = ctx.createBiquadFilter();
  hpFilter.type = "highpass";
  hpFilter.frequency.value = hpfn ? 80 : 20;
  hpFilter.Q.value = 0.707;

  const warmthFilter = ctx.createBiquadFilter();
  warmthFilter.type = "lowshelf";
  warmthFilter.frequency.value = 250;
  warmthFilter.gain.value = warmth;

  const analyser = ctx.createAnalyser();
  analyser.fftSize = 1024;
  analyser.smoothingTimeConstant = 0.72;

  const reverbSendNode = ctx.createGain();
  reverbSendNode.gain.value = reverbSend;
  const convolver = ctx.createConvolver();
  convolver.buffer = buildIR(ctx);
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
const reverbWet      = ctx.createGain();
reverbWet.gain.value = 1;

// Dry path: direct sidetone with no inserted delay nodes
source.connect(gainNode);
gainNode.connect(ctx.destination);

// Parallel meter tap – does not delay the dry path
gainNode.connect(analyser);

// Reverb aux bus (optional ambience)
gainNode.connect(hpFilter);
hpFilter.connect(warmthFilter);
warmthFilter.connect(reverbSendNode);
reverbSendNode.connect(convolver);
convolver.connect(reverbWet);
reverbWet.connect(ctx.destination);

return { ctx, stream, source, gainNode, hpFilter, warmthFilter, reverbSend: reverbSendNode, convolver, reverbWet, analyser };
}

// ——— Monitor settings ———

const MONITOR_SETTINGS_KEY = "gk:main-stage:monitor:v1";

interface MonitorSettings {
  micGain: number;
  hpfOn:   boolean;
  warmth:  number;
  reverbSend: number;
}

function loadMonitorSettings(): MonitorSettings {
  try {
    const raw = localStorage.getItem(MONITOR_SETTINGS_KEY);
    if (raw) {
      const p = JSON.parse(raw) as Partial<MonitorSettings>;
      return {
        micGain:    typeof p.micGain    === "number" ? Math.max(0, Math.min(2, p.micGain)) : 1,
        hpfOn:      p.hpfOn !== false,
        warmth:     typeof p.warmth     === "number" ? Math.max(-3, Math.min(6, p.warmth)) : 0,
        reverbSend: typeof p.reverbSend === "number" ? Math.max(0, Math.min(1, p.reverbSend)) : 0.15,
      };
    }
  } catch { /* storage blocked */ }
  return { micGain: 1, hpfOn: true, warmth: 0, reverbSend: 0.15 };
}

// ——— Duet signaling types ———

type SignalType = "offer" | "answer" | "ice";

interface ArtistProfile {
  artistName: string;
  hometown:   string;
  avatarUrl:  string | null;
}

function isSignalType(v: unknown): v is SignalType {
  return v === "offer" || v === "answer" || v === "ice";
}

function newStageRoomCode(): string {
  return `GK-STAGE-${crypto.randomUUID().replace(/-/g, "").slice(0, 4).toUpperCase()}`;
}

function joinUrlFromCode(code: string): string {
  const basePath = import.meta.env.BASE_URL.endsWith("/")
    ? import.meta.env.BASE_URL
    : `${import.meta.env.BASE_URL}/`;
  return `${window.location.origin}${basePath}stage/duet/${encodeURIComponent(code)}`;
}

// ——— Partner floating HUD ———

interface PartnerHudProps {
  profile: ArtistProfile | null;
  partnerLevel: number; // 0-1 RMS from remote analyser
  volume: number; // 0-1 slider
  onVolumeChange: (v: number) => void;
  onDismissInvite: () => void;
  showInvite: boolean;
  inviteCode: string;
  onOpenInvite: () => void;
  connected: boolean;
}

function PartnerHud({
  profile, partnerLevel, volume, onVolumeChange,
  showInvite, inviteCode, onOpenInvite, onDismissInvite, connected,
}: PartnerHudProps) {
  const [volOpen, setVolOpen] = useState(false);
  const [copied, setCopied] = useState(false);
  const joinUrl = joinUrlFromCode(inviteCode);

  const copyLink = async () => {
    try {
      await navigator.clipboard.writeText(joinUrl);
      setCopied(true);
      setTimeout(() => setCopied(false), 2000);
    } catch { /* clipboard blocked */ }
  };

  const shareLink = async () => {
    try {
      await navigator.share({ title: "Join my GravelKing Duet Stage", url: joinUrl });
    } catch { /* share declined */ }
  };

  // Glow intensity driven by remote audio level
  const glowAlpha = (0.18 + partnerLevel * 0.72).toFixed(3);
  const glowBlur = `${10 + Math.round(partnerLevel * 22)}px`;
  const ringScale = 1 + partnerLevel * 0.18;
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
return (
  <>
    {/** — Floating partner bubble — top-right corner — */}
    <div className="fixed right-4 top-20 z-[55] flex flex-col items-end gap-2 sm:right-6 sm:top-20">
      {connected ? (
        /** Connected partner avatar with audio-reactive glow */
        <button
          type="button"
          onClick={() => setVolOpen(v => !v)}
          aria-label="Toggle partner volume"
          style={{
            boxShadow: `0 0 ${glowBlur} hsl(267 84% 70% / ${glowAlpha}), 0 0 ${glowBlur} hsl(267 84% 70% / ${Number(glowAlpha) * 0.5}.toFixed(3))`,
            transform: `scale(${ringsScale.toFixed(3)})`,
            transition: "transform 80ms linear, box-shadow 80ms linear",
          }}
          className="relative h-14 w-14 overflow-hidden rounded-full border-2 border-violet-400/70 bg-gradient-to-br from-violet-900/80 to-zinc-900 sm:h-16 sm:w-16"
        >
          {profile?.avatarUrl ? (
            <img src={profile.avatarUrl} alt={profile.artistName} className="h-full w-full object-cover" />
          ) : (
            <span className="flex h-full w-full items-center justify-center text-lg font-black text-violet-200">
              {profile?.artistName?.[0]??.toUpperCase() ?? <UserRound className="h-6 w-6" />}
            </span>
          )}
        </button>
      ) : (
        /** Live voice indicator */}
        {partnerLevel > 0.04 && (
          <span className="absolute bottom-1 right-1 h-2.5 w-2.5 rounded-full bg-emerald-400 shadow-[0_0_6px_rgba(34,197,94,0.9)]" />
        )}
      )}
    </div>
  ) : (
    /** No partner yet — glowing invite button */}
    <button
      type="button"
      onClick={onOpenInvite}
      className="flex h-14 w-14 flex-col items-center justify-center rounded-full border border-dashed border-violet-400/55 bg-violet-500/10 text-violet-300 shadow-[0_0_18px_hsl(267_84%_70%/0.22)] transition-all hover:border-violet-400/90 hover:bg-violet-500/18 hover:shadow-[0_0_26px_hsl(267_84%_70%/0.38)] sm:h-16 sm:w-16"
      aria-label="Invite duet partner"
    >
      <span className="text-xl font-black leading-none">+</span>
      <span className="mt-0.5 text-[8px] font-bold uppercase tracking-wide leading-none">Invite</span>
    </button>
  )}

  {/** Partner name tag */}
  <div className="text-right">
    <p className="text-[10px] font-bold text-violet-300/90">
      {connected ? (profile?.artistName ?? "Partner") : "No partner"}
    </p>
    {connected && (
      <p className="text-[9px] uppercase tracking-wide text-white/40">Live ·duet</p>
    )}
  </div>

  {/** Volume popover */}
  {volOpen && connected && (
    <div className="w-40 rounded-2xl border border-white/10 bg-zinc-950/96 p-3 shadow-[0_4px_32px_rgba(0,0,0,0.6)] backdrop-blur-md">
      <div className="mb-2 flex items-center justify-between">
        <span className="text-[10px] font-bold uppercase tracking-widest text-violet-300">Partner vol</span>
        <button
          type="button"
          onClick={() => setVolOpen(false)}
          className="rounded p-0.5 text-white/40 hover:text-white"
          aria-label="Close volume panel"
        >
          <X className="h-3 w-3" />
        </button>
      </div>
      <Slider
        min={0} max={1} step={0.01}
        value={{volume}}
        onValueChange={({[v]}) => { if (v !== undefined) onVolumeChange(v); }}
        aria-label="Partner monitor volume"
      />
      <p className="mt-1 text-right font-mono text-[9px] text-violet-300/70">{Math.round(volume * 100)} %</p>
    </div>
  )}
</div>

  {/** — Invite card modal — */}
  {showInvite && (
    <div
      className="fixed inset-0 z-[65] flex items-center justify-center bg-black/60 backdrop-blur-sm p-4"
      role="dialog"
      aria-modal="true"
      aria-label="Invite duet partner"
    >
      <div className="w-full max-w-sm rounded-3xl border border-white/10 bg-zinc-950/98 p-6 shadow-[0_8px_56px_rgba(0,0,0,0.75)]">
        {/** Header */}
        <div className="mb-5 flex items-start justify-between gap-3">
          <div>
            <p className="text-[10px] font-bold uppercase tracking-[0.22em] text-violet-400">Duet Stage</p>
            <h2 className="mt-0.5 text-lg font-black text-white">Invite a Partner</h2>
          </div>
          <button
            type="button"
            onClick={onDismissInvite}
            className="rounded-full p-1.5 text-white/40 hover:bg-white/8 hover:text-white"
            aria-label="Close invite panel"
          >
            <X className="h-4 w-4" />
          </button>
        </div>

        {/** Room code */}
        <div className="mb-4 rounded-2xl border border-violet-400/30 bg-violet-500/10 px-4 py-3 text-center">
          <p className="mb-1 text-[9px] font-bold uppercase tracking-[0.2em] text-violet-400/80">Stage Room Code</p>
          <p className="font-mono text-2xl font-black tracking-widest text-violet-200">{inviteCode}</p>
        </div>

        {/** Copy link */}
        <button
          type="button"
          onClick={copyLink}
        >
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```

        className=[
          "mb-3 flex w-full items-center justify-center gap-2 rounded-xl border py-3 text-[11px] font-bold uppercase tracking-wider transition-all",
          copied
            ? "border-emerald-400/60 bg-emerald-500/15 text-emerald-300"
            : "border-violet-400/40 bg-violet-500/12 text-violet-200 hover:border-violet-400/70 hover:bg-violet-500/20",
          ].join(" ")
        )
      >
      {copied ? <Check className="h-3.5 w-3.5" /> : <Copy className="h-3.5 w-3.5" />}
      {copied ? "Copied!" : "Copy Invite Link"}
    </button>

    /* Native share */
    {"share" in navigator && (
      <button
        type="button"
        onClick={shareLink}
        className="mb-5 flex w-full items-center justify-center gap-2 rounded-xl border border-white/12 py-3 text-[11px] font-bold uppercase tracking-wider text-white/60 transition-all hover:border-white/25 hover:text-white"
      >
        <Share2 className="h-3.5 w-3.5" />
        Share via...
      </button>
    )}

    /* Waiting indicator */
    <div className="flex items-center justify-center gap-2">
      <span className="h-2 w-2 animate-pulse rounded-full bg-violet-400" />
      <p className="text-[10px] font-medium text-white/50">Waiting for partner to join...</p>
    </div>
  </div>
)}
</>
);
}

// — Main Stage page
export default function MainStage() {
  const { getToken } = useAuth();

  // — Monitor state
  const [monitorOn, setMonitorOn] = useState(false);
  const [fxOpen, setFxOpen] = useState(false);
  const [error, setError] = useState<string | null>(null);
  const [level, setLevel] = useState(0);
  const [settings, setSettings] = useState<MonitorSettings>(loadMonitorSettings);
  const [bluetoothOutput, setBluetoothOutput] = useState(false);
  const [isPlaying, setIsPlaying] = useState(false);
  const [masterVolume, setMasterVolume] = useState(0.8);
  const [playbackPosition, setPlaybackPosition] = useState(0);
  const [durationSec, setDurationSec] = useState(0);
  const [trackName, setTrackName] = useState("");
  const [, setLocation] = useLocation();
  const [stageLyrics, setStageLyrics] = useState<ReturnType<typeof stageCatalogLinesToPrompterLines>>([]);
  const [vaultOpen, setVaultOpen] = useState(false);
  const [vaultEntries, setVaultEntries] = useState<StageCatalogEntry[]>([]);
  const [vaultLoading, setVaultLoading] = useState(false);
  const [vaultError, setVaultError] = useState<string | null>(null);
  interface LibraryTrack { id: string; title: string; artistName?: string; createdAt?: string; }
  const [serverTracks, setServerTracks] = useState<LibraryTrack[]>([]);
  const [isRecording, setIsRecording] = useState(false);
  const [takeBlob, setTakeBlob] = useState<Blob | null>(null);
  const [savingTake, setSavingTake] = useState(false);
  const [reviewUrl, setReviewUrl] = useState<string | null>(null);
  const [takeSavedMessage, setTakeSavedMessage] = useState<string | null>(null);

  const graphRef = useRef<MonitorGraph | null>(null);
  const rafRef = useRef<number>(0);
  const canvasRef = useRef<HTMLCanvasElement>(null);
  const playbackContextRef = useRef<AudioContext | null>(null);
  const backingBufferRef = useRef<AudioBuffer | null>(null);
  const playbackSourceRef = useRef<AudioBufferSourceNode | null>(null);
  const playbackGainRef = useRef<GainNode | null>(null);
  const playbackStartContextRef = useRef<number | null>(null);
  const playbackOffsetRef = useRef(0);
  const playbackRafRef = useRef<number>(0);
  const takeRecorderRef = useRef<MediaRecorder | null>(null);
  const takeChunksRef = useRef<Blob[]>([]);
  const reviewAudioRef = useRef<HTMLAudioElement | null>(null);

  // — Duet state
  const [inviteCode, setInviteCode] = useState(newStageRoomCode);
  const [showInvite, setShowInvite] = useState(false);
  const [peerConnected, setPeerConnected] = useState(false);
  const [partnerProfile, setPartnerProfile] = useState<ArtistProfile | null>(null);
  const [partnerLevel, setPartnerLevel] = useState(0);
  const [partnerVolume, setPartnerVolume] = useState(0.8);
  const [roomRole, setRoomRole] = useState<"host" | "partner" | null>(null);
  const [stageQueue, setStageQueue] = useState<StageQueueSnapshot>({ nowPlaying: null, upNext: [] });
  const [queueBusy, setQueueBusy] = useState(false);

  const peerIdRef = useRef<string>(crypto.randomUUID());
  const remotePeerRef = useRef<string | null>(null);
  const roleRef = useRef<"host" | "partner" | null>(null);
  const sseRef = useRef<EventSource | null>(null);
  const pcRef = useRef<RTCPeerConnection | null>(null);
  const pendingCandidatesRef = useRef<RTCIceCandidateInit[]>([]);
  const partnerAnalyserRef = useRef<AnalyserNode | null>(null);
  const partnerGainRef = useRef<GainNode | null>(null);
  const partnerAudioCtxRef = useRef<AudioContext | null>(null);
  const partnerRafRef = useRef<number>(0);
  const peerMidiStreamRef = useRef<MediaStream | null>(null);
  const dataChannelRef = useRef<RTCDataChannel | null>(null);
  const roomCodeRef = useRef(inviteCode);
  const activeQueueItemRef = useRef<string | null>(null);

  useEffect(() => {
    if (!takeBlob) {
      setReviewUrl(null);
      return;
    }
    const url = URL.createObjectURL(takeBlob);
    setReviewUrl(url);
  }, [takeBlob]);
}
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
    return () => URL.revokeObjectURL(url);
  }, [takeBlob]);

// — Helpers —
const authHeaders = useCallback(async (): Promise<Record<string, string>> => {
  const token = await getToken();
  return {
    "Content-Type": "application/json",
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  };
}, [getToken]);

const sendSignal = useCallback(async (type: SignalType, payload: unknown) => {
  const to = remotePeerRef.current;
  if (!to) return;
  await fetch(`/api/duet/room/${encodeURIComponent(roomCodeRef.current)}/signal`, {
    method: "POST",
    headers: await authHeaders(),
    credentials: "include",
    body: JSON.stringify({ fromPeerId: peerIdRef.current, toPeerId: to, type, payload }),
  });
}, [authHeaders]);

const sendControl = useCallback((message: { kind: "review-start"; targetHostTime: number }) => {
  const channel = dataChannelRef.current;
  if (channel?.readyState === "open") channel.send(JSON.stringify(message));
}, []);

const attachDataChannel = useCallback((channel: RTCDataChannel) => {
  dataChannelRef.current = channel;
  channel.onmessage = (event) => {
    let message: { kind?: string; targetHostTime?: number };
    try { message = JSON.parse(String(event.data)) as typeof message; } catch { return; }
    if (message.kind !== "review-start" || typeof message.targetHostTime !== "number") return;
    const wait = Math.max(0, message.targetHostTime - performance.now());
    window.setTimeout(() => {
      const audio = reviewAudioRef.current;
      if (!audio) return;
      audio.currentTime = 0;
      void audio.play().catch(() => {});
    }, wait);
  };
}, []);

const ensurePeerMic = useCallback(async (): Promise<MediaStream | null> => {
  if (peerMicStreamRef.current) return peerMicStreamRef.current;
  if (graphRef.current?.stream) {
    peerMicStreamRef.current = graphRef.current.stream;
    return peerMicStreamRef.current;
  }
  if (!navigator.mediaDevices?.getUserMedia) return null;
  try {
    peerMicStreamRef.current = await navigator.mediaDevices.getUserMedia({
      audio: {
        echoCancellation: false,
        noiseSuppression: false,
        autoGainControl: false,
        latency: 0,
        channelCount: 1,
        sampleRate: 44100,
      } as MediaTrackConstraints,
    });
    return peerMicStreamRef.current;
  } catch {
    return null;
  }
}, []);

// — Partner audio: attach remote mic stream into an AnalyserNode —
const attachPartnerStream = useCallback((stream: MediaStream) => {
  const ctx = partnerAudioCtxRef.current
  ?? new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });
  partnerAudioCtxRef.current = ctx;
  if (ctx.state === "suspended") void ctx.resume();

  const gainNode = ctx.createGain();
  gainNode.gain.value = partnerVolume;
  const analyser = ctx.createAnalyser();
  analyser.fftSize = 1024;
  analyser.smoothingTimeConstant = 0.72;

  const source = ctx.createMediaStreamSource(stream);
  source.connect(gainNode);
  gainNode.connect(analyser);
  gainNode.connect(ctx.destination);

  partnerGainRef.current = gainNode;
  partnerAnalyserRef.current = analyser;

  // Level meter RAF for partner glow
  const buf = new Uint8Array(analyser.frequencyBinCount);
  const tick = () => {
    partnerRafRef.current = requestAnimationFrame(tick);
    analyser.getByteFrequencyData(buf);
    const start = Math.floor(buf.length * 0.04);
    const end = Math.max(start + 1, Math.floor(buf.length * 0.72));
    let sum = 0;
    for (let i = start; i < end; i++) sum += buf[i] ?? 0;
    setPartnerLevel(Math.min(1, (sum / (end - start)) / 255 * 1.85));
  };
  cancelAnimationFrame(partnerRafRef.current);
  tick();
}, [partnerVolume]);

// Sync partner volume to gainNode when slider changes
useEffect(() => {
  const g = partnerGainRef.current;
  const ctx = partnerAudioCtxRef.current;
  if (g && ctx) g.gain.linearRampToValueAtTime(partnerVolume, ctx.currentTime + 0.02);
}, [partnerVolume]);

// Keep the backing track's master gain live while it is playing.
useEffect(() => {
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
const gain = playbackGainRef.current;
const ctx = playbackContextRef.current;
if (gain && ctx) {
  gain.gain.linearRampToValueAtTime(masterVolume, ctx.currentTime + 0.02);
}
}, [masterVolume]);

// — WebRTC peer connection —
const handleSignal = useCallback(async (msg: { fromPeerId?: string; type?: unknown; payload?: unknown }) => {
  if (!msg.fromPeerId || !isSignalType(msg.type)) return;
  let pc = pcRef.current;
  if (!pc) {
    remotePeerRef.current = msg.fromPeerId;
    pc = new RTCPeerConnection({ iceServers: [{ urls: "stun:stun.l.google.com:19302" }] });
    pcRef.current = pc;
    const mic = await ensurePeerMic();
    if (mic) for (const track of mic.getTracks()) pc.addTrack(track, mic);
    pc.ontrack = (event) => {
      const stream = event.streams[0] ?? new MediaStream([event.track]);
      attachPartnerStream(stream);
    };
    pc.onconnectionstatechange = () => {
      if (pc!.connectionState === "connected") setPeerConnected(true);
      if (pc!.connectionState === "failed" || pc!.connectionState === "closed") setPeerConnected(false);
    };
    pc.onicecandidate = (event) => {
      if (event.candidate) void sendSignal("ice", event.candidate.toJSON());
    };
    pc.ondatachannel = (event) => attachDataChannel(event.channel);
  }
  if (msg.type === "offer") {
    await pc.setRemoteDescription(msg.payload as RTCSessionDescriptionInit);
    for (const c of pendingCandidatesRef.current.splice(0)) await pc.addIceCandidate(c);
    const answer = await pc.createAnswer();
    await pc.setLocalDescription(answer);
    await sendSignal("answer", answer);
  } else if (msg.type === "answer") {
    await pc.setRemoteDescription(msg.payload as RTCSessionDescriptionInit);
    for (const c of pendingCandidatesRef.current.splice(0)) await pc.addIceCandidate(c);
  } else if (msg.type === "ice" && msg.payload) {
    if (!pc.remoteDescription) { pendingCandidatesRef.current.push(msg.payload as RTCIceCandidateInit); return; }
    await pc.addIceCandidate(msg.payload as RTCIceCandidateInit);
  }
}, [attachDataChannel, attachPartnerStream, ensurePeerMic, sendSignal]);

const initiateOffer = useCallback(async (remotePeerId: string) => {
  if (pcRef.current) return;
  remotePeerRef.current = remotePeerId;
  const pc = new RTCPeerConnection({ iceServers: [{ urls: "stun:stun.l.google.com:19302" }] });
  pcRef.current = pc;
  const mic = await ensurePeerMic();
  if (mic) for (const track of mic.getTracks()) pc.addTrack(track, mic);
  pc.ontrack = (event) => {
    const stream = event.streams[0] ?? new MediaStream([event.track]);
    attachPartnerStream(stream);
  };
  pc.onconnectionstatechange = () => {
    if (pc.connectionState === "connected") setPeerConnected(true);
    if (pc.connectionState === "failed" || pc.connectionState === "closed") setPeerConnected(false);
  };
  pc.onicecandidate = (event) => {
    if (event.candidate) void sendSignal("ice", event.candidate.toJSON());
  };
  const channel = pc.createDataChannel("duet-stage", { ordered: true });
  attachDataChannel(channel);
  const offer = await pc.createOffer();
  await pc.setLocalDescription(offer);
  await sendSignal("offer", offer);
}, [attachDataChannel, attachPartnerStream, ensurePeerMic, sendSignal]);

// — Join the room and open the SSE stream —
const joinRoom = useCallback(async (code: string) => {
  roomCodeRef.current = code;
  const headers = await authHeaders();
  const res = await fetch(`/api/duet/room/${encodeURIComponent(code)}/join`, {
    method: "POST", headers, credentials: "include",
    body: JSON.stringify({ peerId: peerIdRef.current }),
  });
  if (!res.ok) return;
  const data = await res.json() as {
    role?: "host" | "partner";
    otherPeerId?: string | null;
    otherPeerProfile?: ArtistProfile | null;
    queue?: StageQueueSnapshot;
  };
  roleRef.current = data.role ?? null;
  setRoomRole(data.role ?? null);
  if (data.queue) setStageQueue(data.queue);
  if (data.otherPeerId) {
    remotePeerRef.current = data.otherPeerId;
    setPartnerProfile(data.otherPeerProfile ?? null);
    setPeerConnected(true);
    if (data.role === "host") void initiateOffer(data.otherPeerId);
  }
}, [initiateOffer, setPartnerProfile, setPeerConnected, setRoomRole, setStageQueue]);

const sse = new EventSource(
  `/api/duet/room/${encodeURIComponent(code)}/events?peerId=${encodeURIComponent(peerIdRef.current)}`,
);
sseRef.current = sse;

sse.addEventListener("peer-joined", (e) => {
  const joined = JSON.parse(e as MessageEvent).data as { peerId?: string; profile?: ArtistProfile | null };
  if (!joined.peerId) return;
  remotePeerRef.current = joined.peerId;
  setPartnerProfile(joined.profile ?? null);
  setPeerConnected(true);
  setShowInvite(false);
  if (roleRef.current === "host") void initiateOffer(joined.peerId);
});
sse.addEventListener("signal", (e) => {
  void handleSignal(JSON.parse(e as MessageEvent).data as { fromPeerId?: string; type?: unknown; payload?: unknown });
});
sse.addEventListener("queue-updated", (e) => {
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
const next = JSON.parse(e as MessageEvent).data) as StageQueueSnapshot;
setStageQueue({
  nowPlaying: next.nowPlaying ?? null,
  upNext: Array.isArray(next.upNext) ? next.upNext : [],
});
});
sse.addEventListener("peer-left", () => {
  setPeerConnected(false);
  setPartnerProfile(null);
  setPartnerLevel(0);
  pcRef.current?.close();
  pcRef.current = null;
  remotePeerRef.current = null;
  cancelAnimationFrame(partnerRafRef.current);
});
}, [authHeaders, handleSignal, initiateOffer]);

// On mount: if a ?room= param exists, join that room
useEffect(() => {
  const pathRoom = window.location.pathname.match(/\/stage\/duet\/([^\/]+)/?.[1];
  const fromUrl = new URLSearchParams(window.location.search).get("room")
  ?? (pathRoom ? decodeURIComponent(pathRoom) : null);
  if (fromUrl) {
    const clean = fromUrl.trim().replace(/[^a-zA-Z0-9_-]/g, "").slice(0, 80);
    setInviteCode(clean);
    void joinRoom(clean);
  }
  return () => {
    sseRef.current?.close();
    pcRef.current?.close();
    cancelAnimationFrame(partnerRafRef.current);
  };
// eslint-disable-next-line react-hooks/exhaustive-deps
}, [1]);

const openInvite = useCallback(async () => {
  setShowInvite(true);
  // Lazily join the room when the host first opens the invite card
  if (!sseRef.current) await joinRoom(inviteCode);
}, [inviteCode, joinRoom]);

// — Monitor settings persistence + live AudioParam updates —————
useEffect(() => {
  try { localStorage.setItem(MONITOR_SETTINGS_KEY, JSON.stringify(settings)); } catch { /* blocked */ }
  const g = graphRef.current;
  if (!g) return;
  g.gainNode.gain.linearRampToValueAtTime(settings.micGain, g.ctx.currentTime + 0.01);
  g.hpFilter.frequency.linearRampToValueAtTime(settings.hpfOn ? 80 : 20, g.ctx.currentTime + 0.02);
  g.warmthFilter.gain.linearRampToValueAtTime(settings.warmth, g.ctx.currentTime + 0.02);
  g.reverbSend.gain.linearRampToValueAtTime(settings.reverbSend, g.ctx.currentTime + 0.02);
}, [settings]);

// — Monitor start / stop —————
const stopMonitor = useCallback(() => {
  cancelAnimationFrame(rafRef.current);
  const g = graphRef.current;
  if (g) {
    try { g.source.disconnect(); } catch { /* ok */ }
    if (peerMicStreamRef.current !== g.stream) g.stream.getTracks().forEach(t => t.stop());
    void g.ctx.close();
    graphRef.current = null;
  }
  setMonitorOn(false);
  setLevel(0);
}, [1]);

const startMonitor = useCallback(async (): Promise<MediaStream | null> => {
  stopMonitor();
  setError(null);
  if (!navigator.mediaDevices?.getUserMedia) {
    setError("Microphone capture is not supported in this browser.");
    return null;
  }
  try {
    const stream = peerMicStreamRef.current ?? await navigator.mediaDevices.getUserMedia({
      audio: {
        echoCancellation: false,
        noiseSuppression: false,
        autoGainControl: false,
        latency: 0,
        channelCount: 1,
        sampleRate: 44100,
      } as MediaTrackConstraints,
    });
    peerMicStreamRef.current = stream;
    const g = buildMonitorGraph(stream, settings.micGain, settings.hpfOn, settings.warmth, settings.reverbSend);
    if (g.ctx.state === "suspended") await g.ctx.resume();
    graphRef.current = g;
    setMonitorOn(true);
    setExOpen(true);
    if (typeof navigator.mediaDevices.enumerateDevices === "function") {
      const devices = await navigator.mediaDevices.enumerateDevices();
      setBluetoothOutput(devices
        .filter((device) => device.kind === "audiooutput")
        .some((device) => /bluetooth|airpods|wireless|buds|jabra|bose|sony\s*(wh|wf)|beats|galaxy\s*buds|pixel\s*buds/i.test(device.label)));
    }

    // Level meter + waveform RAF
    const timeDomain = new Uint8Array(g analyser.frequencyBinCount);
    const draw = () => {
      rafRef.current = requestAnimationFrame(draw);
      g analyser.getByteTimeDomainData(timeDomain);
      let sum = 0;
      for (let i = 0; i < timeDomain.length; i++) {
        const v = (timeDomain[i] - 128) / 128;
        sum += v * v;
      }
      setLevel(Math.min(1, Math.sqrt(sum / timeDomain.length) * 6));

      const canvas = canvasRef.current;
      if (!canvas) return;
      const ctx2d = canvas.getContext("2d");
      if (!ctx2d) return;
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
const W = canvas.width, H = canvas.height;
ctx2d.clearRect(0, 0, W, H);
ctx2d.strokeStyle = "#2c5e";
ctx2d.lineWidth = 1.5;
ctx2d.beginPath();
const slice = W / timeDomain.length;
for (let i = 0; i < timeDomain.length; i++) {
  const y = ((timeDomain[i] - 128) / 128) * (H / 2) + H / 2;
  i === 0 ? ctx2d.moveTo(0, y) : ctx2d.lineTo(i * slice, y);
}
ctx2d.stroke();
};
draw();

// Also add mic tracks to any already-open peer connection for the partner
const pc = pcRef.current;
if (pc) {
  for (const track of stream.getTracks()) {
    const senders = pc.getSenders();
    if (!senders.some(s => s.track === track)) pc.addTrack(track, stream);
  }
}
return stream;
} catch (e: unknown) {
  const name = (e as { name?: string })?.name;
  const message = (e as { message?: string })?.message ?? "Unknown error";
  setError(
    name === "NotAllowedError"
      ? "Microphone access denied – allow it in your browser settings."
      : message,
  );
  return null;
}
}, [settings, stopMonitor]);

const toggleMonitor = useCallback(() => {
  if (monitorOn) { stopMonitor(); setFxOpen(false); }
  else void startMonitor();
}, [monitorOn, startMonitor, stopMonitor]);

// — Pristine local take buffer —————
const stopTake = useCallback(() => {
  const recorder = takeRecorderRef.current;
  if (recorder && recorder.state !== "inactive") recorder.stop();
  takeRecorderRef.current = null;
  setIsRecording(false);
}, []);

const startTake = useCallback(async () => {
  if (typeof MediaRecorder === "undefined") {
    setError("This browser cannot record an in-memory vocal take.");
    return;
  }
  const stream = peerMicStreamRef.current ?? await startMonitor();
  if (!stream) return;
  stopTake();
  takeChunksRef.current = [];
  const mimeType = ["audio/webm;codecs=opus", "audio/webm", "audio/mp4"]
    .find((candidate) => MediaRecorder.isTypeSupported(candidate)) ?? "";
  try {
    const recorder = mimeType ? new MediaRecorder(stream, { mimeType }) : new MediaRecorder(stream);
    recorder.ondataavailable = (event) => {
      if (event.data.size > 0) takeChunksRef.current.push(event.data);
    };
    recorder.onstop = () => {
      const blob = new Blob(takeChunksRef.current, { type: recorder.mimeType || "audio/webm" });
      if (blob.size > 0) setTakeBlob(blob);
    };
    recorder.start(750);
    takeRecorderRef.current = recorder;
    setTakeBlob(null);
    setIsRecording(true);
  } catch (cause) {
    setError(cause instanceof Error ? cause.message : "Could not start the local take recorder.");
  }
}, [startMonitor, stopTake]);

const toggleTake = useCallback(() => {
  if (isRecording) stopTake();
  else void startTake();
}, [isRecording, startTake, stopTake]);

// — AudioContext-backed backing track clock —————
const stopPlayback = useCallback(() => {
  cancelAnimationFrame(playbackRafRef.current);
  try { playbackSourceRef.current?.stop(); } catch { /* already ended */ }
  playbackSourceRef.current = null;
  playbackGainRef.current = null;
  playbackStartContextRef.current = null;
  setIsPlaying(false);
}, []);

const startPlayback = useCallback(() => {
  const buffer = backingBufferRef.current;
  if (!buffer) {
    setError("Load a backing track before pressing play.");
    return;
  }
  const ctx = playbackContextRef.current ?? new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });
  playbackContextRef.current = ctx;
  if (ctx.state === "suspended") void ctx.resume();
  try { playbackSourceRef.current?.stop(); } catch { /* no active source */ }
  const source = ctx.createBufferSource();
  source.buffer = buffer;
  const masterGain = ctx.createGain();
  masterGain.gain.value = masterVolume;
  const analyser = ctx.createAnalyser();
  analyser.fftSize = 1024;
  source.connect(masterGain);
  masterGain.connect(analyser);
  analyser.connect(ctx.destination);
  const offset = Math.max(0, Math.min(playbackOffsetRef.current, Math.max(0, buffer.duration - 0.01)));
  const startAt = ctx.currentTime + 0.03;
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```

    source.start(startAt, offset);
    playbackSourceRef.current = source;
    playbackGainRef.current = masterGain;
    playbackStartContextRef.current = startAt - offset;
    setIsPlaying(true);
    source.onended = () => {
      if (playbackSourceRef.current !== source) return;
      playbackSourceRef.current = null;
      playbackGainRef.current = null;
      playbackStartContextRef.current = null;
      playbackOffsetRef.current = 0;
      setPlaybackPosition(buffer.duration);
      setIsPlaying(false);
      stopTake();
    };
  }, [masterVolume, stopTake]);

const togglePlayback = useCallback(() => {
  if (isPlaying) {
    const ctx = playbackContextRef.current;
    const startAt = playbackStartContextRef.current;
    if (ctx && startAt !== null) {
      playbackOffsetRef.current = Math.max(0, Math.min(durationSec, ctx.currentTime - startAt));
    }
    stopPlayback();
    return;
  }
  startPlayback();
}, [durationSec, isPlaying, startPlayback, stopPlayback]);

const seekPlayback = useCallback(({fraction: number}) => {
  const next = Math.max(0, Math.min(1, fraction)) * durationSec;
  const wasPlaying = isPlaying;
  stopPlayback();
  playbackOffsetRef.current = next;
  setPlaybackPosition(next);
  if (wasPlaying) window.setTimeout(startPlayback, 0);
}, [durationSec, isPlaying, startPlayback, stopPlayback]);

useEffect(() => {
  if (!isPlaying) return;
  const tick = () => {
    const ctx = playbackContextRef.current;
    const startAt = playbackStartContextRef.current;
    if (ctx && startAt !== null) {
      setPlaybackPosition(Math.max(0, Math.min(durationSec, ctx.currentTime - startAt)));
    }
    playbackRafRef.current = requestAnimationFrame(tick);
  };
  playbackRafRef.current = requestAnimationFrame(tick);
  return () => cancelAnimationFrame(playbackRafRef.current);
}, [durationSec, isPlaying]);

const openVault = useCallback(async () => {
  setVaultOpen(true);
  setVaultLoading(true);
  setVaultError(null);
  try {
    const [localEntries, serverResp] = await Promise.allSettled([
      listStageCatalogEntries(),
      fetch("/api/library", { credentials: "include" }).then((r) => r.ok ? r.json() as Promise<{ tracks?: LibraryTrack[] }> : { tracks: [] }),
    ]);
    setVaultEntries(localEntries.status === "fulfilled" ? localEntries.value : []);
    setServerTracks(serverResp.status === "fulfilled" ? (serverResp.value.tracks ?? []) : []);
    if (localEntries.status === "rejected" && serverResp.status === "rejected") {
      setVaultError("Could not load Stage Vault.");
    }
  } finally {
    setVaultLoading(false);
  }
}, []);

const selectServerTrack = useCallback(async (track: LibraryTrack) => {
  setVaultError(null);
  try {
    const context = playbackContextRef.current
    ?? new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });
    playbackContextRef.current = context;
    stopPlayback();
    const resp = await fetch(`/api/tracks/${track.id}/stream`, { credentials: "include" });
    if (!resp.ok) throw new Error("Could not stream track (${resp.status})");
    const buffer = await context.decodeAudioData(await resp.arrayBuffer());
    backingBufferRef.current = buffer;
    playbackOffsetRef.current = 0;
    setPlaybackPosition(0);
    setDurationSec(buffer.duration);
    setTrackName(track.title);
    setStageLyrics([]);
    setTakeBlob(null);
    setError(null);
    setVaultOpen(false);
  } catch (cause) {
    setVaultError(cause instanceof Error ? cause.message : "Could not load track from Library.");
  }
}, [stopPlayback]);

const selectVaultSong = useCallback(async (entry: StageCatalogEntry) => {
  try {
    const context = playbackContextRef.current
    ?? new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });
    playbackContextRef.current = context;
    stopPlayback();
    const buffer = await decodeStageCatalogAudio(context, entry.instrumental);
    backingBufferRef.current = buffer;
    playbackOffsetRef.current = 0;
    setPlaybackPosition(0);
    setDurationSec(buffer.duration);
    setTrackName(entry.title);
    // Tracks without synced lyrics are valid – show a placeholder instead of blocking load.
    setStageLyrics(
      entry.lines.length > 0
      ? stageCatalogLinesToPrompterLines(entry.lines)
      : [],
    );
  }
}, [stopPlayback]);
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
    );
    setTakeBlob(null);
    setError(null);
    setVaultOpen(false);
  } catch (cause) {
    setVaultError(cause instanceof Error ? cause.message : "Could not load this song from Stage Vault.");
  }
}, [stopPlayback]);

// Load a locally available queue winner on the device that submitted it.
// Other peers still receive the ordered deck metadata from the server.
useEffect(() => {
  const active = stageQueue.nowPlaying;
  if (!active || activeQueueItemRef.current === active.id) return;
  activeQueueItemRef.current = active.id;
  if (active.queuedByPeerId !== peerIdRef.current) {
    // Never leave the previous winner playing after the Host advances.
    stopPlayback();
    backingBufferRef.current = null;
    playbackOffsetRef.current = 0;
    setPlaybackPosition(0);
    setDurationSec(active.durationSeconds ?? 0);
    setTrackName(active.title);
    setStageLyrics([]);
    return;
  }
  let cancelled = false;
  void listStageCatalogEntries().then((entries) => {
    const entry = entries.find((candidate) => candidate.id === active.trackId);
    if (!cancelled && entry) void selectVaultSong(entry);
  });
  return () => { cancelled = true; };
}, [selectVaultSong, stageQueue.nowPlaying, stopPlayback]);

const queueVaultSong = useCallback(async (entry: StageCatalogEntry) => {
  if (!roleRef.current) {
    await selectVaultSong(entry);
    return;
  }
  setQueueBusy(true);
  setVaultError(null);
  try {
    const response = await fetch(`/api/duet/room/${encodeURIComponent(roomCodeRef.current)}/queue`, {
      method: "POST",
      headers: await authHeaders(),
      credentials: "include",
      body: JSON.stringify({
        peerId: peerIdRef.current,
        trackId: entry.id,
        title: entry.title,
        durationSeconds: entry.durationSeconds ?? null,
        hasSyncedLyrics: entry.lines.length > 0,
      }),
    });
    const body = await response.json().catch(() => ({})) as {
      error?: string;
      queue?: StageQueueSnapshot;
    };
    if (!response.ok) throw new Error(body.error || "Could not queue this song.");
    if (body.queue) setStageQueue(body.queue);
    setVaultOpen(false);
  } catch (cause) {
    setVaultError(cause instanceof Error ? cause.message : "Could not queue this song.");
  } finally {
    setQueueBusy(false);
  }
}, [authHeaders, selectVaultSong]);

const advanceQueue = useCallback(async () => {
  if (roleRef.current !== "host") return;
  setQueueBusy(true);
  try {
    const response = await fetch(`/api/duet/room/${encodeURIComponent(roomCodeRef.current)}/queue/advance`, {
      method: "POST",
      headers: await authHeaders(),
      credentials: "include",
      body: JSON.stringify({ peerId: peerIdRef.current }),
    });
    const body = await response.json().catch(() => ({})) as {
      error?: string;
      queue?: StageQueueSnapshot;
    };
    if (!response.ok) throw new Error(body.error || "Could not advance the queue.");
    if (body.queue) setStageQueue(body.queue);
  } catch (cause) {
    setError(cause instanceof Error ? cause.message : "Could not advance the queue.");
  } finally {
    setQueueBusy(false);
  }
}, [authHeaders]);

// Arriving from Track & Lyric Prep with ?song=<vault id>: arm that project immediately.
useEffect(() => {
  const songId = new URLSearchParams(window.location.search).get("song");
  if (!songId) return;
  let cancelled = false;
  void listStageCatalogEntries().then((entries) => {
    const match = entries.find((entry) => entry.id === songId);
    if (!cancelled && match) void selectVaultSong(match);
  }).catch((cause) => {
    if (!cancelled) setVaultError(cause instanceof Error ? cause.message : "Could not load the prepared song.");
  });
  return () => { cancelled = true; };
}, [songId]);

// — Review actions: local mixdown, synchronized playback, or RAM flush —
const playReview = useCallback(() => {
  if (!reviewUrl) return;
  const targetHostTime = performance.now() + 500;
  if (roleRef.current === "host") sendControl({ kind: "review-start", targetHostTime });
  window.setTimeout(() => {
    const audio = reviewAudioRef.current;
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
    if (!audio) return;
    audio.currentTime = 0;
    void audio.play().catch(() => {});
  }, Math.max(0, targetHostTime - performance.now()));
}, [reviewUrl, sendControl]);

const saveToVault = useCallback(async () => {
  if (!takeBlob) return;
  setSavingTake(true);
  try {
    const decodeContext = playbackContextRef.current ?? new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });
    const vocal = await decodeContext.decodeAudioData(await takeBlob.arrayBuffer());
    const sampleRate = 44100;
    const backing = backingBufferRef.current;
    const frameCount = Math.max(
      1,
      Math.ceil((backing?.duration ?? 0) * sampleRate),
      Math.ceil(vocal.duration * sampleRate),
    );
    const offline = new OfflineAudioContext(2, frameCount, sampleRate);
    if (backing) {
      const backingSource = offline.createBufferSource();
      backingSource.buffer = backing;
      const backingGain = offline.createGain();
      backingGain.gain.value = 0.82;
      backingSource.connect(backingGain).connect(offline.destination);
      backingSource.start(0);
    }
    const vocalSource = offline.createBufferSource();
    vocalSource.buffer = vocal;
    const vocalGain = offline.createGain();
    vocalGain.gain.value = 0.95;
    vocalSource.connect(vocalGain).connect(offline.destination);
    vocalSource.start(0);
    const rendered = await offline.startRendering();
    const wav = encodePcm16Wav(
      [rendered.getChannelData(0), rendered.getChannelData(1)],
      rendered.sampleRate,
    );
    const createdAt = new Date();
    const baseTitle = trackName || "Main Stage";
    const existing = await listStageCatalogEntries();
    const takeNumber = existing.filter((entry) => entry.title.startsWith(`${baseTitle} - Take `)).length + 1;
    const title = `${baseTitle} - Take ${takeNumber} - ${createdAt.toLocaleString()}`;
    await saveStageCatalogEntry({
      id: crypto.randomUUID(),
      createdAt: createdAt.toISOString(),
      title,
      instrumental: wav,
      guideVocal: null,
      lines: stagelyrics.map((line, lineIndex) => ({
        lineIndex,
        contextTime: 0,
        timeSeconds: line.startTimeMs / 1000,
        text: line.text,
      })),
      durationSeconds: rendered.duration,
    });
    setVaultEntries(await listStageCatalogEntries());
    setTakeSavedMessage(`Saved ${baseTitle} - Take ${takeNumber} to your Library.`);
    setTakeBlob(null);
    setError(null);
  } catch (cause) {
    setError(cause instanceof Error ? cause.message : "Could not save this take to Vault.");
  } finally {
    setSavingTake(false);
  }
}, [stagelyrics, takeBlob, trackName]);

const downloadTake = useCallback(() => {
  if (!takeBlob) return;
  const safeName = (trackName || "main-stage-take")
    .replace(/[^a-z0-9]+/gi, "-")
    .replace(/~/g, "")
    .toLowerCase() || "main-stage-take";
  downloadBlob(takeBlob, `${safeName}.webm`);
}, [takeBlob, trackName]);

const retake = useCallback(() => {
  stopPlayback();
  stopTake();
  takeChunksRef.current = [];
  setTakeBlob(null);
  setPlaybackPosition(0);
  playbackOffsetRef.current = 0;
}, [stopPlayback, stopTake]);

useEffect(() => () => stopMonitor(), [stopMonitor]);

// Unmount cleanup: stop all audio so nothing keeps playing after navigation.
useEffect(() => () => {
  cancelAnimationFrame(playbackRafRef.current);
  try { playbackSourceRef.current?.stop(); } catch { /* already ended */ }
  void playbackContextRef.current?.close().catch(() => {});
  void partnerAudioCtxRef.current?.close().catch(() => {});
  playbackGainRef.current = null;
}, []);

const goToTrackPrep = useCallback(() => {
  stopMonitor();
  setLocation("/studio/track-prep");
}, [setLocation, stopMonitor]);

// Primary CTA card shown on the jumbotron while nothing is loaded on the stage.
const emptyStageCard = (
  <div className="mx-auto flex w-full max-w-md flex-col items-center gap-3 rounded-2xl border border-amber-300/30 bg-black/40 px-5 py-5 text-center" data-testid="stage-empty-prep-card">
    <p className="text-[10px] font-bold uppercase tracking-[0.24em] text-amber-300/80">No track on stages</p>
    <p className="text-sm text-white/70">Prep a backing track and time the lyrics, or load a song you already prepared.</p>
    <button
      type="button"
      onClick={goToTrackPrep}
      className="flex w-full items-center justify-center gap-2 rounded-xl bg-amber-400 px-4 py-3 text-sm font-black uppercase tracking-wide text-zinc-950 shadow-[0_0_24px_rgba(251,191,36,0.35)] transition-colors hover:bg-amber-300"
      aria-label="Prep Backing Track & Sync Lyrics"
    />
  </div>
)
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
>
  << Prep Backing Track &amp; Sync Lyrics
</button>
<button
  type="button"
  onClick={() => void openVault()}
  className="flex min-h-11 w-full items-center justify-center gap-2 rounded-xl border border-amber-300/35 bg-amber-300/10 px-4 py-2.5 text-xs font-semibold uppercase tracking-widest text-amber-200 transition-colors hover:bg-amber-300/20"
>
  < Load from Vault
</button>
</div>
);

const audioOnlyCard = (
  <div className="mx-auto w-full max-w-lg rounded-2xl border border-sky-300/25 bg-black/45 px-5 py-5 text-center" data-testid="audio-only-visualizer">
    <p className="text-[10px] font-bold uppercase tracking-[0.24em] text-sky-300/80">Audio-Only Mode</p>
    <h2 className="mt-2 truncate text-xl font-black text-white">{trackName}</h2>
    <p className="mt-1 font-mono text-xs text-white/45">
      {Math.floor(durationSec / 60)}:{Math.floor(durationSec % 60).toString().padStart(2, "0")} · Free Performance
    </p>
    <div className="mx-auto mt-5 flex h-12 max-w-xs items-end justify-center gap-1.5" aria-label="Audio-only playback visualizer">
      {Array.from({ length: 18 }, (_, index) => (
        <span
          key={index}
          className={`w-2 rounded-full bg-gradient-to-t from-sky-500 to-amber-300 ${isPlaying ? "animate-pulse" : "opacity-40"}`}
          style={{ height: `${20 + ((index * 17) % 75)}%`, animationDelay: `${index * 45}ms` }}
        </span>
      ))}
    </div>
    <p className="mt-3 text-xs text-white/50">Transport, scrubber, live monitor, and recording remain fully active.</p>
  </div>
);

// — Monitor toggle button (bottom rail control slot) —————
const controlSlot = (
  <div className="flex items-center gap-1.5">
    <label className="flex min-h-11 items-center gap-1.5 rounded-full border border-white/15 px-2.5 py-1.5 text-[10px] text-white/65" title="Backing track master volume">
      {masterVolume === 0 ? <VolumeX className="h-3 w-3" /> : <Volume2 className="h-3 w-3" />}
      <span className="hidden sm:inline uppercase tracking-widest">Track</span>
      <input
        type="range"
        min={0}
        max={1}
        step={0.01}
        value={masterVolume}
        onChange={(event) => setMasterVolume(Number(event.target.value))}
        aria-label="Backing track master volume"
        className="h-11 min-h-11 w-16 accent-amber-300 sm:w-20"
      />
      <span className="w-8 text-right font-mono text-[9px]">{Math.round(masterVolume * 100)}%</span>
    </label>
    <button
      type="button"
      onClick={() => void openVault()}
      className="flex min-h-11 min-w-11 items-center justify-center gap-1.5 rounded-full border border-amber-300/35 bg-amber-300/10 px-3 py-1.5 text-[10px] font-semibold uppercase tracking-widest text-amber-200 transition-colors hover:bg-amber-300/20"
      aria-label="Select song from Vault"
    >
      <FolderOpen className="h-3 w-3" />
      <span className="hidden sm:inline">Select song from Vault</span>
    </button>
    <button
      type="button"
      onClick={toggleTake}
      aria-pressed={isRecording}
      aria-label={isRecording ? "Stop recording take" : "Record take"}
      className={[
        "flex min-h-11 min-w-11 items-center justify-center gap-1.5 rounded-full border px-3 py-1.5 text-[10px] font-semibold uppercase tracking-widest transition-all",
        isRecording
          ? "border-red-400/60 bg-red-500/20 text-red-300 shadow-[0_0_10px_rgba(239,68,68,0.28)]"
          : "border-white/15 text-white/65 hover:border-red-300/45 hover:text-red-200",
      ].join(" ")}
    >
      {isRecording ? <Square className="h-3 w-3 fill-current" /> : <span className="h-2 w-2 rounded-full bg-red-400" />}
      <span className="hidden sm:inline">{isRecording ? "Recording" : "Record take"}</span>
    </button>
    <button
      type="button"
      onClick={toggleMonitor}
      aria-pressed={monitorOn}
      aria-label={monitorOn ? "Turn monitor mic off" : "Turn monitor mic on"}
      className={[
        "flex min-h-11 min-w-11 items-center justify-center gap-1.5 rounded-full border px-3 py-1.5 text-[10px] font-semibold uppercase tracking-widest transition-all",
        monitorOn
          ? "border-emerald-400/60 bg-emerald-500/20 text-emerald-300 shadow-[0_0_10px_rgba(34,197,94,0.28)]"
          : "border-white/20 text-white/60 hover:border-white/40 hover:text-white",
      ].join(" ")}
    >
      {monitorOn ? <Headphones className="h-3 w-3" /> : <MicOff className="h-3 w-3" />}
      <span className="hidden sm:inline">{monitorOn ? "Monitor mic" : ""}</span>
    </button>
  </div>
);

return (
  <Layout hideChrome noPadding>
    {/* — Fullscreen karaoke stage: local performer takes full viewport — */}
    <LivePerformanceStage
      title="Main Stage"
      subtitle="Karaoke · Live Monitor"
      trackName={trackName}
      isPlaying={isPlaying}
      currentTimeSec={playbackPosition}
      durationSec={durationSec}
      lyrics={stageLyrics}
      performers={[
        {
          id: "host",
          label: monitorOn ? "Live mic" : "Standby",
          name: "You",
          accent: "gold",
          analyser: graphRef.current?.analyser ?? null,
          status: monitorOn ? "Live mic" : "Standby",
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
    },
  ],
  onExit={() => { stopMonitor(); window.history.back(); }}
  onTogglePlayback={togglePlayback}
  onStop={retake}
  onSeek={seekPlayback}
  onRecord={toggleTake}
  isRecording={isRecording}
  recordDisabled={false}
  controlSlot={controlSlot}
  emptySlot={!trackName ? emptyStageCard : stageLyrics.length === 0 ? audioOnlyCard : undefined}
  stageMode="solo"
  audioContextTimeSec={playbackContextRef.current?.currentTime}
  playbackStartContextTimeSec={playbackStartContextRef.current}
/>

<audio ref={reviewAudioRef} src={reviewUrl ?? undefined} preload="auto" className="hidden" />

{vaultOpen && (
  <div
    className="fixed inset-0 z-[70] flex items-center justify-center bg-black/70 p-4 backdrop-blur-sm"
    role="dialog"
    aria-modal="true"
    aria-label="Select song from Vault"
  >
    <div className="w-full max-w-lg rounded-3xl border border-white/10 bg-zinc-950/98 p-5 shadow-[0_0px_64px_rgba(0,0,0,0.8)]">
      <div className="mb-4 flex items-start justify-between gap-3">
        <div>
          <p className="text-[10px] font-bold uppercase tracking-[0.22em] text-amber-300">Stage Vault</p>
          <h2 className="mt-1 text-xl font-black text-white">Select Song from Vault</h2>
          <p className="mt-1 text-xs text-white/45">All saved audio tracks. Synced lyrics load the prompter; audio-only tracks play immediately.</p>
        </div>
        <button type="button" onClick={() => setVaultOpen(false)} className="rounded-full p-1.5 text-white/45 hover:bg-white/10 hover:text-white" aria-label="Close Vault selector">
          <X className="h-4 w-4" />
        </button>
      </div>

      {vaultLoading && <p className="rounded-xl border border-white/10 px-3 py-4 text-center text-xs text-white/50">Loading prepared songs...</p>}
      {vaultError && <p className="rounded-xl border border-red-400/20 bg-red-400/10 px-3 py-3 text-xs text-red-200">{vaultError}</p>}
      {!vaultLoading && !vaultError && vaultEntries.length === 0 && serverTracks.length === 0 && (
        <div className="rounded-xl border border-dashed border-white/15 px-4 py-8 text-center">
          <FolderOpen className="mx-auto h-6 w-6 text-amber-300/70" />
          <p className="mt-2 text-sm font-semibold text-white/75">Vault is empty</p>
          <p className="mt-1 text-xs text-white/40">Save any audio track from Track &amp; Lyric Prep to see it here.</p>
          <a href="/studio/track-prep" className="mt-4 inline-flex rounded-lg bg-amber-300 px-3 py-2 text-xs font-bold text-zinc-950 hover:bg-amber-200">
            Open Track Prep
          </a>
        </div>
      )}
      {!vaultLoading && (vaultEntries.length > 0 || serverTracks.length > 0) && (
        <div className="max-h-[min(55vh,26rem)] space-y-2 overflow-y-auto">
          {vaultEntries.map((entry) => {
            const hasSyncedLyrics = entry.lines.length > 0;
            const dur = entry.durationSeconds;
            const durStr = typeof dur === "number"
              ? `${Math.floor(dur / 60)}:${Math.floor(dur % 60).toString().padStart(2, "0")}`
              : null;
            return (
              <button
                key={entry.id}
                type="button"
                onClick={() => void (roomRole ? queueVaultSong(entry) : selectVaultSong(entry))}
                disabled={queueBusy}
                className="flex w-full items-center justify-between gap-3 rounded-xl border border-white/10 bg-white/[0.03] px-3 py-3 text-left transition-colors hover:border-amber-300/40 hover:bg-amber-300/10"
              >
                <span className="min-w-0">
                  <span className="block truncate text-sm font-semibold text-white">{entry.title}</span>
                  <span className="mt-1 block text-[10px] text-white/40">
                    {hasSyncedLyrics ? `${entry.lines.length} synced lines` : "Audio only"}
                    {durStr ? ` · ${durStr}` : ""}
                    {` · `}{new Date(entry.createdAt).toLocaleDateString()}
                  </span>
                </span>
                {hasSyncedLyrics ? (
                  <span className="shrink-0 rounded-full border border-amber-300/30 bg-amber-300/10 px-2 py-1 text-[9px] font-bold uppercase tracking-wide text-amber-200"> Synced</span>
                ) : (
                  <span className="shrink-0 rounded-full border border-sky-300/25 bg-sky-300/10 px-2 py-1 text-[9px] font-bold uppercase tracking-wide text-sky-200"> Audio</span>
                )}
                {roomRole && (
                  <span className="shrink-0 text-[9px] font-bold uppercase tracking-wide text-violet-200">+ Queue</span>
                )}
              </button>
            );
          })}
        </div>
      )}
      {serverTracks.length > 0 && (
        <div>
          {vaultEntries.length > 0 && (
            <p className="px-1 pt-2 text-[9px] font-bold uppercase tracking-[0.2em] text-white/30">Library tracks</p>
          )}
          {serverTracks.map((track) => (
            <button
              key={track.id}
              type="button"
              onClick={() => void selectServerTrack(track)}
              disabled={queueBusy}
              className="flex w-full items-center justify-between gap-3 rounded-xl border border-white/10 bg-white/[0.03] px-3 py-3 text-left transition-colors hover:border-amber-300/40 hover:bg-amber-300/10"
            >
              <span className="min-w-0">
                <span className="block truncate text-sm font-semibold text-white">{track.title}</span>
                <span className="mt-1 block text-[10px] text-white/40">
                  {track.artistName ? `${track.artistName} · ` : ""}Library
                  {track.createdAt ? ` · ${new Date(track.createdAt).toLocaleDateString()}` : ""}
                </span>
              </span>
              <span className="shrink-0 rounded-full border border-emerald-300/25 bg-emerald-300/10 px-2 py-1 text-[9px] font-bold uppercase tracking-wide text-emerald-200"> Cloud</span>
            </button>
          )}
        </div>
      )}
    </div>
  )}
</div>
</div>
</div>
</div>
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```
    ))

    /* — Partner HUD: floating corner bubble + invite modal ————— */
    <PartnerHud
      connected={peerConnected}
      profile={partnerProfile}
      partnerLevel={partnerLevel}
      volume={partnerVolume}
      onVolumeChange={setPartnerVolume}
      showInvite={showInvite}
      inviteCode={inviteCode}
      onOpenInvite={() => void openInvite()}
      onDismissInvite={() => setShowInvite(false)}
    />

    {(stageQueue.nowPlaying || stageQueue.upNext.length > 0) && (
      <aside className="fixed right-4 top-20 z-[61] w-[min(88vw,19rem)] rounded-2xl border border-violet-300/20 bg-zinc-950/95 p-3 shadow-2xl backdrop-blur-md" aria-label="Duet stage queue">
        <div className="flex items-center justify-between gap-2">
          <p className="flex items-center gap-2 text-[10px] font-bold uppercase tracking-[0.18em] text-violet-200">
            <ListMusic className="h-3.5 w-3.5" /> Duet Queue
          </p>
          {roomRole === "host" && (
            <button
              type="button"
              onClick={() => void advanceQueue()}
              disabled={queueBusy || !stageQueue.nowPlaying}
              className="inline-flex items-center gap-1 rounded-lg border border-amber-300/30 px-2 py-1 text-[9px] font-bold uppercase text-amber-200 disabled:opacity-35"
              aria-label="Advance duet queue"
            >
              <SkipForward className="h-3 w-3" /> Advance
            </button>
          )}
        </div>
        {stageQueue.nowPlaying && (
          <div className="mt-3 rounded-xl border border-amber-300/25 bg-amber-300/[0.07] p-3">
            <p className="text-[9px] font-bold uppercase tracking-widest text-amber-300">Now Playing</p>
            <p className="mt-1 truncate text-sm font-semibold text-white">{stageQueue.nowPlaying.title}</p>
            <p className="mt-1 text-[10px] text-white/40">
              Queued by {stageQueue.nowPlaying.queuedByRole === "host" ? "Host" : "Guest"} · audition owner
            </p>
          </div>
        )}
        <div className="mt-2 space-y-1.5">
          {stageQueue.upNext.map((item, index) => (
            <div key={item.id} className="flex items-center gap-2 rounded-lg border border-white/8 bg-white/[0.03] px-2.5 py-2">
              <span className="font-mono text-[10px] text-white/30">{index + 1}</span>
              <span className="min-w-0 flex-1 truncate text-xs text-white/70">{item.title}</span>
              <span className="text-[9px] uppercase text-white/35">{item.queuedByRole === "host" ? "Host" : "Guest"}</span>
            </div>
          ))}
          {stageQueue.upNext.length === 0 && <p className="px-1 py-1 text-[10px] text-white/30">Up Next is empty.</p>}
        </div>
      </aside>
    )}

    {takeBlob && reviewUrl && !isRecording && (
      <div className="fixed inset-0 z-[80] flex items-center justify-center bg-black/75 p-4 backdrop-blur-sm" role="dialog" aria-modal="true" aria-label="Post-performance take">
        <DuetReviewCard
          trackName={trackName}
          durationSec={durationSec}
          saving={savingTake}
          onPlayReview={playReview}
          onDownloadTake={downloadTake}
          onSaveToVaults={() => void saveToVault()}
          onRetakes={retake}
          auditionDisabled={Boolean(
            stageQueue.nowPlaying
              && stageQueue.nowPlaying.queuedByPeerId !== peerIdRef.current
          )}
        />
      </div>
    )}

    {takeSavedMessage && (
      <div className="fixed bottom-24 left-1/2 z-[85] flex -translate-x-1/2 items-center gap-2 rounded-xl border border-emerald-300/25 bg-zinc-950/95 px-4 py-3 text-sm text-emerald-200 shadow-2xl" role="status">
        <check className="h-4 w-4" />
        {takeSavedMessage}
        <button type="button" onClick={() => setTakeSavedMessage(null)} className="ml-2 text-white/45 hover:text-white" aria-label="Dismiss saved message"><X className="h-3.5 w-3.5" /></button>
      </div>
    )}

    /* — FX Rack overlay panel ————— */
    {fxOpen && (
      <div
        className={[
          "fixed z-[60] max-h-[calc(100vh-7rem)] w-72 overflow-y-auto rounded-2xl border border-white/10",
          "bg-zinc-950/96 p-4 shadow-[0_4px_40px_rgba(0,0,0,0.6)] backdrop-blur-md",
          // Offset from the right to avoid overlapping the partner HUD bubble
          "bottom-20 left-4 sm:bottom-24 sm:left-6 sm:w-80",
        ].join(" ")}
        role="region"
        aria-label="Live monitor FX rack"
      >
        /* Panel header */
        <div className="mb-3 flex items-center justify-between">
          <div className="flex items-center gap-2">
            <span className={["h-2 w-2 rounded-full", monitorOn ? "animate-pulse bg-emerald-400" : "bg-white/20"].join(" ")} />
            <span className="text-[10px] font-bold uppercase tracking-widest text-emerald-300">Live FX Monitor</span>
          </div>
          <button type="button" onClick={() => setFxOpen(false)} aria-label="Close FX rack" className="rounded p-1 text-white/40 hover:text-white">
            <X className="h-3.5 w-3.5" />
          </button>
        </div>

        /* Waveform oscilloscope */
        <canvas
          ref={canvasRef}
          width={288}
          height={36}
          className="mb-2 w-full rounded-lg border border-white/10 bg-black/50"
          style={{ height: 36 }}
          aria-label="Live input waveform"
        />
      </div>
    )}
```

04. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/pages/main-stage.tsx | source continues without omission

```

    /* Level bar meter */
    <div
      className="mb-4 flex h-2.5 items-end gap-0.5 overflow-hidden rounded-full"
      role="meter" aria-valuenow={Math.round(level * 100)} aria-valuemin={0} aria-valuemax={100}
      aria-label="Input level meter"
    >
      {Array.from({ length: 20 }).map((_, i) => (
        <div
          key={i}
          className="flex-1 rounded-sm transition-all duration-75"
          style={{
            background: i < 13 ? "#22c55e" : i < 17 ? "#f59e0b" : "#ef4444",
            opacity: level * 20 > i ? 1 : 0.15,
          }}
        >
        </div>
      ))}
    </div>

    /* Error */
    {error && (
      <p className="mb-3 rounded-lg bg-red-500/15 px-3 py-2 text-[11px] text-red-400">{error}</p>
    )}
    {bluetoothOutput && (
      <div className="mb-3 rounded-lg border border-sky-300/30 bg-sky-300/10 px-3 py-2 text-[10px] leading-snug text-sky-100">
        <strong>Bluetooth output detected.</strong> Connect wired headphones before monitoring or recording to avoid feedback and timing drift.
      </div>
    )}

    /* FX controls */
    <div className="space-y-5">
      /* Mic Gain */
      <div>
        <div className="mb-1.5 flex items-center justify-between">
          <label className="text-[11px] font-medium text-white/70">Mic Gain</label>
          <span className="font-mono text-[10px] text-emerald-400">{Math.round(settings.micGain * 100)} %</span>
        </div>
        <Slider min={0} max={2} step={0.01} value={settings.micGain}
          onValueChange={({v}) => { if (v !== undefined) setSettings(s => ({ ...s, micGain: v })); }}
          aria-label="Mic Gain" />
        </div>

        /* 80 Hz HPF */
        <div>
          <div className="mb-1.5 flex items-center justify-between">
            <label className="text-[11px] font-medium text-white/70">80 Hz Highpass Cut</label>
            <span className={["rounded-full px-2 py-0.5 text-[10px] font-bold uppercase tracking-wide", settings.hpfOn ? "bg-sky-500/20 text-sky-300" : "bg-white/5 text-white/30"].join(" ")}>
              {settings.hpfOn ? "ON" : "OFF"}
            </span>
          </div>
          <p className="mb-2 text-[9px] leading-snug text-muted-foreground">Removes mic rumble and handling noise. Recommended for live performance.</p>
          <button type="button"
            onClick={() => setSettings(s => ({ ...s, hpfOn: !s.hpfOn })))}
            className={["w-full rounded-lg border py-1.5 text-[11px] font-medium transition-colors", settings.hpfOn ? "border-sky-400/40 bg-sky-400/10 text-sky-300 hover:bg-sky-400/15" : "border-white/10 text-white/40 hover:border-white/20 hover:text-white/70"].join(" ")}>
            {settings.hpfOn ? "HPF Active – click to disable" : "HPF Off (flat) – click to enable"}
          </button>
        </div>

        /* Warmth EQ */
        <div>
          <div className="mb-1.5 flex items-center justify-between">
            <label className="text-[11px] font-medium text-white/70">Studio Warmth EQ</label>
            <span className="font-mono text-[10px] text-amber-400">{settings.warmth >= 0 ? "+" : ""}{settings.warmth.toFixed(1)} dB</span>
          </div>
          <Slider min={-3} max={6} step={0.1} value={settings.warmth}
            onValueChange={({v}) => { if (v !== undefined) setSettings(s => ({ ...s, warmth: v })); }}
            aria-label="Studio Warmth EQ" />
          <p className="mt-1 text-[9px] text-muted-foreground">Low-shelf at 250 Hz – adds body and vocal warmth</p>
        </div>

        /* Reverb Send */
        <div>
          <div className="mb-1.5 flex items-center justify-between">
            <label className="text-[11px] font-medium text-white/70">Reverb Send</label>
            <span className="font-mono text-[10px] text-violet-400">{Math.round(settings.reverbSend * 100)} %</span>
          </div>
          <Slider min={0} max={1} step={0.01} value={settings.reverbSend}
            onValueChange={({v}) => { if (v !== undefined) setSettings(s => ({ ...s, reverbSend: v })); }}
            aria-label="Reverb Send" />
          <p className="mt-1 text-[9px] text-muted-foreground">Warm room reverb – use headphones to prevent feedback</p>
        </div>
      </div>

      /* Monitor toggle in panel footer */
      <div className="mt-5 border-t border-white/10 pt-4">
        <button
          type="button"
          onClick={toggleMonitor}
          className={["flex w-full items-center justify-center gap-2 rounded-xl py-2.5 text-[11px] font-bold uppercase tracking-wider transition-colors",
            monitorOn
              ? "bg-emerald-500/20 text-emerald-300 hover:bg-red-500/15 hover:text-red-300"
              : "bg-white/5 text-white/50 hover:bg-emerald-500/15 hover:text-emerald-300",
            ].join(" ")}>
        </button>
        <Headphones className="h-3.5 w-3.5" />
        {monitorOn ? "Monitor ON – tap to stop" : "Start monitoring"}
      </button>
      <p className="mt-2 text-center text-[9px] text-muted-foreground">Direct sidetone · latencyHint: interactive · Use headphones</p>
    </div>
  </Layout>
);
};
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK

artifacts/gravelkingpro/src/index.css | 1,324 lines | 37,254 bytes | sha256 19d43db9c51c1fa3af2fa5e6e6ffe8ec6449014e3383c4820eddf0269556a73c

```
@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800&family=Space+Mono:ital,wght@0,400;0,700;1,400&display=swap');
@layer theme, base, clerk, components, utilities;
@import "tailwindcss";
@import "@clerk/themes/shadcn.css";
@import "tw-animate-css";
@plugin "@tailwindcss/typography";

@custom-variant dark (&:is(.dark *));

@theme inline {
  --color-background: hsl(var(--background));
  --color-foreground: hsl(var(--foreground));
  --color-border: hsl(var(--border));
  --color-input: hsl(var(--input));
  --color-ring: hsl(var(--ring));

  --color-card: hsl(var(--card));
  --color-card-foreground: hsl(var(--card-foreground));
  --color-card-border: hsl(var(--card-border));

  --color-popover: hsl(var(--popover));
  --color-popover-foreground: hsl(var(--popover-foreground));
  --color-popover-border: hsl(var(--popover-border));

  --color-primary: hsl(var(--primary));
  --color-primary-foreground: hsl(var(--primary-foreground));
  --color-primary-border: var(--primary-border);

  --color-secondary: hsl(var(--secondary));
  --color-secondary-foreground: hsl(var(--secondary-foreground));
  --color-secondary-border: var(--secondary-border);

  --color-muted: hsl(var(--muted));
  --color-muted-foreground: hsl(var(--muted-foreground));
  --color-muted-border: var(--muted-border);

  --color-accent: hsl(var(--accent));
  --color-accent-foreground: hsl(var(--accent-foreground));
  --color-accent-border: var(--accent-border);

  --color-destructive: hsl(var(--destructive));
  --color-destructive-foreground: hsl(var(--destructive-foreground));
  --color-destructive-border: var(--destructive-border);

  --color-chart-1: hsl(var(--chart-1));
  --color-chart-2: hsl(var(--chart-2));
  --color-chart-3: hsl(var(--chart-3));
  --color-chart-4: hsl(var(--chart-4));
  --color-chart-5: hsl(var(--chart-5));

  --color-sidebar: hsl(var(--sidebar));
  --color-sidebar-foreground: hsl(var(--sidebar-foreground));
  --color-sidebar-border: hsl(var(--sidebar-border));
  --color-sidebar-primary: hsl(var(--sidebar-primary));
  --color-sidebar-primary-foreground: hsl(var(--sidebar-primary-foreground));
  --color-sidebar-primary-border: var(--sidebar-primary-border);
  --color-sidebar-accent: hsl(var(--sidebar-accent));
  --color-sidebar-accent-foreground: hsl(var(--sidebar-accent-foreground));
  --color-sidebar-accent-border: var(--sidebar-accent-border);
  --color-sidebar-ring: hsl(var(--sidebar-ring));

  --font-sans: var(--app-font-sans);
  --font-serif: var(--app-font-serif);
  --font-mono: var(--app-font-mono);

  --radius-sm: calc(var(--radius) - 4px);
  --radius-md: calc(var(--radius) - 2px);
  --radius-lg: var(--radius);
  --radius-xl: calc(var(--radius) + 4px);
}

/* LIGHT MODE - Forcing dark mode colors everywhere for this specific app */
:root {
  --button-outline: rgba(255,255,255, .10);
  --badge-outline: rgba(255,255,255, .05);

  /* Automatic computation of border around primary / danger buttons */
  --opaque-button-border-intensity: 9;

  /* Backgrounds applied on top of other backgrounds when hovered/active */
  --elevate-1: rgba(255,255,255, .04);
  --elevate-2: rgba(255,255,255, .09);

  /* Studio-grade dark theme */
  --background: 230 26% 5%;
  --foreground: 0 0% 98%; /* #fafafa */

  --border: 230 18% 20%;

  --card: 230 24% 10%;
  --card-foreground: 0 0% 98%;
  --card-border: 42 34% 28%;

  --sidebar: 230 25% 8%;
  --sidebar-foreground: 0 0% 98%;
  --sidebar-border: 230 18% 20%;
  --sidebar-primary: 43 96% 56%;
  --sidebar-primary-foreground: 230 26% 5%;
  --sidebar-accent: 265 28% 18%;
  --sidebar-accent-foreground: 0 0% 98%;
  --sidebar-ring: 43 96% 56%;

  --popover: 230 24% 10%;
  --popover-foreground: 0 0% 98%;
  --popover-border: 42 34% 28%;

  /* Console gold + violet accent */
  --primary: 43 96% 56%;
  --primary-foreground: 230 26% 5%;

  --secondary: 230 20% 15%;
  --secondary-foreground: 0 0% 98%; /* #fafafa */
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```
--muted: 230 20% 13%;
--muted-foreground: 230 12% 68%;

--accent: 267 84% 70%;
--accent-foreground: 230 26% 5%;

--destructive: 0 84% 60%;
--destructive-foreground: 0 0% 98%;

--input: 230 18% 22%;
--ring: 43 96% 56%;
--chart-1: 43 96% 56%;
--chart-2: 267 84% 70%;
--chart-3: 193 92% 62%;
--chart-4: 154 70% 54%;
--chart-5: 230 12% 72%;

--app-font-sans: 'Inter', sans-serif;
--app-font-serif: Georgia, serif;
--app-font-mono: 'Space Mono', monospace;
--radius: 0.9rem;
--shadow-2xs: 0 2px 4px 0 hsl(230 40% 2% / 0.34);
--shadow-xs: 0 4px 10px 0 hsl(230 40% 2% / 0.34);
--shadow-sm: 0 8px 18px -4px hsl(230 40% 2% / 0.45);
--shadow: 0 12px 28px -8px hsl(230 40% 2% / 0.55);
--shadow-md: 0 16px 38px -12px hsl(230 40% 2% / 0.62);
--shadow-lg: 0 24px 54px -16px hsl(230 40% 2% / 0.7);
--shadow-xl: 0 30px 70px -20px hsl(230 40% 2% / 0.74);
--shadow-2xl: 0 36px 90px -24px hsl(230 40% 2% / 0.8);
--tracking-normal: 0em;
--spacing: 0.25rem;

/* Fallback for older browsers */
--sidebar-primary-border: hsl(var(--sidebar-primary));
--sidebar-primary-border: hsl(from hsl(var(--sidebar-primary)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);

/* Fallback for older browsers */
--sidebar-accent-border: hsl(var(--sidebar-accent));
--sidebar-accent-border: hsl(from hsl(var(--sidebar-accent)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);

/* Fallback for older browsers */
--primary-border: hsl(var(--primary));
--primary-border: hsl(from hsl(var(--primary)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);

/* Fallback for older browsers */
--secondary-border: hsl(var(--secondary));
--secondary-border: hsl(from hsl(var(--secondary)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);

/* Fallback for older browsers */
--muted-border: hsl(var(--muted));
--muted-border: hsl(from hsl(var(--muted)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);

/* Fallback for older browsers */
--accent-border: hsl(var(--accent));
--accent-border: hsl(from hsl(var(--accent)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);

/* Fallback for older browsers */
--destructive-border: hsl(var(--destructive));
--destructive-border: hsl(from hsl(var(--destructive)) h s calc(l + var(--opaque-button-border-intensity)) / alpha);
}

.dark {
  --button-outline: rgba(255,255,255, .10);
  --badge-outline: rgba(255,255,255, .05);

  --opaque-button-border-intensity: 9; /* In terms of percentages */

  /* Backgrounds applied on top of other backgrounds when hovered/active */
  --elevate-1: rgba(255,255,255, .04);
  --elevate-2: rgba(255,255,255, .09);

  --background: 230 26% 5%;
  --foreground: 0 0% 98%; /* #fafafa */

  --border: 230 18% 20%;

  --card: 230 24% 10%;
  --card-foreground: 0 0% 98%;
  --card-border: 42 34% 28%;

  --sidebar: 230 25% 8%;
  --sidebar-foreground: 0 0% 98%;
  --sidebar-border: 230 18% 20%;
  --sidebar-primary: 43 96% 56%;
  --sidebar-primary-foreground: 230 26% 5%;
  --sidebar-accent: 265 28% 18%;
  --sidebar-accent-foreground: 0 0% 98%;
  --sidebar-ring: 43 96% 56%;

  --popover: 230 24% 10%;
  --popover-foreground: 0 0% 98%;
  --popover-border: 42 34% 28%;

  /* Deep Amber Accent */
  --primary: 43 96% 56%;
  --primary-foreground: 230 26% 5%;

  --secondary: 230 20% 15%;
  --secondary-foreground: 0 0% 98%; /* #fafafa */

  --muted: 230 20% 13%;
  --muted-foreground: 230 12% 68%;

  --accent: 267 84% 70%;
  --accent-foreground: 230 26% 5%;

  --destructive: 0 84% 60%;
  --destructive-foreground: 0 0% 98%;

  --input: 230 18% 22%;
  --ring: 43 96% 56%;
  --chart-1: 43 96% 56%;
  --chart-2: 267 84% 70%;
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```
--chart-3: 193 92% 62%;
--chart-4: 154 70% 54%;
--chart-5: 230 12% 72%;

--shadow-2xs: 0 2px 4px 0 hsl(230 40% 2% / 0.34);
--shadow-xs: 0 4px 10px 0 hsl(230 40% 2% / 0.34);
--shadow-sm: 0 8px 18px -4px hsl(230 40% 2% / 0.45);
--shadow: 0 12px 28px -8px hsl(230 40% 2% / 0.55);
--shadow-md: 0 18px 38px -12px hsl(230 40% 2% / 0.62);
--shadow-lg: 0 24px 54px -16px hsl(230 40% 2% / 0.7);
--shadow-xl: 0 30px 70px -20px hsl(230 40% 2% / 0.74);
--shadow-2xl: 0 36px 90px -24px hsl(230 40% 2% / 0.8);

}

@layer base {
  * {
    @apply border-border;
    /* Remove blue tap flash on Android / Samsung Internet */
    -webkit-tap-highlight-color: transparent;
  }

  body {
    @apply font-sans antialiased bg-background text-foreground;
    min-height: 100vh;
    background:
      radial-gradient(circle at 12% -8%, hsl(267 70% 36% / 0.28), transparent 34rem),
      radial-gradient(circle at 88% 4%, hsl(43 96% 56% / 0.12), transparent 28rem),
      radial-gradient(circle at 52% 82%, hsl(193 92% 62% / 0.07), transparent 36rem),
      hsl(var(--background));
    background-attachment: fixed;
    /* iOS momentum scroll on the root */
    -webkit-overflow-scrolling: touch;
  }

  /* Prevent iOS Safari from auto-zooming into inputs – 16 px floor */
  input:not([type="range"]):not([type="checkbox"]):not([type="radio"]),
  textarea,
  select {
    font-size: max(1rem, 1em);
  }

  /* Eliminate the 300 ms tap delay on tappable elements */
  a, button, [role="button"] {
    touch-action: manipulation;
  }

  /* Respect iPhone / Galaxy fold notch / home-bar */
  header {
    padding-top: env(safe-area-inset-top);
  }

  footer {
    padding-bottom: env(safe-area-inset-bottom);
  }

  button,
  [role="button"],
  a {
    transition-property: transform, box-shadow, background-color, border-color, color, opacity;
    transition-duration: 180ms;
    transition-timing-function: cubic-bezier(0.22, 1, 0.36, 1);
    will-change: transform;
  }

  button:not(:disabled):active,
  [role="button"]:not([aria-disabled="true"]):active {
    transform: scale(0.97);
  }

  button:focus-visible,
  [role="button"]:focus-visible,
  a:focus-visible {
    outline: 2px solid hsl(var(--primary) / 0.85);
    outline-offset: 3px;
  }
}

/**
 * Using the elevate system.
 * Automatic contrast adjustment.
 *
 * <element className="hover-elevate" />
 * <element className="active-elevate-2" />
 *
 * // Using the tailwind utility when a data attribute is "on"
 * <element className="toggle-elevate data-[state=on]:toggle-elevated" />
 * // Or manually controlling the toggle state
 * <element className="toggle-elevate toggle-elevated" />
 *
 * Elevation systems have to handle many states.
 * - not-hovered, vs. hovered vs. active (three mutually exclusive states)
 * - toggled or not
 * - focused or not (this is not handled with these utilities)
 *
 * Even without handling focused or not, this is six possible combinations that
 * need to be distinguished from eachother visually.
 */
@layer utilities {

  /* Hide ugly search cancel button in Chrome until we can style it properly */
  input[type="search"]::-webkit-search-cancel-button {
    @apply hidden;
  }

  /* Placeholder styling for contentEditable div */
  [contenteditable][data-placeholder]:empty:before {
    content: attr(data-placeholder);
    color: hsl(var(--muted-foreground));
    pointer-events: none;
  }

  /* .no-default-hover-elevate/no-default-active-elevate is an escape hatch so consumers of
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```

    * buttons/badges can remove the automatic brightness adjustment on interactions
    * and program their own. */
.no-default-hover-elevate {}

.no-default-active-elevate {}

/**
 * Toggleable backgrounds go behind the content. Hoverable/active goes on top.
 * This way they can stack/compound. Both will overlap the parent's borders!
 * So borders will be automatically adjusted both on toggle, and hover/active,
 * and they will be compounded.
 */
.toggle-elevate::before,
.toggle-elevate-2::before {
  content: "";
  pointer-events: none;
  position: absolute;
  inset: 0px;
  /*border-radius: inherit;   match rounded corners */
  border-radius: inherit;
  z-index: -1;
  /* sits behind content but above backdrop */
}

.toggle-elevate.toggle-elevated::before {
  background-color: var(--elevate-2);
}

/* If there's a 1px border, adjust the inset so that it covers that parent's border */
.border.toggle-elevate::before {
  inset: -1px;
}

/* Does not work on elements with overflow:hidden! */
.hover-elevate:not(.no-default-hover-elevate),
.active-elevate:not(.no-default-active-elevate),
.hover-elevate-2:not(.no-default-hover-elevate),
.active-elevate-2:not(.no-default-active-elevate) {
  position: relative;
  z-index: 0;
}

.hover-elevate:not(.no-default-hover-elevate)::after,
.active-elevate:not(.no-default-active-elevate)::after,
.hover-elevate-2:not(.no-default-hover-elevate)::after,
.active-elevate-2:not(.no-default-active-elevate)::after {
  content: "";
  pointer-events: none;
  position: absolute;
  inset: 0px;
  /*border-radius: inherit;   match rounded corners */
  border-radius: inherit;
  z-index: 999;
  /* sits in front of content */
}

.hover-elevate: hover: not(.no-default-hover-elevate)::after,
.active-elevate: active: not(.no-default-active-elevate)::after {
  background-color: var(--elevate-1);
}

.hover-elevate-2: hover: not(.no-default-hover-elevate)::after,
.active-elevate-2: active: not(.no-default-active-elevate)::after {
  background-color: var(--elevate-2);
}

/* If there's a 1px border, adjust the inset so that it covers that parent's border */
.border.hover-elevate: not(.no-hover-interaction-elevate)::after,
.border.active-elevate: not(.no-active-interaction-elevate)::after,
.border.hover-elevate-2: not(.no-hover-interaction-elevate)::after,
.border.active-elevate-2: not(.no-active-interaction-elevate)::after,
.border.hover-elevate: not(.no-hover-interaction-elevate)::after {
  inset: -1px;
}
}

@layer components {
  .app-shell {
    position: relative;
    isolation: isolate;
    background:
      linear-gradient(135deg, hsl(230 30% 7% / 0.9), hsl(267 32% 9% / 0.72) 52%, hsl(230 28% 6% / 0.92));
  }

  .phase-one-app {
    min-height: 100vh;
    padding-top: 3.5rem;
    padding-bottom: 5.75rem;
    background:
      repeating-linear-gradient(0deg, hsl(0 0% 100% / 0.018) 0, hsl(0 0% 100% / 0.018) 1px, transparent 1px, transparent 4px);
  }

  .phase-one-app .studio-chrome,
  .phase-one-app .app-shell > footer {
    display: none;
  }

  .phase-one-topbar,
  .phase-one-dock {
    background:
      linear-gradient(140deg, hsl(230 28% 14% / 0.96), hsl(230 18% 8% / 0.96)),
      repeating-linear-gradient(0deg, hsl(0 0% 100% / 0.035) 0, hsl(0 0% 100% / 0.035) 1px, transparent 1px, transparent 3px);
    box-shadow: 0 18px 42px hsl(230 45% 2% / 0.42), inset 0 1px 0 hsl(0 0% 100% / 0.06);
    backdrop-filter: blur(18px) saturate(125%);
    -webkit-backdrop-filter: blur(18px) saturate(125%);
  }

  .app-shell::before {
    content: "";
    position: fixed;
    inset: 0;
    z-index: -1;
  }
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```

    pointer-events: none;
    opacity: 0.22;
    background-image:
      linear-gradient(hsl(0 0% 100% / 0.025) 1px, transparent 1px),
      linear-gradient(90deg, hsl(0 0% 100% / 0.025) 1px, transparent 1px);
    background-size: 42px 42px;
    mask-image: linear-gradient(to bottom, black, transparent 78%);
  }

.studio-chrome,
.studio-card {
  position: relative;
  overflow: hidden;
  border-color: hsl(43 48% 44% / 0.24);
  background:
    linear-gradient(145deg, hsl(230 26% 13% / 0.88), hsl(267 24% 12% / 0.72)),
    repeating-linear-gradient(0deg, hsl(0 0% 100% / 0.026) 0 1px, transparent 1px 4px);
  box-shadow:
    inset 0 1px 0 hsl(0 0% 100% / 0.055),
    inset 0 0 0 1px hsl(0 0% 100% / 0.025),
    0 18px 44px hsl(230 45% 2% / 0.34);
  backdrop-filter: blur(18px) saturate(125%);
  -webkit-backdrop-filter: blur(18px) saturate(125%);
}

.studio-led {
  display: inline-block;
  width: 0.42rem;
  height: 0.42rem;
  border-radius: 999px;
  background: hsl(43 96% 56%);
  box-shadow: 0 0 9px hsl(43 96% 56% / 0.8);
}

.studio-led-dim {
  background: hsl(230 12% 35%);
  box-shadow: none;
}

.studio-vu {
  display: flex;
  height: 0.5rem;
  align-items: stretch;
  gap: 2px;
}

.studio-vu > span {
  width: 0.28rem;
  border-radius: 1px;
  background: linear-gradient(to top, hsl(43 96% 56%), hsl(8 82% 58%));
  box-shadow: 0 0 6px hsl(43 96% 56% / 0.5);
}

.studio-vu > span:nth-child(1) { opacity: 0.42; }
.studio-vu > span:nth-child(2) { opacity: 0.58; }
.studio-vu > span:nth-child(3) { opacity: 0.72; }
.studio-vu > span:nth-child(4) { opacity: 0.86; }
.studio-vu > span:nth-child(5) { opacity: 1; }

.studio-chrome {
  box-shadow: 0 12px 32px hsl(230 45% 2% / 0.34);
}

.studio-button {
  position: relative;
  overflow: hidden;
  border-width: 1px;
  box-shadow: inset 0 1px 0 hsl(0 0% 100% / 0.1), 0 6px 16px hsl(230 45% 2% / 0.28);
}

.studio-button::before {
  content: "";
  position: absolute;
  inset: 0;
  pointer-events: none;
  background: linear-gradient(115deg, transparent 18%, hsl(0 0% 100% / 0.16), transparent 52%);
  transform: translateX(-120%);
  transition: transform 420ms cubic-bezier(0.22, 1, 0.36, 1);
}

.studio-button:hover::before {
  transform: translateX(120%);
}

.studio-button-primary {
  box-shadow: inset 0 1px 0 hsl(0 0% 100% / 0.24), 0 0 0 1px hsl(43 96% 56% / 0.2), 0 8px 24px hsl(43 96% 56% / 0.2);
}

.studio-button-primary:hover,
.studio-button-primary:focus-visible {
  box-shadow: inset 0 1px 0 hsl(0 0% 100% / 0.3), 0 0 26px hsl(43 96% 56% / 0.32), 0 10px 26px hsl(230 45% 2% / 0.32);
}

.studio-button-destructive:hover,
.studio-button-destructive:focus-visible {
  box-shadow: 0 0 22px hsl(0 84% 60% / 0.25);
}

.studio-badge {
  border-color: hsl(43 48% 44% / 0.3);
  background: linear-gradient(135deg, hsl(230 22% 18% / 0.75), hsl(267 28% 20% / 0.58));
  box-shadow: inset 0 1px 0 hsl(0 0% 100% / 0.06);
}

.audio-reactive {
  --audio-level: 0;
  --audio-ring: 0px;
  --audio-ring-wide: 0px;
  --audio-opacity: 0;
  --audio-secondary-opacity: 0;
  position: relative;
  isolation: isolate;
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```
.audio-reactive::before,
.audio-reactive::after {
  content: "";
  position: absolute;
  inset: calc(-5px - var(--audio-ring));
  z-index: -1;
  border: 1px solid hsl(43 96% 56% / var(--audio-opacity));
  border-radius: inherit;
  pointer-events: none;
  opacity: 0;
  transform: scale(1);
  transition: opacity 180ms ease, transform 180ms ease, inset 180ms ease;
}

.audio-reactive[data-audio-active="true"]::before,
.audio-reactive[data-audio-active="true"]::after {
  opacity: 1;
}

.audio-reactive[data-audio-active="true"]::after {
  inset: calc(-11px - var(--audio-ring-wide));
  border-color: hsl(267 84% 70% / var(--audio-secondary-opacity));
  animation: audio-ring-breathe 1.7s ease-in-out infinite;
}

.performer-badge {
  position: relative;
  box-shadow: 0 0 0 1px hsl(43 96% 56% / 0.16), 0 0 18px hsl(267 84% 70% / 0.12);
}

@keyframes audio-ring-breathe {
  0%, 100% { transform: scale(1); }
  50% { transform: scale(1.025); }
}

@media (prefers-reduced-motion: reduce) {
  .studio-button::before,
  .audio-reactive[data-audio-active="true"]::after {
    animation: none;
    transition: none;
  }
}

.live-performance-stage {
  --stage-audio-level: 0;
  --stage-beat-scale: 0;
  background:
    radial-gradient(ellipse at 50% 72%, hsl(267 86% 58% / 0.18), transparent 44%),
    radial-gradient(ellipse at 50% 8%, hsl(43 96% 56% / 0.12), transparent 38%),
    hsl(230 31% 4%);
  color: hsl(0 0% 98%);
  isolation: isolate;
}

.stage-ambient-glow {
  position: absolute;
  z-index: -1;
  width: 34rem;
  height: 34rem;
  border-radius: 999px;
  filter: blur(70px);
  opacity: 0.16;
  pointer-events: none;
}

.stage-ambient-glow-left {
  top: 18%;
  left: -22rem;
  background: hsl(267 84% 70%);
}

.stage-ambient-glow-right {
  top: 24%;
  right: -22rem;
  background: hsl(43 96% 56%);
}

.stage-command-bar,
.stage-control-rail {
  background: linear-gradient(180deg, hsl(230 30% 11% / 0.9), hsl(230 30% 7% / 0.82));
  border-color: hsl(0 0% 100% / 0.1);
  backdrop-filter: blur(22px) saturate(135%);
  -webkit-backdrop-filter: blur(22px) saturate(135%);
}

.stage-command-bar {
  border-bottom: 1px solid hsl(0 0% 100% / 0.09);
}

.stage-eyebrow {
  color: hsl(43 96% 70% / 0.75);
  font-size: 0.58rem;
  font-weight: 800;
  letter-spacing: 0.18em;
  line-height: 1.2;
  text-transform: uppercase;
}

.stage-icon-button,
.stage-transport-button {
  display: inline-flex;
  align-items: center;
  justify-content: center;
  border: 1px solid hsl(0 0% 100% / 0.12);
  color: hsl(0 0% 100% / 0.7);
  background: hsl(230 26% 18% / 0.65);
  transition: transform 180ms ease, border-color 180ms ease, color 180ms ease, background-color 180ms ease;
}

.stage-icon-button {
  height: 2.15rem;
  width: 2.15rem;
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```

    border-radius: 0.7rem;
  }

.stage-transport-button {
  height: 2rem;
  width: 2rem;
  border-radius: 999px;
}

.stage-icon-button:hover,
.stage-transport-button:hover {
  border-color: hsl(43 96% 56% / 0.62);
  color: hsl(43 96% 78%);
  background: hsl(43 96% 56% / 0.1);
  transform: translateY(-1px);
}

.stage-viewport {
  position: relative;
  display: flex;
  flex-direction: column;
  gap: clamp(0.65rem, 1.7vh, 1.15rem);
  min-height: 0;
}

.stage-truss {
  position: absolute;
  inset: 0 0 auto;
  height: 43%;
  min-height: 10rem;
  overflow: hidden;
  pointer-events: none;
}

.stage-truss-beam {
  position: absolute;
  top: 0.8rem;
  height: 0.4rem;
  border: 1px solid hsl(43 64% 58% / 0.58);
  border-radius: 999px;
  background: linear-gradient(90deg, transparent, hsl(0 0% 100% / 0.2), transparent);
  box-shadow: 0 0 18px hsl(43 96% 56% / 0.22);
}

.stage-truss-beam-back {
  left: 4%;
  right: 4%;
  transform: rotate(-1deg);
}

.stage-truss-beam-front {
  left: 13%;
  right: 13%;
  top: 2.4rem;
  opacity: 0.52;
  transform: rotate(1.2deg);
}

.stage-spotlight-svg {
  position: absolute;
  inset: 0;
  height: 100%;
  width: 100%;
  overflow: visible;
}

.stage-spotlight {
  transform-box: fill-box;
  transform-origin: top center;
  opacity: 0.5;
  mix-blend-mode: screen;
  filter: blur(1px) brightness(calc(1 + var(--stage-audio-level)));
}

.live-performance-stage[data-playing="true"] .stage-spotlight {
  animation: stage-spotlight-pulse 1.15s ease-in-out infinite;
}

.stage-spotlight-center {
  animation-delay: -0.38s !important;
}

.stage-spotlight-right {
  animation-delay: -0.76s !important;
}

@keyframes stage-spotlight-pulse {
  0%, 100% {
    opacity: 0.34;
    transform: scaleY(0.94);
  }
  50% {
    opacity: 0.7;
    transform: scaleY(1.02);
  }
}

.stage-jumbotron {
  position: relative;
  z-index: 1;
  width: min(100%, 56rem);
  margin: clamp(0.25rem, 1.5vh, 0.8rem) auto 0;
  flex: 0 1 40%;
  min-height: 0.7rem;
}

.live-performance-stage[data-stage-mode="solo"] .stage-jumbotron {
  width: min(100%, 76rem);
  flex-basis: 46%;
}

.live-performance-stage[data-stage-mode="solo"] .stage-lyric-line {
  font-size: clamp(1.05rem, 3vw, 2rem);
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```

    line-height: 1.22;
  }

.stage-jumbotron-bezel {
  position: relative;
  height: calc(100% - 1.05rem);
  overflow: hidden;
  border: 1px solid hsl(267 84% 78% / 0.34);
  border-radius: 1.35rem;
  background: linear-gradient(145deg, hsl(230 30% 13% / 0.84), hsl(267 38% 12% / 0.74));
  box-shadow:
    inset 0 1px 0 hsl(0 0% 100% / 0.12),
    0 0 1px hsl(43 96% 56% / 0.08),
    0 22px 48px hsl(230 45% 2% / 0.55),
    0 0 36px hsl(267 84% 70% / 0.14);
  backdrop-filter: blur(20px) saturate(135%);
  -webkit-backdrop-filter: blur(20px) saturate(135%);
}

.stage-jumbotron-bezel::after {
  content: "";
  position: absolute;
  inset: 0.5rem;
  border: 1px solid hsl(0 0% 100% / 0.08);
  border-radius: 1rem;
  pointer-events: none;
}

.stage-jumbotron-mesh {
  position: absolute;
  inset: 0;
  opacity: 0.3;
  background:
    linear-gradient(115deg, transparent 0 42%, hsl(267 84% 70% / 0.13) 50%, transparent 58%),
    repeating-linear-gradient(0deg, transparent 0 0.65rem, hsl(0 0% 100% / 0.025) 0.7rem 0.74rem);
  mask-image: linear-gradient(to bottom, transparent, black 20%, black 80%, transparent);
}

.stage-jumbotron-content {
  position: relative;
  display: flex;
  height: 100%;
  flex-direction: column;
  justify-content: center;
  padding: 1.3rem clamp(1rem, 5vw, 4.4rem);
  text-align: center;
}

.stage-jumbotron-label {
  color: hsl(43 96% 70% / 0.7);
  font-size: 0.56rem;
  font-weight: 800;
  letter-spacing: 0.24em;
  text-transform: uppercase;
}

.stage-lyric-stack {
  position: relative;
  z-index: 1;
  margin-top: 0.42rem;
  min-height: 3.9rem;
  overflow: hidden;
}

.stage-lyric-line {
  margin: 0.15rem 0;
  color: hsl(0 0% 100% / 0.26);
  font-size: clamp(0.85rem, 2.1vw, 1.22rem);
  font-weight: 600;
  letter-spacing: 0.01em;
  line-height: 1.3;
  transition: color 180ms ease, filter 180ms ease, transform 180ms ease;
}

.stage-lyric-line-active {
  color: hsl(43 96% 76%);
  filter: drop-shadow(0 0 13px hsl(43 96% 56% / 0.46));
  transform: scale(1.025);
}

.stage-cadence-ribbon {
  position: absolute;
  right: 0.9rem;
  bottom: 0;
  left: 0.9rem;
  height: 0.72rem;
  overflow: hidden;
  border: 1px solid hsl(0 0% 100% / 0.1);
  border-radius: 999px;
  background: hsl(230 30% 4% / 0.84);
  box-shadow: inset 0 1px 2px hsl(0 0% 0% / 0.5);
}

.stage-cadence-block {
  position: absolute;
  top: 0.16rem;
  height: 0.34rem;
  border-radius: 999px;
  background: linear-gradient(90deg, hsl(267 84% 70% / 0.35), hsl(43 96% 56% / 0.46));
  transition: background-color 180ms ease, box-shadow 180ms ease;
}

.stage-cadence-block-active {
  background: hsl(43 96% 68%);
  box-shadow: 0 0 12px hsl(43 96% 56% / 0.85);
}

.stage-strike-cursor {
  position: absolute;
  top: -0.3rem;
  bottom: -0.3rem;
  z-index: 2;
  width: 1px;
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```

    background: hsl(0 0% 100% / 0.88);
    box-shadow: 0 0 10px hsl(43 96% 70% / 0.9);
    transition: left 80ms linear;
  }

.stage-deck-wrap {
  position: relative;
  z-index: 2;
  display: flex;
  min-height: 10.5rem;
  flex: 1 1 42%;
  align-items: flex-end;
  justify-content: center;
  perspective: 600px;
  perspective-origin: center 62%;
}

.stage-deck {
  position: relative;
  width: min(100%, 61rem);
  height: 91%;
  min-height: 10rem;
  transform: rotateX(45deg) translateY(9%);
  transform-style: preserve-3d;
}

.stage-deck::before {
  content: "";
  position: absolute;
  inset: -1.2rem -0.8rem;
  border: 1px solid hsl(43 96% 56% / 0.34);
  border-radius: 1.1rem;
  box-shadow:
    0 0 0 1px hsl(267 84% 70% / 0.16),
    0 0 26px hsl(43 96% 56% / 0.2),
    inset 0 0 24px hsl(267 84% 70% / 0.15);
  pointer-events: none;
}

.stage-deck-grid {
  position: absolute;
  inset: 0;
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  grid-template-rows: repeat(4, 1fr);
  overflow: hidden;
  border: 1px solid hsl(267 84% 70% / 0.26);
  border-radius: 0.95rem;
  background: linear-gradient(145deg, hsl(230 28% 13% / 0.9), hsl(267 30% 11% / 0.96));
  box-shadow:
    inset 0 0 36px hsl(267 84% 70% / 0.17),
    0 18px 38px hsl(230 45% 2% / 0.62);
}

.stage-deck-tile {
  border-right: 1px solid hsl(267 84% 70% / 0.12);
  border-bottom: 1px solid hsl(267 84% 70% / 0.15);
  background: linear-gradient(135deg, hsl(0 0% 100% / 0.028), transparent 45%);
}

.live-performance-stage[data-playing="true"] .stage-deck-tile:nth-child(3n) {
  animation: stage-tile-shimmer 1.8s ease-in-out infinite;
}

.stage-performer-row {
  position: absolute;
  inset: -2.6rem 8% auto;
  display: flex;
  align-items: flex-end;
  justify-content: space-around;
  gap: 2.5rem;
  transform: rotateX(-45deg) translateZ(4.8rem);
  transform-origin: center bottom;
}

.live-performance-stage[data-stage-mode="solo"] .stage-performer-row {
  inset-right: 22%;
  inset-left: 22%;
  justify-content: center;
}

.live-performance-stage[data-stage-mode="solo"] .stage-performer-riser {
  min-width: min(72vw, 20rem);
  min-height: 9.5rem;
  flex: 0 1 22rem;
}

.stage-performer-riser {
  --mic-level: 0;
  --mic-scale: 0;
  --mic-glow: 0.18;
  --mic-blur: 10px;
  position: relative;
  display: flex;
  min-width: min(34vw, 13rem);
  min-height: 8rem;
  flex: 1 1 0;
  flex-direction: column;
  align-items: center;
  justify-content: flex-end;
  padding: 0.6rem 0.4rem 1.2rem;
  border-radius: 1.25rem 1.25rem 0 0.55rem;
  background: linear-gradient(180deg, hsl(230 28% 17% / 0.9), hsl(230 32% 8% / 0.98));
  box-shadow:
    inset 0 1px 0 hsl(0 0% 100% / 0.12),
    0 1.8rem 2.8rem hsl(230 45% 2% / 0.58);
  transform-style: preserve-3d;
}

.stage-performer-riser::before {
  content: "";
  position: absolute;
  inset: -0.65rem;

```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```
border: 1px solid hsl(43 96% 56% / var(--mic-glow));
border-radius: 1.55rem;
box-shadow: 0 0 var(--mic-blur) hsl(43 96% 56% / var(--mic-glow));
opacity: 0.78;
pointer-events: none;
transform: scale(calc(1 + var(--mic-scale)));
transition: transform 100ms linear, box-shadow 100ms linear, border-color 100ms linear;
}

.stage-performer-riser-violet::before {
border-color: hsl(267 84% 70% / var(--mic-glow));
box-shadow: 0 0 var(--mic-blur) hsl(267 84% 70% / var(--mic-glow));
}

.stage-performer-riser[data-mic-live="true"]::before {
animation: stage-mic-ring 0.82s ease-in-out infinite;
}

@keyframes stage-mic-ring {
0%, 100% { opacity: 0.62; }
50% { opacity: 1; }
}

.stage-performer-avatar {
position: relative;
z-index: 1;
display: flex;
height: clamp(3.3rem, 9vw, 5.2rem);
width: clamp(3.3rem, 9vw, 5.2rem);
align-items: center;
justify-content: center;
overflow: hidden;
border: 2px solid hsl(43 96% 70% / 0.72);
border-radius: 999px;
color: hsl(43 96% 76%);
background: radial-gradient(circle at 35% 24%, hsl(0 0% 100% / 0.16), hsl(43 96% 56% / 0.14) 42%, hsl(230 35% 8% / 0.9));
box-shadow: 0 0 22px hsl(43 96% 56% / 0.26);
}

.stage-performer-riser-violet .stage-performer-avatar {
border-color: hsl(267 84% 78% / 0.72);
color: hsl(267 84% 82%);
background: radial-gradient(circle at 35% 24%, hsl(0 0% 100% / 0.16), hsl(267 84% 70% / 0.18) 42%, hsl(230 35% 8% / 0.9));
box-shadow: 0 0 22px hsl(267 84% 70% / 0.3);
}

.stage-invite-partner-button {
position: relative;
z-index: 2;
display: flex;
min-height: 5.5rem;
width: calc(100% - 1rem);
flex-direction: column;
align-items: center;
justify-content: center;
gap: 0.45rem;
border: 1px dashed hsl(267 84% 76% / 0.58);
border-radius: 1rem;
color: hsl(267 84% 86%);
background: hsl(267 84% 70% / 0.08);
box-shadow: inset 0 0 24px hsl(267 84% 70% / 0.08), 0 0 22px hsl(267 84% 70% / 0.16);
font-size: 0.65rem;
font-weight: 800;
letter-spacing: 0.08em;
text-transform: uppercase;
transition: border-color 180ms ease, background-color 180ms ease, box-shadow 180ms ease;
}

.stage-invite-partner-button:hover {
border-color: hsl(267 84% 82% / 0.9);
background: hsl(267 84% 70% / 0.14);
box-shadow: inset 0 0 28px hsl(267 84% 70% / 0.12), 0 0 28px hsl(267 84% 70% / 0.28);
}

.stage-invite-plus {
display: grid;
height: 2.35rem;
width: 2.35rem;
place-items: center;
border: 1px solid hsl(267 84% 82% / 0.55);
border-radius: 999px;
font-size: 1.35rem;
line-height: 1;
}

.stage-performer-avatar img {
height: 100%;
width: 100%;
object-fit: cover;
}

.stage-performer-name,
.stage-performer-status {
position: relative;
z-index: 1;
max-width: 100%;
overflow: hidden;
text-overflow: ellipsis;
white-space: nowrap;
}

.stage-performer-name {
margin-top: 0.5rem;
color: white;
font-size: clamp(0.7rem, 1.7vw, 0.9rem);
font-weight: 800;
}

.stage-performer-status {
display: flex;
align-items: center;
gap: 0.28rem;
margin-top: 0.2rem;
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```

    color: hsl(43 96% 70% / 0.75);
    font-size: 0.55rem;
    font-weight: 800;
    letter-spacing: 0.13em;
    text-transform: uppercase;
  }

.stage-performer-riser-violet .stage-performer-status {
  color: hsl(267 84% 80% / 0.8);
}

.stage-riser-top {
  position: absolute;
  right: 0.9rem;
  bottom: 0.58rem;
  left: 0.9rem;
  height: 0.23rem;
  border-radius: 999px;
  background: hsl(43 96% 64% / 0.58);
  box-shadow: 0 0 12px hsl(43 96% 56% / 0.7);
}

.stage-performer-riser-violet .stage-riser-top {
  background: hsl(267 84% 76% / 0.6);
  box-shadow: 0 0 12px hsl(267 84% 70% / 0.72);
}

.stage-riser-front {
  position: absolute;
  right: 0.2rem;
  bottom: -0.72rem;
  left: 0.2rem;
  height: 0.95rem;
  border: 1px solid hsl(43 96% 56% / 0.32);
  border-top: 0;
  border-radius: 0 0 0.45rem 0.45rem;
  background: linear-gradient(180deg, hsl(43 96% 56% / 0.18), hsl(230 30% 7% / 0.96));
  transform: translateZ(-0.5rem);
}

.stage-performer-riser-violet .stage-riser-front {
  border-color: hsl(267 84% 70% / 0.34);
  background: linear-gradient(180deg, hsl(267 84% 70% / 0.18), hsl(230 30% 7% / 0.96));
}

.stage-floor-front-led {
  position: absolute;
  right: 1.2rem;
  bottom: -0.38rem;
  left: 1.2rem;
  height: 0.25rem;
  border-radius: 999px;
  background: linear-gradient(90deg, transparent, hsl(43 96% 62%), hsl(267 84% 76%), hsl(43 96% 62%), transparent);
  box-shadow: 0 0 16px hsl(43 96% 56% / 0.68);
  transform: translateZ(0.2rem);
}

@keyframes stage-tile-shimmer {
  0%, 100% { background-color: transparent; }
  50% { background-color: hsl(43 96% 56% / 0.08); }
}

.stage-control-rail {
  border-top: 1px solid hsl(0 0% 100% / 0.1);
}

.stage-control-inner {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  margin: 0 auto;
  max-width: 72rem;
  padding: 0.65rem 0.8rem;
  border: 1px solid hsl(43 48% 44% / 0.24);
  border-radius: 1rem;
  background: linear-gradient(145deg, hsl(230 26% 14% / 0.88), hsl(267 24% 12% / 0.76));
  box-shadow: inset 0 1px 0 hsl(0 0% 100% / 0.07), 0 12px 28px hsl(230 45% 2% / 0.42);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
}

.live-performance-stage[data-stage-mode="solo"] .stage-control-inner {
  max-width: none;
}

.pb-safe {
  padding-bottom: max(0.75rem, env(safe-area-inset-bottom, 0px));
}

.stage-progress-slider {
  min-height: 2.75rem;
  height: 2.75rem;
  accent-color: hsl(43 96% 56%);
  cursor: pointer;
}

.stage-record-button {
  display: inline-flex;
  height: 2.25rem;
  width: 2.25rem;
  align-items: center;
  justify-content: center;
  border: 1px solid hsl(0 84% 66% / 0.58);
  border-radius: 999px;
  color: white;
  background: hsl(0 72% 52% / 0.84);
  box-shadow: 0 0 16px hsl(0 84% 60% / 0.24);
}

.stage-record-button:hover {
  background: hsl(0 80% 60%);
  box-shadow: 0 0 24px hsl(0 84% 60% / 0.42);
}
```

05. MAIN STAGE & RESPONSIVE TRANSPORT DOCK (continued)

artifacts/gravelkingpro/src/index.css | source continues without omission

```
}

.stage-record-button-active {
  border-color: hsl(0 0% 100% / 0.8);
  color: hsl(0 70% 45%);
  background: white;
  animation: stage-record-pulse 1.1s ease-in-out infinite;
}

@keyframes stage-record-pulse {
  0%, 100% { box-shadow: 0 0 0 0 hsl(0 84% 60% / 0.35); }
  50% { box-shadow: 0 0 8px hsl(0 84% 60% / 0); }
}

@media (max-width: 640px) {
  .stage-viewport {
    gap: 0.55rem;
    padding-right: 0.7rem;
    padding-left: 0.7rem;
  }

  .stage-jumbotron {
    min-height: 7.9rem;
  }

  .stage-jumbotron-content {
    padding-right: 0.7rem;
    padding-left: 0.7rem;
  }

  .stage-deck-wrap {
    min-height: 9.5rem;
  }

  .stage-deck {
    height: 88%;
    min-height: 9rem;
  }

  .stage-performer-row {
    right: 3%;
    left: 3%;
    gap: 0.7rem;
  }

  .stage-performer-riser {
    min-width: 0;
    min-height: 6.8rem;
    padding-bottom: 0.9rem;
  }

  .stage-performer-status {
    font-size: 0.44rem;
    letter-spacing: 0.08em;
  }

  .stage-control-inner {
    gap: 0.5rem;
    padding-right: 0.55rem;
    padding-left: 0.55rem;
  }

  .stage-control-actions {
    width: 100%;
  }

  .stage-control-actions > button,
  .stage-control-actions > label {
    min-height: 2.75rem;
  }
}

@media (prefers-reduced-motion: reduce) {
  .live-performance-stage[data-playing="true"] .stage-spotlight,
  .live-performance-stage[data-playing="true"] .stage-deck-tile:nth-child(3n),
  .stage-performer-riser[data-mic-live="true"]::before,
  .stage-record-button-active {
    animation: none;
  }
}
```

06. DUET INGRESS & LEAD CAPTURE

artifacts/gravelkingpro/src/pages/duet.tsx | 972 lines | 46,650 bytes | sha256 d6f2058bf26620ba4b167c3544b1ccd6073d07e84b025157929f02276e92e5fc

```
import { useCallback, useEffect, useRef, useState } from "react";
import { Layout } from "@components/layout";
import { Button } from "@components/ui/button";
import { useToast } from "@hooks/use-toast";
import { useAuth } from "@clerk/react";
import { Copy, Link2, Loader2, Mic2, Radio, Square, Users } from "lucide-react";
import { LivePerformanceStage, type StageLyric } from "@components/live-performance-stage";
import { alignDuetBufferTarget, DuetReviewCard } from "@components/stage/DuetRoom";
import { downloadBlob } from "@lib/download";

type RoomRole = "host" | "partner";
type SignalType = "offer" | "answer" | "ice";
type ArtistProfile = {
  artistName: string;
  hometown: string;
  avatarUrl: string | null;
};
type KaraokeTrack = {
  id: string;
  assetUrl: string;
  title: string;
  bpm: number | null;
  duration: number;
  uploaderArtistName: string;
};
type TimedLyricLine = {
  id: string;
  text: string;
  startTimeMs: number;
  endTimeMs: number;
};
type SyncMessage =
  | { kind: "buffer-ready" }
  | { kind: "ready" }
  | { kind: "clock-probe"; t0: number }
  | { kind: "clock-response"; t0: number; t1: number; t2: number }
  | { kind: "start"; targetHostTime: number }
  | { kind: "review-start"; targetHostTime: number }
  | { kind: "community-track"; track: KaraokeTrack; lines: TimedLyricLine[] };

const ROOM_ID_LENGTH = 8;
const PING_MAGIC = 0x042c;

function newRoomId(): string {
  return crypto.randomUUID().replace(/-/g, "").slice(0, ROOM_ID_LENGTH).toUpperCase();
}

function audioContext(): AudioContext {
  return new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });
}

function isSignalType(value: unknown): value is SignalType {
  return value === "offer" || value === "answer" || value === "ice";
}

export default function DuetRoom() {
  const { getToken } = useAuth();
  const { toast } = useToast();
  const [roomId, setRoomId] = useState(() => {
    const fromUrl = new URLSearchParams(window.location.search).get("room");
    return fromUrl?.replace(/["a-zA-Z0-9_-]/g, "").slice(0, 80) || newRoomId();
  });
  const [roomInput, setRoomInput] = useState(roomId);
  const [joined, setJoined] = useState(false);
  const [role, setRole] = useState<RoomRole | null>(null);
  const [peerConnected, setPeerConnected] = useState(false);
  const [channelOpen, setChannelOpen] = useState(false);
  const [trackName, setTrackName] = useState("");
  const [duration, setDuration] = useState(0);
  const [bufferDecoded, setBufferDecoded] = useState(false);
  const [remoteBufferReady, setRemoteBufferReady] = useState(false);
  const [localReady, setLocalReady] = useState(false);
  const [remoteReady, setRemoteReady] = useState(false);
  const [loadingTrack, setLoadingTrack] = useState(false);
  const [syncStatus, setSyncStatus] = useState("Create a room or join a partner.");
  const [rttMs, setRttMs] = useState<number | null>(null);
  const [clockOffsetMs, setClockOffsetMs] = useState<number | null>(null);
  const [isPlaying, setIsPlaying] = useState(false);
  const [playbackPosition, setPlaybackPosition] = useState(0);
  const [localMicAnalyser, setLocalMicAnalyser] = useState<AnalyserNode | null>(null);
  const [remoteMicAnalyser, setRemoteMicAnalyser] = useState<AnalyserNode | null>(null);
  const [localProfile, setLocalProfile] = useState<ArtistProfile | null>(null);
  const [partnerProfile, setPartnerProfile] = useState<ArtistProfile | null>(null);
  const [libraryTracks, setLibraryTracks] = useState<KaraokeTrack[]>([]);
  const [selectedCommunityTrack, setSelectedCommunityTrack] = useState<KaraokeTrack | null>(null);
  const [communityLines, setCommunityLines] = useState<TimedLyricLine[]>([]);
  const [communityLyrics, setCommunityLyrics] = useState("");
  const [uploadTitle, setUploadTitle] = useState("");
  const [uploadBpm, setUploadBpm] = useState("");
  const [uploadFile, setUploadFile] = useState<File | null>(null);
  const [uploadingTrack, setUploadingTrack] = useState(false);
  const [takeRecording, setTakeRecording] = useState(false);
  const [localTakeBlob, setLocalTakeBlob] = useState<Blob | null>(null);
  const [localTakeUrl, setLocalTakeUrl] = useState<string | null>(null);
  const [savingTake, setSavingTake] = useState(false);

  const { current: peerId } = useRef(crypto.randomUUID());
  const roomRef = useRef(roomId);
  const roleRef = useRef<RoomRole | null>(null);
  const remotePeerRef = useRef<string | null>(null);
  const streamRef = useRef<EventSource | null>(null);
  const pcRef = useRef<RTCPeerConnection | null>(null);
  const dataChannelRef = useRef<RTCDataChannel | null>(null);
  const contextRef = useRef<AudioContext | null>(null);
  const bufferRef = useRef<AudioBuffer | null>(null);
  const sourceRef = useRef<AudioBufferSourceNode | null>(null);
  const playbackAnalyserRef = useRef<AnalyserNode | null>(null);
  const playbackStartContextTimeRef = useRef<number | null>(null);
  const micStreamRef = useRef<MediaStream | null>(null);
  const micContextRef = useRef<AudioContext | null>(null);
  const remoteMicStreamIdRef = useRef<string | null>(null);
  const pendingIceCandidatesRef = useRef<RTCIceCandidateInit[]>([]);
  const pingTimerRef = useRef<number | null>(null);
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
const pingSequenceRef = useRef(0);
const sentPingsRef = useRef(new Map<number, number>());
const rtthalfRef = useRef(0);
const clockOffsetRef = useRef(0);
const localReadyRef = useRef(false);
const remoteReadyRef = useRef(false);
const remoteBufferReadyRef = useRef(false);
const scheduledRef = useRef(false);
const takeRecorderRef = useRef<MediaRecorder | null>(null);
const takeChunksRef = useRef<Blob[]>([]);
const takeStartTimerRef = useRef<number | null>(null);
const reviewAudioRef = useRef<HTMLAudioElement | null>(null);
const trackFileRef = useRef<File | null>(null);
const reviewingRef = useRef(false);

useEffect(() => {
  void Promise.all([
    fetch("/api/user/profile", { credentials: "include" })
      .then((response) => response.ok ? response.json() as Promise<{ profile: ArtistProfile }> : null)
      .catch(() => null),
    fetch("/api/karaoke/tracks", { credentials: "include" })
      .then((response) => response.ok ? response.json() as Promise<{ tracks: KaraokeTrack[] }> : null)
      .catch(() => null),
  ]).then(([profileData, libraryData]) => {
    if (profileData?.profile) setLocalProfile(profileData.profile);
    if (libraryData?.tracks) setLibraryTracks(libraryData.tracks);
  });
}, []);

useEffect(() => {
  if (!localTakeBlob) {
    setLocalTakeUrl(null);
    return;
  }
  const url = URL.createObjectURL(localTakeBlob);
  setLocalTakeUrl(url);
  return () => URL.revokeObjectURL(url);
}, [localTakeBlob]);

const requestHeaders = useCallback(async (json = true): Promise<Record<string, string>> => {
  const token = await getToken();
  return {
    ...(json ? { "Content-Type": "application/json" } : {}),
    ...(token ? { Authorization: `Bearer ${token}` } : {}),
  };
}, [getToken]);

const sendSignal = useCallback(async (type: SignalType, payload: unknown) => {
  const target = remotePeerRef.current;
  if (!target) return;
  await fetch(`/api/duet/room/${encodeURIComponent(roomRef.current)}/signal`, {
    method: "POST",
    headers: await requestHeaders(),
    credentials: "include",
    body: JSON.stringify({ fromPeerId: peerId, toPeerId: target, type, payload }),
  });
}, [peerId, requestHeaders]);

const sendControl = useCallback((message: SyncMessage) => {
  const channel = dataChannelRef.current;
  if (channel?.readyState === "open") channel.send(JSON.stringify(message));
}, []);

const ensureLocalMic = useCallback(async (): Promise<MediaStream | null> => {
  if (micStreamRef.current) return micStreamRef.current;
  if (!navigator.mediaDevices?.getUserMedia) return null;
  try {
    const stream = await navigator.mediaDevices.getUserMedia({
      audio: {
        echoCancellation: false,
        noiseSuppression: false,
        autoGainControl: false,
      },
    });
  } catch {
    // Microphone visuals are optional; the synchronized backing track can still play.
    return null;
  }
}, []);

const attachRemoteMic = useCallback((stream: MediaStream) => {
  if (remoteMicStreamIdRef.current === stream.id) return;
  const ctx = micContextRef.current ?? new AudioContext({ latencyHint: "interactive", sampleRate: 44100 });
  micContextRef.current = ctx;
  if (ctx.state === "suspended") await ctx.resume();
  const source = ctx.createMediaStreamSource(stream);
  const analyser = ctx.createAnalyser();
  analyser.fftSize = 1024;
  analyser.smoothingTimeConstant = 0.72;
  source.connect(analyser);
  micStreamRef.current = stream;
  setLocalMicAnalyser(analyser);
  return stream;
}, []);

const stopLocalTake = useCallback(() => {
  const recorder = takeRecorderRef.current;
  if (recorder && recorder.state !== "inactive") recorder.stop();
  takeRecorderRef.current = null;
  setTakeRecording(false);
}, []);

const startLocalTake = useCallback(() => {
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
const stream = micStreamRef.current;
if (!stream || typeof MediaRecorder === "undefined") return;
stopLocalTake();
takeChunksRef.current = [];
const mimeType = ["audio/webm;codecs=opus", "audio/webm", "audio/mp4"].find((type) => MediaRecorder.isTypeSupported(type)) ?? "";
try {
  const recorder = mimeType ? new MediaRecorder(stream, { mimeType }) : new MediaRecorder(stream);
  recorder.ondataavailable = (event) => { if (event.data.size > 0) takeChunksRef.current.push(event.data); };
  recorder.onstop = () => {
    const blob = new Blob(takeChunksRef.current, { type: recorder.mimeType || "audio/webm" });
    if (blob.size > 0) setLocalTakeBlob(blob);
  };
  recorder.start(750);
  takeRecorderRef.current = recorder;
  setLocalTakeBlob(null);
  setTakeRecording(true);
} catch {
  setSyncStatus("Your browser could not open an in-memory take recorder.");
}
}, [stopLocalTake]);

const scheduleReviewAudio = useCallback((targetPerformanceMs: number) => {
  const audio = reviewAudioRef.current;
  if (!audio) return;
  const wait = Math.max(0, targetPerformanceMs - performance.now());
  window.setTimeout(() => {
    if (!reviewingRef.current) return;
    audio.currentTime = 0;
    void audio.play().catch(() => {});
  }, wait);
}, []);

const scheduleTrack = useCallback((targetPerformanceMs: number, review = false) => {
  const buffer = bufferRef.current;
  if (!buffer) return;
  const ctx = contextRef.current ?? audioContext();
  contextRef.current = ctx;
  if (ctx.state === "suspended") void ctx.resume();
  sourceRef.current?.stop();
  const source = ctx.createBufferSource();
  source.buffer = buffer;
  const analyser = ctx.createAnalyser();
  analyser.fftSize = 1024;
  analyser.smoothingTimeConstant = 0.8;
  source.connect(analyser);
  analyser.connect(ctx.destination);
  const delaySeconds = Math.max(0, targetPerformanceMs - performance.now()) / 1000;
  const startAt = ctx.currentTime + delaySeconds;
  source.start(startAt);
  source.onended = () => {
    stopLocalTake();
    if (reviewingRef.current) {
      reviewingRef.current = false;
      reviewAudioRef.current?.pause();
    }
    playbackStartContextTimeRef.current = null;
    playbackAnalyserRef.current = null;
    scheduledRef.current = false;
    setPlaybackPosition(0);
    setIsPlaying(false);
  };
  sourceRef.current = source;
  playbackAnalyserRef.current = analyser;
  playbackStartContextTimeRef.current = startAt;
  scheduledRef.current = true;
  setPlaybackPosition(0);
  setIsPlaying(true);
  if (review) {
    reviewingRef.current = true;
    scheduleReviewAudio(targetPerformanceMs);
  } else {
    takeStartTimerRef.current = window.setTimeout(() => startLocalTake(), Math.max(0, targetPerformanceMs - performance.now()));
  }
  setSyncStatus("Both singers are locked. Playback is synchronized.");
}, [scheduleReviewAudio, startLocalTake, stopLocalTake]);

useEffect(() => {
  if (!isPlaying) return;
  let frame = 0;
  const tick = () => {
    const ctx = contextRef.current;
    const startAt = playbackStartContextTimeRef.current;
    if (ctx && startAt != null) {
      setPlaybackPosition(Math.max(0, ctx.currentTime - startAt));
    }
    frame = requestAnimationFrame(tick);
  };
  frame = requestAnimationFrame(tick);
  return () => cancelAnimationFrame(frame);
}, [isPlaying]);

const maybeScheduleHostStart = useCallback(() => {
  if (
    roleRef.current !== "host" ||
    !localReadyRef.current ||
    !remoteReadyRef.current ||
    !bufferRef.current ||
    !remoteBufferReadyRef.current ||
    scheduledRef.current
  ) return;
  const targetHostTime = performance.now() + 5000 + rttHalfRef.current;
  sendControl({ kind: "start", targetHostTime });
  scheduleTrack(targetHostTime);
  setSyncStatus("Starting both tracks in five seconds...");
}, [scheduleTrack, sendControl]);

const startPingLoop = useCallback(() => {
  if (pingTimerRef.current != null) return;
  const tick = () => {
    const channel = dataChannelRef.current;
    if (channel?.readyState === "open") {
      const sequence = ((roleRef.current === "partner" ? 0x8000 : 0) | (pingSequenceRef.current++ & 0xffff)) & 0xffff;
      const packet = new Uint8Array(4);
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
    packet[0] = (PING_MAGIC >> 8) & 0xff;
    packet[1] = PING_MAGIC & 0xff;
    packet[2] = (sequence >> 8) & 0xff;
    packet[3] = sequence & 0xff;
    sentPingsRef.current.set(sequence, performance.now());
    channel.send(packet);
    sendControl({ kind: "clock-probe", t0: performance.now() });
  }
  pingTimerRef.current = window.setTimeout(tick, 200);
};
tick();
}, [sendControl]);

const attachDataChannel = useCallback((channel: RTCDataChannel) => {
  dataChannelRef.current = channel;
  channel.binaryType = "arraybuffer";
  channel.onopen = () => {
    setChannelOpen(true);
    startPingLoop();
    if (bufferRef.current) sendControl({ kind: "buffer-ready" });
    setSyncStatus("Direct duet link ready. Load the same backing track on both devices.");
  };
  channel.onclose = () => {
    setChannelOpen(false);
    if (pingTimerRef.current != null) window.clearTimeout(pingTimerRef.current);
    pingTimerRef.current = null;
  };
  channel.onerror = () => setSyncStatus("The direct link reported an error. Try rejoining the room.");
  channel.onmessage = (event) => {
    if (event.data instanceof ArrayBuffer || ArrayBuffer.isView(event.data)) {
      const bytes = event.data instanceof ArrayBuffer ? new Uint8Array(event.data) : new Uint8Array(event.data.buffer);
      if (bytes.length !== 4 || bytes[0] !== (PING_MAGIC >> 8) || bytes[1] !== (PING_MAGIC & 0xff)) return;
      const sequence = (bytes[2] << 8) | bytes[3];
      const sentAt = sentPingsRef.current.get(sequence);
      if (sentAt != null) {
        const rtt = performance.now() - sentAt;
        sentPingsRef.current.delete(sequence);
        rttHalfRef.current = rtt / 2;
        setRttMs(Math.round(rtt));
      } else {
        channel.send(bytes.buffer as ArrayBuffer);
      }
      return;
    }
    let message: SyncMessage;
    try { message = JSON.parse(String(event.data)) as SyncMessage; } catch { return; }
    if (message.kind === "buffer-ready") {
      remoteBufferReadyRef.current = true;
      setRemoteBufferReady(true);
      setSyncStatus("Both backing tracks are decoded. Press Ready when you are set.");
    } else if (message.kind === "ready") {
      remoteReadyRef.current = true;
      setRemoteReady(true);
      maybeScheduleHostStart();
    } else if (message.kind === "clock-probe") {
      const t1 = performance.now();
      channel.send(JSON.stringify({ kind: "clock-response", t0: message.t0, t1, t2: performance.now() }));
    } else if (message.kind === "clock-response") {
      const t3 = performance.now();
      const offset = ((message.t1 - message.t0) + (message.t2 - t3)) / 2;
      clockOffsetRef.current = offset;
      setClockOffsetMs(Math.round(offset));
    } else if (message.kind === "start") {
      const alignment = alignDuetBufferTarget(message.targetHostTime, performance.now(), clockOffsetRef.current);
      scheduleTrack(alignment.localTargetTimeMs);
    } else if (message.kind === "review-start") {
      const alignment = alignDuetBufferTarget(message.targetHostTime, performance.now(), clockOffsetRef.current);
      scheduleTrack(alignment.localTargetTimeMs, true);
    } else if (message.kind === "community-track") {
      setSelectedCommunityTrack(message.track);
      setCommunityLines(message.lines);
      setCommunityLyrics(message.lines.map((line) => line.text).join("\n"));
      setSyncStatus(`Community track selected: ${message.track.title}. Load it locally to join playback.`);
    }
  };
}, [maybeScheduleHostStart, scheduleTrack, sendControl, startPingLoop]);

const createPeerConnection = useCallback(async (remotePeerId: string) => {
  if (pcRef.current) return;
  await ensureLocalMic();
  remotePeerRef.current = remotePeerId;
  const pc = new RTCPeerConnection({
    iceServers: [{ urls: "stun:stun.l.google.com:19302" }],
  });
  pcRef.current = pc;
  for (const track of micStreamRef.current?.getTracks() ?? []) {
    pc.addTrack(track, micStreamRef.current!);
  }
  pc.onicecandidate = (event) => {
    if (event.candidate) void sendSignal("ice", event.candidate.toJSON());
  };
  pc.ontrack = (event) => {
    const stream = event.streams[0] ?? new MediaStream([event.track]);
    if (stream) attachRemoteMic(stream);
  };
  pc.onconnectionstatechange = () => {
    if (pc.connectionState === "connected") setPeerConnected(true);
    if (pc.connectionState === "failed" || pc.connectionState === "closed") setPeerConnected(false);
  };
  pc.ondatachannel = (event) => attachDataChannel(event.channel);
  if (roleRef.current === "host") {
    const channel = pc.createDataChannel("duet-sync", { ordered: false, maxRetransmits: 0 });
    attachDataChannel(channel);
    const offer = await pc.createOffer();
    await pc.setLocalDescription(offer);
    await sendSignal("offer", offer);
  }
}, [attachDataChannel, attachRemoteMic, ensureLocalMic, sendSignal]);

const handleSignal = useCallback(async (message: { fromPeerId?: string; type?: unknown; payload?: unknown }) => {
  if (!message.fromPeerId || !isSignalType(message.type)) return;
  await createPeerConnection(message.fromPeerId);
  const pc = pcRef.current;
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
    if (!pc) return;
    if (message.type === "offer") {
      await pc.setRemoteDescription(message.payload as RTCSessionDescriptionInit);
      for (const candidate of pendingIceCandidatesRef.current.splice(0)) {
        await pc.addIceCandidate(candidate);
      }
      const answer = await pc.createAnswer();
      await pc.setLocalDescription(answer);
      await sendSignal("answer", answer);
    } else if (message.type === "answer") {
      await pc.setRemoteDescription(message.payload as RTCSessionDescriptionInit);
      for (const candidate of pendingIceCandidatesRef.current.splice(0)) {
        await pc.addIceCandidate(candidate);
      }
    } else if (message.type === "ice" && message.payload) {
      if (!pc.remoteDescription) {
        pendingIceCandidatesRef.current.push(message.payload as RTCIceCandidateInit);
        return;
      }
      await pc.addIceCandidate(message.payload as RTCIceCandidateInit);
    }
  }, [createPeerConnection, sendSignal]);

const connectToRoom = useCallback(async () => {
  const nextRoom = roomInput.trim().replace(/["a-zA-Z0-9_-]/g, "").slice(0, 80);
  if (!nextRoom) return;
  setSyncStatus("Joining room...");
  const response = await fetch(`/api/duet/room/${encodeURIComponent(nextRoom)}/join`, {
    method: "POST",
    headers: await requestHeaders(),
    credentials: "include",
    body: JSON.stringify({ peerId }),
  });
  const data = await response.json().catch(() => ({})) as {
    error?: string;
    role?: RoomRole;
    otherPeerId?: string | null;
    profile?: ArtistProfile | null;
    otherPeerProfile?: ArtistProfile | null;
  };
  if (!response.ok || !data.role) {
    setSyncStatus(data.error || "Could not join this room.");
    toast({ title: "Duet room unavailable", description: data.error || "Try a new room.", variant: "destructive" });
    return;
  }
  roomRef.current = nextRoom;
  roleRef.current = data.role;
  setRoomId(nextRoom);
  setRole(data.role);
  setLocalProfile(data.profile ?? localProfile);
  setPartnerProfile(data.otherPeerProfile ?? null);
  setJoined(true);
  window.history.replaceState({}, "", `/duet?room=${encodeURIComponent(nextRoom)}`);
  const stream = new EventSource(`/api/duet/room/${encodeURIComponent(nextRoom)}/events?peerId=${encodeURIComponent(peerId)}`);
  streamRef.current = stream;
  stream.addEventListener("peer-joined", (event) => {
    const joinedPeer = JSON.parse(event as MessageEvent).data as { peerId?: string; profile?: ArtistProfile | null };
    if (joinedPeer.peerId) {
      if (joinedPeer.peerId) {
        remotePeerRef.current = joinedPeer.peerId;
        setPartnerProfile(joinedPeer.profile ?? null);
        setPeerConnected(true);
        void createPeerConnection(joinedPeer.peerId);
      }
    }
  });
  stream.addEventListener("signal", (event) => {
    void handleSignal(JSON.parse(event as MessageEvent).data as { fromPeerId?: string; type?: unknown; payload?: unknown });
  });
  stream.addEventListener("peer-left", () => {
    setPeerConnected(false);
    setPartnerProfile(null);
    setChannelOpen(false);
    setRemoteBufferReady(false);
    remoteBufferReadyRef.current = false;
    setRemoteReady(false);
    remoteReadyRef.current = false;
    setSyncStatus("Your partner left the room.");
  });
  if (data.otherPeerId) {
    remotePeerRef.current = data.otherPeerId;
    setPeerConnected(true);
    await createPeerConnection(data.otherPeerId);
  } else {
    setSyncStatus(`You are the ${data.role}. Share the room link with your duet partner.`);
  }
}, [createPeerConnection, handleSignal, localProfile, peerId, requestHeaders, roomInput, toast]);

const loadTrack = useCallback(async (file: File | null) => {
  if (!file) return;
  setLoadingTrack(true);
  setBufferDecoded(false);
  setLocalReady(false);
  localReadyRef.current = false;
  scheduledRef.current = false;
  try {
    const ctx = contextRef.current ?? audioContext();
    contextRef.current = ctx;
    if (ctx.state === "suspended") await ctx.resume();
    const decoded = await ctx.decodeAudioData(await file.arrayBuffer()).slice(0);
    bufferRef.current = decoded;
    trackFileRef.current = file;
    setTrackName(file.name);
    setDuration(decoded.duration);
    setBufferDecoded(true);
    if (dataChannelRef.current?.readyState === "open") sendControl({ kind: "buffer-ready" });
    setSyncStatus(remoteBufferReadyRef.current ? "Both backing tracks are decoded. Press Ready." : "Track decoded locally. Waiting for your partner's decoded buffer.");
  } catch {
    bufferRef.current = null;
    setSyncStatus("That audio file could not be decoded in this browser.");
    toast({ title: "Track decode failed", description: "Choose a WAV, MP3, M4A, or WebM backing track.", variant: "destructive" });
  } finally {
    setLoadingTrack(false);
  }
}, [sendControl, toast]);
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
const selectCommunityTrack = useCallback(async (track: KaraokeTrack) => {
  setSelectedCommunityTrack(track);
  try {
    const response = await fetch(track.assetUrl, { credentials: "include" });
    if (!response.ok) throw new Error("Track download failed");
    const blob = await response.blob();
    await loadTrack(new File([blob], `${track.title}.audio`, { type: blob.type || "audio/mpeg" }));
    setTrackName(track.title);
    setSyncStatus(`Loaded ${track.title}. Both singers should choose the same community track.`);
  } catch {
    toast({ title: "Community track unavailable", description: "The public audio asset could not be loaded.", variant: "destructive" });
  }
}, [loadTrack, toast]);

const broadcastCommunityTrack = useCallback(async () => {
  if (!selectedCommunityTrack || !channelOpen) return;
  let lines = communityLines;
  if (communityLyrics.trim()) {
    const form = new FormData();
    form.append("lyrics", communityLyrics);
    form.append("durationMs", String(selectedCommunityTrack.duration * 1000));
    const response = await fetch("/api/karaoke/transcribe", { method: "POST", body: form, credentials: "include" });
    if (response.ok) {
      const data = await response.json() as { lines?: TimedLyricLine[] };
      lines = data.lines ?? [];
      setCommunityLines(lines);
    }
  }
  sendControl({ kind: "community-track", track: selectedCommunityTrack, lines });
  toast({ title: "Track shared", description: "The selected community track and lyric timestamps were sent to your partner." });
}, [channelOpen, communityLines, communityLyrics, selectedCommunityTrack, sendControl, toast]);

const uploadCommunityTrack = useCallback(async () => {
  if (!uploadFile || !uploadTitle.trim()) return;
  setUploadingTrack(true);
  try {
    const form = new FormData();
    form.append("audio", uploadFile);
    form.append("title", uploadTitle.trim());
    if (uploadBpm.trim()) form.append("bpm", uploadBpm.trim());
    const response = await fetch("/api/karaoke/tracks", { method: "POST", body: form, credentials: "include" });
    const data = await response.json().catch(() => ({})) as { track?: KaraokeTrack; error?: string };
    if (!response.ok || !data.track) throw new Error(data.error || "Upload failed");
    setLibraryTracks((current) => [data.track!, ...current]);
    setUploadTitle("");
    setUploadBpm("");
    setUploadFile(null);
    toast({ title: "Track added to the public library", description: "It is now available to all duet rooms." });
  } catch (error) {
    toast({ title: "Track upload failed", description: error instanceof Error ? error.message : "Try again.", variant: "destructive" });
  } finally {
    setUploadingTrack(false);
  }
}, [uploadBpm, uploadFile, uploadTitle, toast]);

const setReady = useCallback(() => {
  if (!bufferRef.current || !remoteBufferReadyRef.current || !channelOpen) return;
  localReadyRef.current = true;
  setLocalReady(true);
  sendControl({ kind: "ready" });
  setSyncStatus(remoteReadyRef.current ? "Both singers are ready. Host is scheduling playback." : "Ready sent. Waiting for your partner.");
  maybeScheduleHostStart();
}, [channelOpen, maybeScheduleHostStart, sendControl]);

const stopPlayback = useCallback(() => {
  if (takeStartTimerRef.current != null) window.clearTimeout(takeStartTimerRef.current);
  takeStartTimerRef.current = null;
  stopLocalTake();
  reviewingRef.current = false;
  reviewAudioRef.current?.pause();
  try { sourceRef.current?.stop(); } catch { /* already ended */ }
  sourceRef.current = null;
  playbackAnalyserRef.current = null;
  playbackStartContextTimeRef.current = null;
  scheduledRef.current = false;
  setPlaybackPosition(0);
  setIsPlaying(false);
  setLocalReady(false);
  localReadyRef.current = false;
  setSyncStatus("Playback stopped. Both singers can ready again.");
}, [stopLocalTake]);

const playReview = useCallback(() => {
  if (!localTakeBlob || !bufferRef.current || !channelOpen) return;
  if (roleRef.current === "host") {
    const targetHostTime = performance.now() + 2500 + rttHalfRef.current;
    sendControl({ kind: "review-start", targetHostTime });
    scheduleTrack(targetHostTime, true);
    setSyncStatus("Review playback is synchronized for both singers.");
  } else {
    setSyncStatus("The host controls synchronized review playback.");
  }
}, [channelOpen, localTakeBlob, scheduleTrack, sendControl]);

const saveReviewToVault = useCallback(async () => {
  if (!localTakeBlob || !trackFileRef.current) return;
  setSavingTake(true);
  try {
    const form = new FormData();
    form.append("tracks", trackFileRef.current, trackFileRef.current.name);
    form.append("tracks", localTakeBlob, "duet-vocal-take.webm");
    form.append("arrangement", "Layer");
    // Saving a duet mix is an explicit user-owned audio submission to the
    // studio ingest route, which requires the ownership warranty.
    form.append("author_assertion", "true");
    const mixResponse = await fetch("/api/kernel/studio-mix", { method: "POST", body: form, credentials: "include" });
    if (!mixResponse.ok) throw new Error((await mixResponse.text()).catch(() => "") || `Mixdown failed (${mixResponse.status})`);
    const vaultForm = new FormData();
    vaultForm.append("audio", await mixResponse.blob(), "duet-performance.wav");
    vaultForm.append("title", `${trackName || "Duet"} - duet performance`);
    const vaultResponse = await fetch("/api/duet/review", { method: "POST", body: vaultForm, credentials: "include" });
    const result = await vaultResponse.json().catch(() => ({})) as { error?: string };
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
    if (!vaultResponse.ok) throw new Error(result.error || `Vault save failed (${vaultResponse.status})`);
    toast({ title: "Saved to Vault", description: "The combined backing and vocal performance is private to your library." });
  } catch (error) {
    toast({ title: "Could not save review", description: error instanceof Error ? error.message : "Try again.", variant: "destructive" });
  } finally {
    setSavingTake(false);
  }
}, [localTakeBlob, toast, trackName]);

const downloadTake = useCallback(() => {
  if (!localTakeBlob) return;
  const safeName = (trackName || "duet-take")
    .replace(/[/\?a-z0-9]+/gi, "-.")
    .replace(/~|-|$/g, "")
    .toLowerCase() || "duet-take";
  downloadBlob(localTakeBlob, `${safeName}.webm`);
}, [localTakeBlob, trackName]);

const leaveRoom = useCallback(() => {
  void fetch(`/api/duet/room/${encodeURIComponent(roomRef.current)}/leave`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    credentials: "include",
    body: JSON.stringify({ peerId }),
  });
  streamRef.current?.close();
  pcRef.current?.close();
  if (takeStartTimerRef.current != null) window.clearTimeout(takeStartTimerRef.current);
  takeStartTimerRef.current = null;
  stopLocalTake();
  reviewAudioRef.current?.pause();
  try { sourceRef.current?.stop(); } catch { /* already ended */ }
  micStreamRef.current?.getTracks().forEach((track) => track.stop());
  void micContextRef.current?.close();
  if (pingTimerRef.current != null) window.clearTimeout(pingTimerRef.current);
  streamRef.current = null;
  pcRef.current = null;
  dataChannelRef.current = null;
  micStreamRef.current = null;
  micContextRef.current = null;
  remoteMicStreamIdRef.current = null;
  pendingIceCandidatesRef.current = [];
  playbackAnalyserRef.current = null;
  playbackStartContextTimeRef.current = null;
  setLocalMicAnalyser(null);
  setRemoteMicAnalyser(null);
  setPlaybackPosition(0);
  setJoined(false);
  setRole(null);
  setPeerConnected(false);
  setChannelOpen(false);
  setLocalReady(false);
  setRemoteReady(false);
  setRemoteBufferReady(false);
  localReadyRef.current = false;
  remoteReadyRef.current = false;
  remoteBufferReadyRef.current = false;
  scheduledRef.current = false;
  setIsPlaying(false);
  setSyncStatus("Create a room or join a partner.");
}, [peerId, stopLocalTake]);

useEffect(() => () => leaveRoom(), [leaveRoom]);

const roomLink = `${window.location.origin}${import.meta.env.BASE_URL.endsWith("/") ? import.meta.env.BASE_URL : `${import.meta.env.BASE_URL}/`}stage/duet/${encodeURIComponent(roomId)}`;
const readyEnabled = joined && channelOpen && bufferDecoded && remoteBufferReady && !localReady && !isPlaying;
const stageLyrics: StageLyric[] = communityLines
  .map((line) => ({
    id: line.id,
    text: line.text,
    startTimeMs: line.startTimeMs,
    endTimeMs: Math.max(line.startTimeMs, line.endTimeMs),
  })))
  .filter((line) => line.text.trim() && line.endTimeMs > line.startTimeMs);
const localIsHost = role === "host";

if (isPlaying) {
  return (
    <
      <audio ref={reviewAudioRef} src={localTakeUrl ?? undefined} preload="auto" className="hidden" />
      <LivePerformanceStage
        title={trackName || "Duet Room"}
        subtitle="Two-person synchronized room"
        trackName={trackName}
        isPlaying={isPlaying}
        currentTimeSec={playbackPosition}
        durationSec={duration}
        lyrics={stageLyrics}
        backIngAnalyser={playbackAnalyserRef.current}
        onExit={stopPlayback}
        onStop={stopPlayback}
        performers={
          [
            {
              id: localIsHost ? "host" : "partner",
              label: localProfile?.artistName || (localIsHost ? "Host" : "Partner"),
              name: localProfile?.artistName || (localIsHost ? "Host" : "Partner"),
              avatarUrl: localProfile?.avatarUrl,
              accent: localIsHost ? "gold" : "violet",
              analyser: localMicAnalyser,
              status: localMicAnalyser ? (takeRecording ? "Recording in RAM" : "Live mic") : "Mic unavailable",
            },
            {
              id: localIsHost ? "partner" : "host",
              label: partnerProfile?.artistName || (localIsHost ? "Partner" : "Host"),
              name: partnerProfile?.artistName || (localIsHost ? "Partner" : "Host"),
              avatarUrl: partnerProfile?.avatarUrl,
              accent: localIsHost ? "violet" : "gold",
              analyser: remoteMicAnalyser,
              status: remoteMicAnalyser ? "Live mic" : "Waiting for mic",
            },
          ],
        }
      />
    </>
  )
}
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
    });
  }

  return (
    <Layout>
      <div className="mx-auto max-w-4xl space-y-8">
        <audio ref={reviewAudioRef} src={localTakeUrl ?? undefined} preload="auto" className="hidden" />
        {localTakeBlob && !isPlaying && (
          <DuetReviewCard
            trackName={trackName}
            durationSec={duration}
            saving={savingTake}
            onPlayReview={playReview}
            onDownloadTake={downloadTake}
            onSaveToVault={() => void saveReviewToVault()}
            onRetake={() => { setLocalTakeBlob(null); stopPlayback(); }}
          />
        )}
      </div>
      <header className="space-y-3">
        <div className="flex items-center gap-2 text-amber-400 text-xs uppercase tracking-[0.2em]">
          <Radio className="h-4 w-4" /> Two-person synchronized room
        </div>
        <h1 className="text-3xl font-semibold tracking-tight">Duet Room</h1>
        <p className="max-w-2xl text-sm text-muted-foreground">
          Load the same backing track on both devices, establish a direct peer link, and start together on a shared clock.
          The room never carries audio; it only relays SDP and ICE setup messages.
        </p>
      </header>

      <section className="grid gap-4 sm:grid-cols-2">
        {[{ label: "You", profile: localProfile }, { label: "Partner", profile: partnerProfile }].map(({ label, profile }) => {
          const name = profile?.artistName || (label === "You" ? "Guest singer" : peerConnected ? "Partner connected" : "Waiting for partner");
          const initials = name.split(/s+/).map((part) => part[0]).join("").slice(0, 2).toUpperCase();
          return (
            <div key={label} className="flex items-center gap-3 rounded-2xl border border-border/50 bg-card/40 p-4">
              {profile?.avatarUrl ? (
                <img src={profile.avatarUrl} alt={name} className="h-12 w-12 rounded-full border border-amber-500/30 object-cover" />
              ) : (
                <div className="flex h-12 w-12 items-center justify-center rounded-full border border-amber-500/30 bg-amber-500/10 font-semibold text-amber-300">
                  {initials || "GK"}
                </div>
              )}
              <div className="min-w-0">
                <div className="text-[10px] uppercase tracking-wider text-muted-foreground">{label}</div>
                <div className="truncate font-semibold">{name}</div>
                <div className="truncate text-xs text-muted-foreground">{profile?.hometown || "Artist profile"}</div>
              </div>
              <span className="ml-auto rounded-full border border-amber-500/20 px-2 py-1 text-[10px] text-amber-300">Artist</span>
            </div>
          );
        })];
      </section>

      <section className="grid gap-4 md:grid-cols-[1fr auto]">
        <div className="rounded-2xl border border-border/50 bg-card/40 p-5 space-y-4">
          <div className="flex items-center justify-between gap-3">
            <div>
              <div className="text-xs uppercase tracking-wide text-muted-foreground">Room code</div>
              <div className="mt-1 font-mono text-2xl tracking-[0.18em]">{roomId}</div>
            </div>
            <div className="flex items-center gap-2 text-xs text-muted-foreground">
              <Users className="h-4 w-4" /> {joined ? `${role === "host" ? "Host" : "Partner"} · ${peerConnected ? "2/2" : "1/2"}` : "Not connected"}
            </div>
          </div>
          {!joined && (
            <div className="flex flex-col gap-2 sm:flex-row">
              <input
                value={roomInput}
                onChange={(event) => setRoomInput(event.target.value.toUpperCase())}
                aria-label="Duet room code"
                className="min-w-0 flex-1 rounded-lg border border-border/60 bg-background px-3 py-2 font-mono text-sm tracking-widest"
              />
              <Button onClick={() => void connectToRoom()} className="bg-amber-500 text-black hover:bg-amber-400">
                Join room
              </Button>
            </div>
          )}
          {joined && (
            <div className="flex flex-wrap gap-2">
              <Button variant="outline" onClick={() => void navigator.clipboard?.writeText(roomLink)}>
                <Copy className="mr-2 h-4 w-4" /> Copy invite link
              </Button>
              <Button variant="ghost" onClick={leaveRoom}>Leave room</Button>
            </div>
          )}
          <div className="rounded-lg bg-black/20 px-3 py-2 text-xs text-muted-foreground">{syncStatus}</div>
        </div>
        <div>
          <div className="rounded-2xl border border-border/50 bg-card/40 p-5 text-sm space-y-3 min-w-[210px]">
            <div className="flex items-center gap-2"><Link2 className="h-4 w-4 text-amber-400" /> Share this room</div>
            <div className="break-all text-xs text-muted-foreground">{roomLink}</div>
            <div className="text-xs text-muted-foreground">Room capacity is strictly capped at Host + Partner.</div>
          </div>
        </section>

        <section className="rounded-2xl border border-amber-500/25 bg-amber-500/5 p-5 space-y-5">
          <div>
            <h2 className="font-semibold">Community karaoke library</h2>
            <p className="text-xs text-muted-foreground">Public backing tracks persist for every session. Select one, optionally paste lyrics for deterministic timestamps, then share it with your partner.</p>
          </div>
          <div>
            <div>
              <div className="grid gap-3 md:grid-cols-[1fr auto]">
                <div className="space-y-2">
                  <label className="text-[10px] uppercase tracking-wider text-muted-foreground">Available tracks</label>
                  <select
                    value={selectedCommunityTrack?.id ?? ""}
                    onChange={(event) => {
                      const track = libraryTracks.find((candidate) => candidate.id === event.target.value);
                      if (track) void selectCommunityTrack(track);
                    }}
                    className="w-full rounded-lg border border-border/60 bg-background px-3 py-2 text-sm"
                  >
                    <option value="">Choose a public backing track</option>
                    {libraryTracks.map((track) => (
                      <option key={track.id} value={track.id}>{track.title} · {track.uploaderArtistName} · {Math.round(track.duration)}s</option>
                    ))}
                  </select>
                </div>
              </div>
            </div>
          </div>
        </section>
      </Layout>
    );
  }
}
```

06. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/pages/duet.tsx | source continues without omission

```
    )))
  </select>
  {selectedCommunityTrack && <p className="text-xs text-amber-300">{selectedCommunityTrack.title} · {selectedCommunityTrack.bpm} BPM · ` : ""}{communityLines.length} timed lines</p>}
</div>
<Button variant="outline" disabled={!selectedCommunityTrack || !channelOpen} onClick={() => void broadcastCommunityTrack()}>
  Share with partner
</Button>
</div>
<textarea
  value={communityLyrics}
  onChange={(event) => setCommunityLyrics(event.target.value)}
  placeholder="Optional pasted lyrics. Each non-empty line becomes a timed bar when shared."
  className="min-h-20 w-full rounded-lg border border-border/60 bg-background px-3 py-2 text-sm"
/>
<div className="grid gap-2 md:grid-cols-[1fr 120px auto]">
  <input value={uploadTitle} onChange={(event) => setUploadTitle(event.target.value)} placeholder="Upload title" maxLength={200} className="rounded-lg border border-border/60 bg-background px-3 py-2 text-sm" />
  <input value={uploadBpm} onChange={(event) => setUploadBpm(event.target.value)} placeholder="BPM" inputMode="numeric" className="rounded-lg border border-border/60 bg-background px-3 py-2 text-sm" />
  <label className="flex cursor-pointer items-center justify-center rounded-lg border border-border/60 px-3 py-2 text-xs hover:border-amber-500/50">
    {uploadFile ? uploadFile.name : "Choose audio"}
    <input type="file" accept="audio/*,.m4a,.wav,.mp3,.webm" className="hidden" onChange={(event) => setUploadFile(event.target.files?.[0] ?? null)} />
  </label>
</div>
<Button variant="secondary" disabled={!uploadFile || !uploadTitle.trim() || uploadingTrack} onClick={() => void uploadCommunityTrack()}>
  {uploadingTrack ? <Loader2 className="mr-2 h-4 w-4 animate-spin" /> : <Mic2 className="mr-2 h-4 w-4" />}
  {uploadingTrack ? "Uploading..." : "Add public backing track"}
</Button>
</section>

<section className="rounded-2xl border border-border/50 bg-card/40 p-5 space-y-5">
  <div className="flex flex-col gap-3 sm:flex-row sm:items-center sm:justify-between">
    <div>
      <h2 className="font-semibold">Backing track</h2>
      <p className="text-xs text-muted-foreground">The Ready button stays locked until both devices decode a local buffer.</p>
    </div>
    <label className="inline-flex cursor-pointer items-center rounded-lg border border-amber-500/40 px-3 py-2 text-xs text-amber-300 hover:bg-amber-500/10">
      {loadingTrack ? <Loader2 className="mr-2 h-4 w-4 animate-spin" /> : <Mic2 className="mr-2 h-4 w-4" />}
      Choose audio
      <input className="hidden" type="file" accept="audio/*,.m4a,.wav,.mp3,.webm" onChange={(event) => void loadTrack(event.target.files?.[0] ?? null)} />
    </label>
  </div>
  <div className="grid gap-3 sm:grid-cols-3">
    <div className="rounded-lg bg-background/50 px-3 py-3"><div className="text-[10px] uppercase text-muted-foreground">Local buffers</div><div className="mt-1 text-sm">{bufferDecoded ? "Decoded" : "Not loaded"}</div></div>
    <div className="rounded-lg bg-background/50 px-3 py-3"><div className="text-[10px] uppercase text-muted-foreground">Partner buffer</div><div className="mt-1 text-sm">{remoteBufferReady ? "Decoded" : "Waiting"}</div></div>
    <div className="rounded-lg bg-background/50 px-3 py-3"><div className="text-[10px] uppercase text-muted-foreground">Clock link</div><div className="mt-1 text-sm">{rttMs === null ? "Measuring..." : `${rttMs} ms RTT`}</div></div>
  </div>
  {trackName && <div className="text-xs text-muted-foreground">{trackName} · {Math.round(duration)}s</div>}
</section>

<section className="rounded-2xl border border-amber-500/30 bg-amber-500/5 p-5 space-y-5">
  <div className="flex items-center justify-between gap-3">
    <div>
      <h2 className="font-semibold">Double-ready trigger</h2>
      <p className="text-xs text-muted-foreground">The host schedules AudioBufferSourceNode.start() five seconds ahead with the measured half-RTT correction.</p>
    </div>
    {clockOffsetMs != null && <span className="font-mono text-[10px] text-muted-foreground">clock {clockOffsetMs}ms</span>}
  </div>
  <div className="flex flex-wrap items-center gap-3">
    <Button disabled={!readyEnabled} onClick={setReady} className="bg-amber-500 text-black hover:bg-amber-400 disabled:opacity-40">
      <Radio className="mr-2 h-4 w-4" /> {localReady ? "Ready sent" : "I'm ready"}
    </Button>
    {isPlaying && <Button variant="outline" onClick={() => { sourceRef.current?.stop(); setIsPlaying(false); }}><Square className="mr-2 h-4 w-4" /> Stop</Button>}
    <span className="text-xs text-muted-foreground">{remoteReady ? "Partner ready" : "Partner not ready"}</span>
  </div>
</section>
</div>
</Layout>
);
}
```

07. DUET INGRESS & LEAD CAPTURE

artifacts/gravelkingpro/src/components/email-gate.tsx | 108 lines | 3,919 bytes | sha256 95e84510cd293deed94fd82302a8eda2dd64ca930fa0cfa5e08b35b9edfb425d

```
import { useCallback, useEffect, useState } from "react";
import { Button } from "@components/ui/button";
import { Mail, Loader2 } from "lucide-react";

const FLAG = "gkp_email_captured";

/**
 * Free-tool email gate. Visitors must leave an email before using the free
 * tools; the server independently enforces this on the mastering route
 * (403 EMAIL_REQUIRED), at which point `forceGate()` re-shows the gate.
 */
export function useEmailGate() {
  const [gated, setGated] = useState<boolean>(() => {
    try { return localStorage.getItem(FLAG) !== "1"; } catch { return false; }
  });

  useEffect(() => {
    fetch("/api/email-capture/status", { credentials: "include" })
      .then((r) => (r.ok ? r.json() : { hasEmail: false }))
      .then((d: { hasEmail?: boolean }) => {
        if (d.hasEmail) {
          try { localStorage.setItem(FLAG, "1"); } catch { /* ignore */ }
          setGated(false);
        } else {
          try { localStorage.removeItem(FLAG); } catch { /* ignore */ }
          setGated(true);
        }
      })
      .catch(() => { /* keep the local decision on network errors */ });
  }, []);

  const unlock = useCallback(() => {
    try { localStorage.setItem(FLAG, "1"); } catch { /* ignore */ }
    setGated(false);
  }, []);

  const forceGate = useCallback(() => {
    try { localStorage.removeItem(FLAG); } catch { /* ignore */ }
    setGated(true);
  }, []);

  return { gated, unlock, forceGate };
}

export function EmailGate({ tool, onUnlocked }: { tool: string; onUnlocked: () => void }) {
  const [email, setEmail] = useState("");
  const [busy, setBusy] = useState(false);
  const [error, setError] = useState("");

  const submit = async (e: React.FormEvent) => {
    e.preventDefault();
    setBusy(true);
    setError("");
    try {
      const resp = await fetch("/api/email-capture", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        credentials: "include",
        body: JSON.stringify({ email, source: tool }),
      });
      const data = (await resp.json().catch(() => ({}))) as { error?: string };
      if (!resp.ok) throw new Error(data.error ?? "Something went wrong – try again.");
      onUnlocked();
    } catch (err) {
      setError(err instanceof Error ? err.message : "Something went wrong – try again.");
    } finally {
      setBusy(false);
    }
  };

  return (
    <div className="max-w-md mx-auto py-20">
      <form
        onSubmit={submit}
        className="rounded-2xl border border-amber-500/30 bg-card/40 p-8 space-y-5 text-center">
        >
        <div className="w-12 h-12 rounded-full bg-amber-500/10 flex items-center justify-center mx-auto">
          <Mail className="w-6 h-6 text-amber-500" />
        </div>
        <div className="space-y-1.5">
          <h2 className="text-lg font-bold">Unlock the free tools</h2>
          <p className="text-sm text-muted-foreground">
            Drop your email once and every free tool is yours – mastering, karaoke, vocal booth preview.
          </p>
        </div>
        <input
          type="email"
          required
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          placeholder="you@example.com"
          className="w-full bg-secondary/40 border border-amber/40 rounded-lg px-4 py-2.5 text-sm focus:outline-none focus:border-amber-500/60 transition-colors">
        </>
        {error && <p className="text-xs text-destructive">{error}</p>}
        <Button
          type="submit"
          disabled={busy || !email.trim()}
          className="w-full bg-amber-500 hover:bg-amber-600 text-black font-semibold">
          >
          {busy ? <Loader2 className="w-4 h-4 animate-spin" /> : "Continue"}
        </Button>
        <p className="text-[18px] text-muted-foreground/70">
          No spam – occasional product updates from GravelKing Pro only.
        </p>
      </form>
    </div>
  );
}
```

08. DUET INGRESS & LEAD CAPTURE

artifacts/gravelkingpro/src/components/lead-capture-modal.tsx | 221 lines | 9,786 bytes | sha256 f80e321d935ccd34e766675e33df6c820e0a3de3ec7af183c0ac8ab0d4a8505d

```
import { useState } from "react";
import { motion, AnimatePresence } from "framer-motion";
import { Button } from "@components/ui/button";
import { X, Building2, User, Mail, Briefcase, CheckCircle2, Loader2, Key, Send } from "lucide-react";

const BASE = import.meta.env.BASE_URL.replace(/\/$/, "");

const ROLES = ["AGR", "Publisher", "Producer", "Platform Developer", "Other"] as const;

interface LeadCaptureModalProps {
  open: boolean;
  onClose: () => void;
  /** Optional pre-fill from a prior verify session */
  verificationActivity?: Record<string, unknown>;
}

interface FormState {
  name: string;
  email: string;
  organization: string;
  role: string;
}

type Stage = "form" | "sending" | "success" | "error";

export function LeadCaptureModal({ open, onClose, verificationActivity }: LeadCaptureModalProps) {
  const [form, setForm] = useState<FormState>({ name: "", email: "", organization: "", role: "" });
  const [stage, setStage] = useState<Stage>("form");
  const [demoKey, setDemoKey] = useState<string | null>(null);
  const [errorMsg, setErrorMsg] = useState("");

  function update(field: keyof FormState, value: string) {
    setForm(prev => ({ ...prev, [field]: value }));
  }

  async function submit(e: React.FormEvent) {
    e.preventDefault();
    if (!form.name.trim() || !form.email.trim() || !form.organization.trim() || !form.role) return;

    setStage("sending");
    setErrorMsg("");

    try {
      const res = await fetch(`${BASE}/api/v1/lead-capture`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ ...form, verification_activity: verificationActivity }),
      });
      const data = await res.json() as { success?: boolean; demoApiKey?: string; error?: string; errors?: string[] };

      if (!res.ok || !data.success) {
        throw new Error(data.error ?? data.errors?.join("; ") ?? "Submission failed");
      }

      setDemoKey(data.demoApiKey ?? null);
      setStage("success");
    } catch (err) {
      setErrorMsg(err instanceof Error ? err.message : "Submission failed. Please try again.");
      setStage("error");
    }
  }

  function reset() {
    setStage("form");
    setErrorMsg("");
    setForm({ name: "", email: "", organization: "", role: "" });
  }

  return (
    <AnimatePresence>
      {open && (
        <motion.div
          initial={{ opacity: 0 }}
          animate={{ opacity: 1 }}
          exit={{ opacity: 0 }}
          className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/70 backdrop-blur-sm"
          onClick={(e) => { if (e.target === e.currentTarget) onClose(); }}
        >
          <motion.div
            initial={{ opacity: 0, scale: 0.95, y: 16 }}
            animate={{ opacity: 1, scale: 1, y: 0 }}
            exit={{ opacity: 0, scale: 0.95, y: 16 }}
            transition={{ type: "spring", stiffness: 400, damping: 30 }}
            className="w-full max-w-md bg-card border border-border/60 rounded-xl shadow-2xl relative"
          >
            <button
              onClick={onClose}
              className="absolute right-4 top-4 text-muted-foreground hover:text-foreground transition-colors"
              aria-label="Close"
            >
              <X className="w-4 h-4" />
            </button>

            <div className="p-6">
              {stage === "success" ? (
                <motion.div initial={{ opacity: 0, y: 8 }} animate={{ opacity: 1, y: 0 }} className="text-center space-y-4 py-4">
                  <div className="flex justify-center">
                    <div className="w-16 h-16 rounded-full bg-emerald-500/10 border border-emerald-500/30 flex items-center justify-center">
                      <CheckCircle2 className="w-8 h-8 text-emerald-400" />
                    </div>
                  </div>
                  <div>
                    <h3 className="text-lg font-bold">Brief Sent</h3>
                    <p className="text-sm text-muted-foreground">
                      Check your inbox for the GravelKing Pro Technical Brief PDF and your demo API key.
                    </p>
                    {demoKey && (
                      <div className="bg-muted/30 border border-border/50 rounded-lg p-4 text-left">
                        <p className="text-xs text-muted-foreground mb-2 flex items-center gap-1.5">
                          <Key className="w-3 h-3" /> 7-Day Demo API Key
                        </p>
                        <code className="text-xs font-mono text-amber-400 break-all">{demoKey}</code>
                      </div>
                    )}
                  </div>
                </motion.div>
              ) : (
                <div>
                  <h3 className="text-lg font-bold">Brief Sent</h3>
                  <p>
                    <input type="text" value={form.name} />
                    <input type="text" value={form.email} />
                    <input type="text" value={form.organization} />
                    <input type="text" value={form.role} />
                  </p>
                  <div>
                    <input type="text" value={demoKey} />
                  </div>
                  <div>
                    <input type="text" value={errorMsg} />
                  </div>
                </div>
              )}
            </div>
          </motion.div>
        </>
      )}
    </AnimatePresence>
  );
}
```

08. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/gravelkingpro/src/components/lead-capture-modal.tsx | source continues without omission

```

        <Button onClick={onClose} className="bg-amber-500 hover:bg-amber-600 text-black font-semibold w-full">
          Continue
        </Button>
      </motion.div>
    ) : (
      <div className="mb-5">
        <h3 className="text-lg font-bold flex items-center gap-2">
          <Building2 className="w-5 h-5 text-amber-400" />
          Request Enterprise Demo
        </h3>
        <p className="text-sm text-muted-foreground mt-1">
          Get the GravelKing Pro MLK V4 Technical Brief PDF + a 7-day demo API key sent to your inbox.
        </p>
      </div>

      <form onSubmit={submit} className="space-y-4">
        <div>
          <label className="text-xs font-medium text-muted-foreground mb-1 flex items-center gap-1.5">
            <User className="w-3 h-3" /> Name
          </label>
          <input
            type="text"
            value={form.name}
            onChange={e => update("name", e.target.value)}
            placeholder="Your full name"
            required
            className="w-full bg-muted/30 border border-border/60 rounded-lg px-3 py-2 text-sm placeholder:text-muted-foreground/50 focus:outline-none focus:ring-1 focus:ring-amber-500/50"
          />
        </div>

        <div>
          <label className="text-xs font-medium text-muted-foreground mb-1 flex items-center gap-1.5">
            <Mail className="w-3 h-3" /> Work Email
          </label>
          <input
            type="email"
            value={form.email}
            onChange={e => update("email", e.target.value)}
            placeholder="you@label.com"
            required
            className="w-full bg-muted/30 border border-border/60 rounded-lg px-3 py-2 text-sm placeholder:text-muted-foreground/50 focus:outline-none focus:ring-1 focus:ring-amber-500/50"
          />
        </div>

        <div>
          <label className="text-xs font-medium text-muted-foreground mb-1 flex items-center gap-1.5">
            <Building2 className="w-3 h-3" /> Organization
          </label>
          <input
            type="text"
            value={form.organization}
            onChange={e => update("organization", e.target.value)}
            placeholder="Label, studio, or platform"
            required
            className="w-full bg-muted/30 border border-border/60 rounded-lg px-3 py-2 text-sm placeholder:text-muted-foreground/50 focus:outline-none focus:ring-1 focus:ring-amber-500/50"
          />
        </div>

        <div>
          <label className="text-xs font-medium text-muted-foreground mb-1 flex items-center gap-1.5">
            <Briefcase className="w-3 h-3" /> Role
          </label>
          <select
            value={form.role}
            onChange={e => update("role", e.target.value)}
            required
            className="w-full bg-muted/30 border border-border/60 rounded-lg px-3 py-2 text-sm text-foreground focus:outline-none focus:ring-1 focus:ring-amber-500/50"
          >
            <option value="">Select your role_</option>
            {ROLES.map(r => <option key={r} value={r}>{r}</option>)}
          </select>
        </div>

        {stage === "error" && (
          <p className="text-sm text-red-400 bg-red-500/10 border border-red-500/20 rounded-lg px-3 py-2">
            {errorMsg}
          </p>
        )}

        <div className="flex gap-3 pt-1">
          {stage === "error" && (
            <Button type="button" variant="outline" onClick={reset} className="flex-1">
              Try Again
            </Button>
          )}
          <Button
            type="submit"
            disabled={stage === "sending"}
            className="flex-1 bg-amber-500 hover:bg-amber-600 text-black font-semibold"
          >
            {stage === "sending" ? (
              <<=Loader2 className="w-4 h-4 mr-2 animate-spin" /> Sending_</>
            ) : (
              <<=Send className="w-4 h-4 mr-2" /> Get the Brief</>
            )}
          </Button>
        </div>
      </form>
    </>
  )}
</motion.div>
</motion.div>
  )}
</AnimatePresence>
);
}
```

09. DUET INGRESS & LEAD CAPTURE

artifacts/api-server/src/routes/email-capture.ts | 251 lines | 8,739 bytes | sha256 25d2e40344b9e3b548bc09b210e6e03485684bf7b227fa5f4ca7cedd4719cf35

```
/**
 * Visitor email capture + admin bulk email.
 *
 * Public:
 *   POST /api/email-capture - store a visitor's email, tied to their session user
 *   GET  /api/email-capture/status - does the current visitor have an email on file?
 *
 * Admin:
 *   GET  /api/admin/email-list - every captured address, newest first
 *   POST /api/admin/email-blast - send one email to every captured address via Gmail (BCC batches)
 */
import { Router, type Request, type Response } from "express";
import { db, usersTable, emailCaptureTable } from "@workspace/db";
import { desc, eq } from "drizzle-orm";
import { requireAdmin, isAdminAuthenticated } from "../lib/adminAuth";
import { rateLimit } from "../lib/rateLimiter";
import { getUsageUser } from "../lib/usage";
import { sendGmail, getGmailAddress } from "../lib/gmail";
import { storage } from "../storage";

const emailRouter = Router();

const EMAIL_RE = /^[^\s@]+@[^\s@]+\.[^\s@]{2,}$/;

function isUniqueViolation(err: unknown): boolean {
  return typeof err === "object" && err !== null && (err as { code?: unknown }).code === "23505";
}

/**
 * Attach an anonymous visitor's session to the canonical user for the email.
 * The session_id column is unique, so clear the anonymous row before moving
 * the token to an existing account.
 */
async function findOrLinkCapturedEmail(
  req: Request,
  usageUser: Awaited<ReturnType<typeof getUsageUser>>,
  normalized: string,
): Promise<void> {
  // getUsageUser creates the cookie on the response when this is a first-time
  // visitor, so req.cookies does not contain it yet. The returned user row does.
  const sessionId =
    (req.cookies as Record<string, string> | undefined)?.gk_session ??
    usageUser.sessionId ??
    undefined;

  const [emailOwner] = await db
    .select({ id: usersTable.id })
    .from(usersTable)
    .where(eq(usersTable.email, normalized))
    .limit(1);

  if (emailOwner && emailOwner.id !== usageUser.id) {
    if (!sessionId) return;
    await db.transaction(async (tx) => {
      await tx
        .update(usersTable)
        .set({ sessionId: null })
        .where(eq(usersTable.id, usageUser.id));
      await tx
        .update(usersTable)
        .set({ sessionId })
        .where(eq(usersTable.id, emailOwner.id));
    });
    return;
  }

  if (usageUser.email?.trim()) return;

  try {
    await db
      .update(usersTable)
      .set({ email: normalized })
      .where(eq(usersTable.id, usageUser.id));
  } catch (err) {
    // A concurrent request may have claimed the email after the lookup.
    // Re-resolve and link instead of returning a duplicate-key 500.
    if (!isUniqueViolation(err)) throw err;
    const [concurrentOwner] = await db
      .select({ id: usersTable.id })
      .from(usersTable)
      .where(eq(usersTable.email, normalized))
      .limit(1);
    if (!concurrentOwner || !sessionId) throw err;
    await db.transaction(async (tx) => {
      await tx
        .update(usersTable)
        .set({ sessionId: null })
        .where(eq(usersTable.id, usageUser.id));
      await tx
        .update(usersTable)
        .set({ sessionId })
        .where(eq(usersTable.id, concurrentOwner.id));
    });
  }
}

const captureRateLimit = rateLimit({
  windowMs: 10 * 60_000,
  max: 10,
  message: "Too many attempts. Wait a few minutes and try again.",
});

/** POST /api/email-capture - store a visitor email (deduped), tied to their session user. */
emailRouter.post("/email-capture", captureRateLimit, async (req: Request, res: Response) => {
  const { email, source } = (req.body ?? {}) as { email?: string; source?: string };
  const normalized = email?.trim().toLowerCase() ?? "";
  if (!EMAIL_RE.test(normalized)) {
    res.status(400).json({ error: "Enter a valid email address." });
    return;
  }
  try {
    // Tie the email to the visitor's user row so the free-tool gate recognises them.
    const usageUser = await getUsageUser(req, res);
    await findOrLinkCapturedEmail(req, usageUser, normalized);
  }
});
```

09. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/api-server/src/routes/email-capture.ts | source continues without omission

```
const [existing] = await db
  .select({ id: emailCaptureTable.id })
  .from(emailCaptureTable)
  .where(eq(emailCaptureTable.email, normalized))
  .limit(1);
if (!existing) {
  await db.insert(emailCaptureTable).values({
    email: normalized,
    source: (source ?? "free-tools").slice(0, 50),
    referrer: typeof req.headers.referrer === "string" ? req.headers.referrer.slice(0, 500) : null,
    userAgent: typeof req.headers["user-agent"] === "string" ? req.headers["user-agent"].slice(0, 500) : null,
  });
}
res.json({ ok: true });
} catch (err: unknown) {
  res.status(500).json({ error: err instanceof Error ? err.message : "Failed" });
}
});

/** GET /api/email-capture/status - does the current visitor have an email on file? */
emailRouter.get("/email-capture/status", async (req: Request, res: Response) => {
  try {
    // Admins are never gated.
    if (isAdminAuthenticated(req)) {
      res.json({ hasEmail: true });
      return;
    }
    if (req.dbUser) {
      res.json({ hasEmail: Boolean(req.dbUser.email?.trim()) });
      return;
    }
    const sessionId = (req.cookies as Record<string, string>)?._gk_session;
    if (!sessionId) {
      res.json({ hasEmail: false });
      return;
    }
    const user = await storage.getUserBySession(sessionId);
    res.json({ hasEmail: Boolean(user?.email?.trim()) });
  } catch {
    res.json({ hasEmail: false });
  }
});

/** GET /api/admin/email-list - every captured address, newest first. */
emailRouter.get("/admin/email-list", async (req: Request, res: Response) => {
  if (!await requireAdmin(req, res)) return;
  try {
    const [capturedRows, registeredUsers] = await Promise.all([
      db
        .select()
        .from(emailCaptureTable)
        .orderBy(desc(emailCaptureTable.createdAt))
        .limit(1000),
      db
        .select({
          email: usersTable.email,
          createdAt: usersTable.createdAt,
          source: usersTable.id,
        })
        .from(usersTable),
    ]);
    const byEmail = new Map<string, typeof capturedRows[number]>();
    for (const row of capturedRows) byEmail.set(row.email, row);
    for (const user of registeredUsers) {
      const email = user.email?.trim().toLowerCase();
      if (email && !byEmail.has(email)) {
        byEmail.set(email, {
          id: 0,
          email,
          source: "registered-user",
          referrer: null,
          userAgent: null,
          ipHash: null,
          createdAt: user.createdAt,
        });
      }
    }
    const rows = [...byEmail.values()].sort((a, b) => b.createdAt.getTime() - a.createdAt.getTime());
    res.json({ ok: true, total: rows.length, emails: rows });
  } catch (err: unknown) {
    res.status(500).json({ error: err instanceof Error ? err.message : "Failed" });
  }
});

const BCC_BATCH = 40;

/**
 * POST /api/admin/email-blast - send one email to every captured address via
 * the owner's Gmail. Recipients are BCC'd in batches so addresses stay private.
 */
emailRouter.post("/admin/email-blast", async (req: Request, res: Response) => {
  if (!await requireAdmin(req, res)) return;
  const { subject, body } = (req.body ?? {}) as { subject?: string; body?: string };
  if (!subject?.trim() || !body?.trim()) {
    res.status(400).json({ error: "subject and body are required" });
    return;
  }
  try {
    const rows = await db.select({ email: emailCaptureTable.email }).from(emailCaptureTable);
    const emails = [...new Set(rows.map((r) => r.email))];
    if (emails.length === 0) {
      res.json({ ok: true, sent: 0, failed: 0, batches: 0, total: 0 });
      return;
    }
    // Gmail rejects "undisclosed-recipients" To headers via the API - send to
    // the owner's own address with all recipients in BCC instead.
    const sender = await getGmailAddress();
    if (!sender) {
      res.status(502).json({ error: "Could not reach your connected Gmail account. Try again in a moment." });
      return;
    }
  }
```

09. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/api-server/src/routes/email-capture.ts | source continues without omission

```
let sent = 0;
let failed = 0;
let batches = 0;
for (let i = 0; i < emails.length; i += BCC_BATCH) {
  const chunk = emails.slice(i, i + BCC_BATCH);
  const ok = await sendGmail({
    to: sender,
    bcc: chunk.join(", "),
    subject: subject.trim(),
    text: body.trim(),
  });
  if (ok) sent += chunk.length; else failed += chunk.length;
  batches += 1;
  if (i + BCC_BATCH < emails.length) {
    await new Promise((r) => setTimeout(r, 300));
  }
}
res.json({ ok: failed === 0, sent, failed, batches, total: emails.length });
} catch (err: unknown) {
  res.status(500).json({ error: err instanceof Error ? err.message : "Failed" });
}
});

export default emailRouter;
```

10. DUET INGRESS & LEAD CAPTURE

artifacts/api-server/src/routes/lead-capture.ts | 139 lines | 6,856 bytes | sha256 562b38a36309f89da3ecd1d3f94731c90412fb86f2318e8e18260a676f653740

```
/**
 * POST /api/v1/lead-capture
 *
 * Enterprise B2B lead capture. Records the contact, issues a 7-day demo API
 * key, and sends the GravelKing Pro Technical Brief PDF to their inbox via
 * Resend. Falls back silently if Resend is not configured (key is still saved).
 */
import { Router, Request, Response } from "express";
import { randomBytes } from "crypto";
import { db, enterpriseLeadsTable } from "@workspace/db";
import { logger } from "../lib/logger";
import { logToolError } from "../lib/errorTracker";

const router = Router();

const VALID_ROLES = ["AGR", "Publisher", "Producer", "Platform Developer", "Other"] as const;

router.post("/v1/lead-capture", async (req: Request, res: Response): Promise<void> => {
  const { name, email, organization, role, verification_activity } = req.body as {
    name?: string;
    email?: string;
    organization?: string;
    role?: string;
    verification_activity?: unknown;
  };

  // — Validate —————
  const errors: string[] = [];
  if (!name?.trim()) errors.push("name is required");
  if (!email?.trim() || !email.includes("@")) errors.push("valid email is required");
  if (!organization?.trim()) errors.push("organization is required");
  if (!role?.trim()) errors.push("role is required");

  if (errors.length > 0) {
    res.status(400).json({ success: false, errors });
    return;
  }

  // — Generate demo API key (7-day TTL) —————
  const demoApiKey = randomBytes(32).toString("hex");
  const demoApiKeyExpiresAt = new Date(Date.now() + 7 * 24 * 60 * 60 * 1000);

  // — Persist —————
  let lead;
  try {
    [lead] = await db
      .insert(enterpriseLeadsTable)
      .values({
        name: name!.trim(),
        email: email!.trim().toLowerCase(),
        organization: organization!.trim(),
        role: (VALID_ROLES as readonly string[]).includes(role!.trim()) ? role!.trim() : "Other",
        demoApiKey,
        demoApiKeyExpiresAt,
        verificationActivity: verification_activity ? JSON.stringify(verification_activity) : null,
      })
      .returning();
  } catch (err) {
    // Duplicate email – still return the key
    logger.warn({ err, email }, "lead capture insert failed; may be duplicate");
    res.status(409).json({
      success: false,
      error: "An enterprise inquiry from this email already exists. Please check your inbox for the brief.",
    });
    return;
  }

  // — Send email via Resend —————
  let emailSent = false;
  const resendKey = process.env["RESEND_API_KEY"]?.trim() ?? "";
  if (resendKey) {
    try {
      const { Resend } = await import("resend");
      const resend = new Resend(resendKey);

      const whitepaperUrl = "https://gravelkingpro.com/api/whitepaper.pdf";

      const { error } = await resend.emails.send({
        from: "GravelKing Pro <noreply@gravelkingpro.com>",
        to: email!.trim(),
        subject: "Your GravelKing Pro Technical Brief + Demo API Key",
        html: `
<div style="font-family: -apple-system, sans-serif; max-width: 560px; margin: 0 auto; background: #09090b; color: #fafafa; padding: 32px; border-radius: 8px; border: 1px solid #27272a;">
  <div style="display: flex; align-items: center; gap: 8px; margin-bottom: 24px;">
    <span style="color: #f59e0b; font-size: 20px;"></span>
    <span style="font-weight: 700; font-size: 16px; letter-spacing: 0.05em; text-transform: uppercase;">GravelKing Productions</span>
  </div>
  <h1 style="font-size: 22px; font-weight: 700; margin-bottom: 8px;">Your Enterprise Brief & Demo Key</h1>
  <p style="color: #alalaa; margin-bottom: 24px;">Hi ${name!.trim()}, thanks for your interest in GravelKing Pro MLK V4.</p>

  <div style="background: #18181b; border: 1px solid #3f3f46; border-radius: 8px; padding: 20px; margin-bottom: 24px;">
    <p style="font-size: 12px; text-transform: uppercase; letter-spacing: 0.1em; color: #alalaa; margin-bottom: 8px;">Your Demo API Key</p>
    <code style="font-family: monospace; font-size: 13px; color: #f59e0b; word-break: break-all;">${demoApiKey}</code>
    <p style="font-size: 11px; color: #71717a; margin-top: 8px;">Valid for 7 days · ${demoApiKeyExpiresAt.toUTCString()}</p>
  </div>

  <p style="color: #alalaa; margin-bottom: 16px;">Attached is our full technical brief covering the MLK V4 dual-anchor LSB steganography architecture, chain-of-custody certification, and enterprise integration guide.</p>

  <a href="${whitepaperUrl}"
    style="display: inline-block; background: #f59e0b; color: #000; font-weight: 600; padding: 12px 24px; border-radius: 6px; text-decoration: none; margin-bottom: 24px;">
    Download Technical Brief (PDF)
  </a>

  <p style="color: #alalaa; margin-bottom: 8px;">Try the live signal verification at:</p>
  <a href="https://gravelkingpro.com/verify" style="color: #f59e0b;">gravelkingpro.com/verify</a>

  <hr style="border: none; border-top: 1px solid #27272a; margin: 24px 0;" />
  <p style="font-size: 12px; color: #52525b;">GravelKing Productions · All N One LLC · <a href="mailto:kevm@gravelkingpro.com" style="color: #f59e0b;">kevm@gravelkingpro.com</a></p>
</div>
`,
      });
    } catch (err) {
      logToolError(err, "Resend email failed");
    }
  }
}
```

10. DUET INGRESS & LEAD CAPTURE (continued)

artifacts/api-server/src/routes/lead-capture.ts | source continues without omission

```
        emailSent = true;
        await db.update(enterpriseLeadsTable)
            .set({ emailSent: true })
            // @ts-ignore drizzle eq import
            .where((await import("drizzle-orm")).eq(enterpriseLeadsTable.id, lead.id));
    }
} catch (err) {
    void logToolError("Lead Capture Email", "EMAIL_SEND", err);
    logger.warn({ err }, "lead capture email failed; lead was still saved");
}

logger.info({ leadId: lead.id, email: lead.email, organization: lead.organization }, "enterprise lead captured");

res.json({
    success: true,
    demoApiKey,
    demoApiKeyExpiresAt: demoApiKeyExpiresAt.toISOString(),
    emailSent,
    message: emailSent
        ? "Your demo key and technical brief have been sent to your inbox."
        : "Your demo key has been issued. The technical brief PDF is available at /api/whitepaper.pdf.",
});
});

export default router;
```

11. VIDEO EXTRACTION & INGESTION

artifacts/api-server/src/routes/audio.ts | 898 lines | 37,134 bytes | sha256 a0719f7317fbfe48f629ba1600bf54de630a23e0547d19019fe207f7fee34f19

```
import { Router, Request, Response } from "express";
import { streamBuffer } from "../lib/streamResponse";
import { createAudioJob, completeAudioJob } from "../lib/firestore";
import multer from "multer";
import { execFile } from "child_process";
import { promisify } from "util";
import { writeFile, readFile, unlink, stat } from "fs/promises";
import { randomUUID } from "crypto";
import { zipSync } from "fflate";
import { telemetryBus, type TelemetryEvent } from "../lib/telemetry";
import { db, processRunsTable } from "@workspace/db";
import {
  mlkVocalRemoval,
  mlkStemSplit,
  uvrVocalRemoval,
  MLK_PROTOCOL,
  MLK_KERNEL,
  MLK_STACK,
  UVR_PROTOCOL,
  UVR_MODEL,
  UVR_STACK,
} from "../gkp-separator";
import { transcribeWithGemini } from "../geminiTranscribe";
import {
  demucsVoiceRemove,
  demucsStemSplit,
  isDemucsConfigured,
} from "../demucsCloudRun";
import {
  replicateStemSplit,
  isConfigured as isReplicateConfigured,
} from "../replicateDemucs";
import { ratelimit } from "../lib/ratelimiter";
import { concurrencyLimit } from "../lib/concurrencyLimit";
import { probeAudioDuration, probeFileDuration, sanitizeExt, MAX_AUDIO_DURATION_S } from "../lib/audioGuards";
import { hasStudio, hasUnlimitedSplits, resolveTier } from "../lib/entitlement";
import { getUsageUser, incrementUsage, FREE_LIMITS, type UsageField } from "../lib/usage";
import { checkExportQuota, consumeExport, exportLimitPayload } from "../lib/exportQuota";
import { logToolError } from "../lib/errorTracker";
import { recordActivity } from "../lib/activityTracker";
import { validateAssetIngestion } from "../middlewares/validateAssetIngestion";
import type { NextFunction } from "express";
import { parsePastedLyrics, transcriptToTimedLines } from "../lib/lyricTiming";

const execFileAsync = promisify(execFile);

/** Auth-gating middleware: process-audio requires King tier.
 * Runs BEFORE multer so unauthenticated requests never write to /tmp
 * and cannot reach the ingestion validator. */
async function requireStudioForAudio(req: Request, res: Response, next: NextFunction): Promise<void> {
  if (!await hasStudio(req)) {
    res.status(403).json({
      success: false,
      error: "Processing audio requires a King subscription.",
      code: "STUDIO_REQUIRED",
    });
    return;
  }
  next();
}

const upload = multer({
  storage: multer.diskStorage({
    destination: "/tmp",
    filename: (_req, file, cb) => {
      cb(null, `gk_audio_${randomUUID()}.${sanitizeExt(file.originalname)}`);
    },
  }),
  // Camera-roll uploads may be video containers; keep the same 250 MB
  // contract as the paid upload flow before ffmpeg strips the video stream.
  limits: { fileSize: 250 * 1024 * 1024 },
});

const audioRouter = Router();

const audioRateLimit = ratelimit({ windowMs: 10 * 60_000, max: 10 });
const audioConcurrency = concurrencyLimit(2);

type ProcessMode = "standard" | "voice_remove" | "stem_split";

// — Audio helpers
async function getAudioInfo(filePath: string): Promise<{ sampleRate: number; channels: number }> {
  let sampleRate = 44100;
  let channels = 2;
  try {
    const { stdout } = await execFileAsync("ffprobe", ["-v", "quiet", "-print_format", "json", "-show_streams", filePath], { timeout: 10_000 });
    const info = JSON.parse(stdout);
    const stream = info.streams?.find((s: any) => s.codec_type === "audio");
    if (stream) {
      sampleRate = parseInt(stream.sample_rate) || 44100;
      channels = parseInt(stream.channels) || 2;
    }
  } catch { /* use defaults */ }
  return { sampleRate, channels };
}

/**
 * MLK V4 (Morris Law Kernel V4) via ffmpeg filter chain – multi-band carving with no in-process RAM allocation.
 * Streams entirely on disk; handles any file size without OOM.
 */
async function processWithMLKV3Ffmpeg(
  filePath: string,
  ext: string,
  multiplier: number,
): Promise<{ wavBuffer: Buffer; parity: string; efficiency: string; decayRate: string; sampleCount: number }> {
  const id = randomUUID();
  const outPath = `/tmp/gk_mlk_out_${id}.wav`;

  // Per-band multipliers match mlk_v3 JS implementation:
  // low: multiplier * 1.15 (capped at 2.0)
  // mid: multiplier
  // high: multiplier * 0.80 (min 0.1)
  const lowMult = Math.min(2.0, multiplier * 1.15).toFixed(4);
  const midMult = multiplier.toFixed(4);
  const highMult = Math.max(0.1, multiplier * 0.80).toFixed(4);
```

11. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/audio.ts | source continues without omission

```
// 3-band split -> per-band volume -> recombine -> normalize
// dynaudnorm is chained with , (not ;) so it receives the amix output directly.
// Using ; would give dynaudnorm no labeled input and ffmpeg rejects the graph.
const filter = [
  `asplit=3[low][mid][high]`,
  `[low]lowpass=f=250,volume=${lowMult}[l]`,
  `[mid]highpass=f=250,lowpass=f=4000,volume=${midMult}[m]`,
  `[high]highpass=f=4000,volume=${highMult}[h]`,
  `[l][m][h]amix=inputs=3:normalize=0,dynaudnorm=p=0.9:m=10:s=5`,
].join(";");

try {
  await execFileAsync("ffmpeg", [
    "-y", "-i", filePath,
    "-filter_complex", filter,
    "-ac", "2",
    "-acodec", "pcm_s16le",
    outputPath,
  ], { timeout: 180_000 });

  const wavBuffer = await readFile(outputPath);
  const sampleCount = Math.floor((wavBuffer.length - 44) / 2);
  const decayRate = (1 - multiplier).toFixed(4);
  const efficiency = multiplier.toFixed(4);
  const parity = "MLK_V3_VALIDATED";

  return { wavBuffer, parity, efficiency, decayRate, sampleCount };
} finally {
  await unlink(outputPath).catch(() => {});
}

let _rubberbandAvailable: boolean | null = null;
function rubberbandAvailable(): boolean {
  if (_rubberbandAvailable !== null) return _rubberbandAvailable;
  try {
    const { execFileSync } = require("child_process");
    const filters = execFileSync("ffmpeg", ["-filters"], { encoding: "utf8", stdio: "pipe" });
    _rubberbandAvailable = filters.includes("rubberband");
  } catch {
    _rubberbandAvailable = false;
  }
  return _rubberbandAvailable!;
}

/** Apply rubberband tempo/pitch shift as a post-processing step.
 * Falls back to passthrough when rubberband is not available in ffmpeg.
 */
async function applyTempoAndPitch(filePathOrBuf: string | Buffer, tempo: number, semitones: number): Promise<Buffer> {
  if (Math.abs(tempo - 1.0) < 0.01 && Math.abs(semitones) < 0.1) {
    if (typeof filePathOrBuf === "string") return readFile(filePathOrBuf);
    return filePathOrBuf;
  }
  // rubberband filter is not available in this ffmpeg build - skip gracefully
  if (!rubberbandAvailable()) {
    if (typeof filePathOrBuf === "string") return readFile(filePathOrBuf);
    return filePathOrBuf;
  }
  const pitchRatio = Math.pow(2, semitones / 12);
  const filter = `rubberband=tempo=${tempo.toFixed(3)}:pitch=${pitchRatio.toFixed(4)}`;
  if (typeof filePathOrBuf === "string") return processWithFilter(filePathOrBuf, filter, 2);
  const id = randomUUID();
  const inPath = `tmp/gk_tap_in_${id}.wav`;
  await writeFile(inPath, filePathOrBuf);
  try {
    return await processWithFilter(inPath, filter, 2);
  } finally {
    await unlink(inPath).catch(() => {});
  }
}

/** Apply an ffmpeg audio filter directly to a file and return a WAV buffer. */
async function processWithFilter(
  filePath: string,
  filter: string,
  outputChannels: number = 2
): Promise<Buffer> {
  const id = randomUUID();
  const outputPath = `tmp/gkflt_out_${id}.wav`;
  try {
    await execFileAsync("ffmpeg", [
      "-y", "-i", filePath,
      "-af", filter,
      "-ac", String(outputChannels),
      "-acodec", "pcm_s16le",
      outputPath,
    ], { timeout: 120_000 });
    return await readFile(outputPath);
  } finally {
    await unlink(outputPath).catch(() => {});
  }
}

/** Split audio into frequency-band stems and return a ZIP buffer. */
async function buildStemsZip(filePath: string, channels: number): Promise<Buffer> {
  const stems: Array<{ name: string; filter: string; ch: number }> = [
    { name: "bass.wav", filter: "lowpass=f=250", ch: channels },
    { name: "midrange.wav", filter: "highpass=f=250,lowpass=f=4000", ch: channels },
    { name: "highs.wav", filter: "highpass=f=4000", ch: channels },
  ];

  if (channels >= 2) {
    stems.push({ name: "instrumental.wav", filter: "pan=stereo|c0=c0-c1|c1=c1-c0", ch: 2 });
  }

  const wavFiles = await Promise.all(
    stems.map(async (s) => {
      const buf = await processWithFilter(filePath, s.filter, s.ch);
      return { name: s.name, buf };
    })
  );
}
```

11. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/audio.ts | source continues without omission

```
const zipInput: Record<string, Uint8Array> = {};
for (const { name, buf } of wavFiles) {
  zipInput[name] = new Uint8Array(buf);
}

return Buffer.from(zipSync(zipInput));
}

// — Telemetry SSE endpoint —————
audioRouter.get("/kernel/telemetry", (req: Request, res: Response) => {
  res.setHeader("Content-Type", "text/event-stream");
  res.setHeader("Cache-Control", "no-cache");
  res.setHeader("Connection", "keep-alive");
  res.setHeader("X-Accel-Buffering", "no");
  res.flushHeaders();

  const heartbeat = setInterval(() => res.write(": heartbeat\n\n"), 25_000);

  const handler = (event: TelemetryEvent) => {
    res.write(`data: ${JSON.stringify(event)}\n\n`);
  };

  telemetryBus.on("run", handler);

  req.on("close", () => {
    clearInterval(heartbeat);
    telemetryBus.off("run", handler);
  });
});

// — Config / health endpoint —————
audioRouter.get("/kernel/routing", (_req, res) => {
  res.json({ mode: "local", remoteUrl: null, remoteStatus: "not_configured", localKernel: "active" });
});

// — Usage status —————
// Drives the free-tier UI (remaining runs + studio gate) without consuming a run.
audioRouter.get("/usage/status", async (req: Request, res: Response) => {
  const tier = await resolveTier(req);
  const unlimited = tier === "free";
  const usageUser = await getUsageUser(req, res);
  const remaining = (field: UsageField): number =>
    unlimited ? -1 : Math.max(0, FREE_LIMITS[field] - (usageUser[field] ?? 0));

  const exportQuota = checkExportQuota(usageUser);
  res.json({
    tier,
    limits: FREE_LIMITS,
    remaining: {
      voice_remove: remaining("freeVoiceRemovals"),
      stem_split: remaining("freeStemSplits"),
      master: remaining("freeMasterDownloads"),
    },
    exports: {
      used: exportQuota.used,
      limit: exportQuota.limit,
      remaining: exportQuota.unlimited ? -1 : Math.max(0, exportQuota.limit - exportQuota.used),
      unlimited: exportQuota.unlimited ?? false,
      resetsAt: exportQuota.resetsAt,
    },
  });
});

// — Main processing endpoint —————
audioRouter.post(
  "/kernel/process-audio",
  audioRateLimit,
  audioConcurrency,
  requireStudioForAudio,
  upload.single("audio"),
  validateAssetIngestion({ requireAudio: true }),
  async (req: Request, res: Response) => {
    if (!req.file) {
      res.status(400).json({ success: false, error: "No audio file uploaded." });
      return;
    }

    let filePath = req.file.path;
    const ext = sanitizeExt(req.file.originalname);

    // If the upload is a video container, extract the audio stream to WAV first.
    // This prevents the UVR Python runner from hanging on MP4/MOV/etc files
    // (it only speaks audio formats). ffmpeg handles video extraction in seconds.
    // Pre-convert every non-WAV format to 44.1 kHz stereo WAV so that:
    // (a) torchaudio on Cloud Run can always decode the input
    // (b) the DSP fallback's 5 parallel ffmpeg processes each read a WAV
    // instead of decoding MP3/AAC/FLAC 5x in parallel (much faster)
    const VIDEO_CONTAINER_EXTS = new Set([
      "mp4", "mov", "m4v", "avi", "mkv", "webm", "wmv", "flv",
      "mp3", "m4a", "aac", "ogg", "oga", "opus", "wma", "flac", "aiff", "au",
    ]);
    if (VIDEO_CONTAINER_EXTS.has(ext.toLowerCase())) {
      const wavId = randomUUID();
      const wavPath = `/tmp/gk_vid2aud_${wavId}.wav`;
      try {
        await execFileAsync("ffmpeg", [
          "-y", "-i", filePath, "-vn",
          "-acodec", "pcm_s16le", "-ar", "44100", "-ac", "2",
          wavPath,
        ], { timeout: 120_000 });
        req.log.info(
          {
            originalName: req.file.originalname,
            sourceExtension: ext.toLowerCase(),
            extractedExtension: "wav",
          },
          "[audio-ingest] video container demuxed to audio",
        );
      } catch {
        await unlink(filePath).catch(() => {});
        filePath = wavPath;
      } catch {
        await unlink(filePath).catch(() => {});
        res.status(422).json({
```

11. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/audio.ts | source continues without omission

```

    success: false,
    error: "Could not extract audio from the video file. Try converting it to MP3 or WAV using the Convert tool first.",
  });
  return;
}

const multiplier = parseFloat((req.body.multiplier as string) ?? "0.75");
const sliceSize = parseInt((req.body.slice_size as string) ?? "2");
const mode: ProcessMode = (req.body.mode as ProcessMode) ?? "standard";
const source = (req.body.source as string) ?? "";
const stemsOnly = (req.body.stemsOnly as string) ?? "";
// Only render vocals + instrumental when the caller doesn't need all 5 stems
// (voice splitter, vocal booth). Avoids 250 MB+ ZIP OOM on iOS.
const primaryStemsOnly = source !== "studio" || stemsOnly === "primary";
const tempo = Math.min(2.5, Math.max(0.25, parseFloat((req.body.tempo as string) ?? "1.0")));
const semitones = Math.min(12, Math.max(-12, parseFloat((req.body.semitones as string) ?? "0")));

// — Duration guard —
const duration = await probeFileDuration(filePath);
if (duration > MAX_AUDIO_DURATION_S) {
  await unlink(filePath).catch(() => {});
  res.status(422).json({
    success: false,
    error: `Audio exceeds the maximum allowed duration of ${Math.floor(MAX_AUDIO_DURATION_S / 60)} minutes.`,
  });
  return;
}

// — Free-tier limits for split modes —
// Pro+ tiers are unlimited; free users get a fixed number of runs each,
// tracked server-side per user / gk session. On exhaustion we return a
// 402 LIMIT_REACHED payload that drives the payroll funnel.
//
// Studio stem split path: no free trial – requires King subscription.
// When source === "studio" for stem_split, the free-tier counter is bypassed
// entirely and a Pro check is enforced instead.
let isFreeUse = false;
let usageField: UsageField | null = null;
let usageUserId: string | null = null;
let freeRemaining = 0;
if (mode === "voice_remove" || mode === "stem_split") {
  if (mode === "stem_split" && source === "studio") {
    // Studio path: King subscription required – no free trial allowance
    if (!await hasStudio(req)) {
      await unlink(filePath).catch(() => {});
      res.status(402).json({
        success: false,
        code: "UPGRADE_REQUIRED",
        feature: "stem_split",
        error: "Stem Splitting in Studio requires a King subscription.",
        fallback: { action: "subscribe", url: "/pricing" },
      });
      return;
    }
  }
  // King confirmed – proceed with no usage accounting
} else {
  const unlimited = await hasUnlimitedSplits(req);
  if (!unlimited) {
    usageField = mode === "voice_remove" ? "freeVoiceRemovals" : "freeStemSplits";
    const usageUser = await getUsageUser(req, res);
    usageUserId = usageUser.id;
    const used = usageUser[usageField] ?? 0;
    const limit = FREE_LIMITS[usageField];
    if (used >= limit) {
      await unlink(filePath).catch(() => {});
      res.status(402).json({
        success: false,
        code: "LIMIT_REACHED",
        feature: mode,
        limit,
        used,
        error:
          mode === "voice_remove"
            ? `You've used all ${limit} free voice removals. Subscribe to GravelKing Pro for unlimited access.`
            : `You've used your free stem split. Subscribe to GravelKing Pro for unlimited access.`,
        fallback: { action: "subscribe", url: "/pricing" },
      });
      return;
    }
  }
  const usedTotal = usageUser.totalDownloads ?? 0;
  if (usedTotal >= FREE_LIMITS.totalDownloads) {
    await unlink(filePath).catch(() => {});
    res.status(402).json({
      success: false,
      code: "LIMIT_REACHED",
      feature: mode,
      limit: FREE_LIMITS.totalDownloads,
      used: usedTotal,
      error: "You've used your free download. Subscribe to GravelKing Pro for unlimited access.",
      fallback: { action: "subscribe", url: "/pricing" },
    });
    return;
  }
}
isFreeUse = true;
freeRemaining = limit - used - 1;
}
}

// — Tier-aware WAV export quota —
// Pro has 10 WAV exports per rolling 7 days; King has 40 per rolling 30
// days. Checked before expensive processing and consumed only after a
// successful WAV-producing run, right before bytes stream.
const exportQuotaUser = await getUsageUser(req, res);
{
  const quota = checkExportQuota(exportQuotaUser);
  if (!quota.allowed) {
    await unlink(filePath).catch(() => {});
    res.status(429).json(exportLimitPayload(quota));
    return;
  }
}
```

11. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/audio.ts | source continues without omission

```
/** Consume one export: on exhaustion sends the 429 and returns false. */
const consumeExportOrReject = async (): Promise<boolean> => {
  const quota = await consumeExport(exportQuotaUser);
  if (!quota.allowed) {
    res.status(429).json(exportLimitPayload(quota));
    return false;
  }
  return true;
};

// — Voice removal – neural for paid, DSP for free —————
if (mode === "voice_remove") {
  let fsJobId: string | undefined;
  try {
    const tier = await resolveTier(req);
    const useNeural = tier === "node_auditor";
    const { channels } = await getAudioInfo(filePath);

    recordActivity((req.cookies as Record<string, string>)?["gk_session"], "Singing Booth");

    fsJobId = createAudioJob({
      fileName: req.file.originalname,
      mode: "voice_remove",
      tier,
      status: "STAGE_1_PROCESSING",
      sessionId: (req.cookies as Record<string, string>)?["gk_session"],
    });

    let result;
    // Honest engine reporting: "remote" when real neural Replicate Demucs
    // ran, "local" for the in-process DSP / local-neural fallback paths.
    let routing: "local" | "remote" = "local";
    let stack: string;
    let model: string;
    let protocol: string;

    if (isDemucsConfigured()) {
      // Primary: Cloud Run Demucs htdemucs – real neural AI separation.
      // Falls back to DSP so the route never 500s.
      try {
        result = await demucsVoiceRemove(filePath, multiplier);
        routing = "remote";
        stack = result.stack;
        model = result.model;
        protocol = result.protocol;
        req.log.info(
          { separator: "cloudrun_demucs_htdemucs", ext },
          "voice_remove via Cloud Run Demucs htdemucs",
        );
      } catch (demucsErr: any) {
        req.log.warn(
          { err: demucsErr?.message, separator: "cloudrun_demucs" },
          "Cloud Run Demucs failed; falling back to DSP",
        );
      }
      result = await mlkVocalRemoval(filePath, ext, channels, multiplier);
      req.log.info({ separator: "MLK v3 DSP", ext }, "voice_remove fallback via DSP");
      stack = MLK_STACK; model = MLK_KERNEL; protocol = MLK_PROTOCOL;
    }
  } else if (useNeural) {
    // No Replicate token – Node Auditor tier: UVR → DSP.
    try {
      result = await uvrVocalRemoval(filePath, ext, multiplier);
      req.log.info(
        { separator: "UVR_MDXNET", ext },
        "voice_remove via neural UVR-MDX-Net for Node Auditor",
      );
    } catch (neuralErr: any) {
      req.log.warn(
        { err: neuralErr?.message, separator: "UVR_MDXNET", ext, channels },
        "neural UVR unavailable; falling back to DSP center extraction",
      );
    }
    result = await mlkVocalRemoval(filePath, ext, channels, multiplier);
    stack = MLK_STACK; model = MLK_KERNEL; protocol = MLK_PROTOCOL;
  } else {
    // No Replicate token – instant DSP vocal removal (center-channel extraction).
    result = await mlkVocalRemoval(filePath, ext, channels, multiplier);
    req.log.info(
      { separator: "MLK_v3_DSP", ext, channels },
      "voice_remove via DSP center extraction",
    );
    stack = MLK_STACK; model = MLK_KERNEL; protocol = MLK_PROTOCOL;
  }

  completeAudioJob(fsJobId, { status: "STAGE_1_COMPLETE", stack, model });

  const instrumentalBuf = result.instrumental;
  const kernelParity = result.kernelParity;
  let wavBuffer = await applyTempoAndPitch(instrumentalBuf, tempo, semitones);

  // Cleanup uploaded file after processing
  await unlink(filePath).catch(() => {});

  const event: TelemetryEvent = {
    routing,
    parity: kernelParity,
    efficiency: "1.0000",
    decayRate: "0.0000",
    sampleCount: String(wavBuffer.length / 2),
    timestamp: new Date().toISOString(),
  };
  telemetryBus.emit("run", event);

  if (req.dbUser) {
    db.insert(processRunsTable).values({
      userId: req.dbUser.id,
      routing,
      parity: kernelParity,
      efficiency: 1,
      decayRate: 0,
      sampleCount: wavBuffer.length / 2,
      fileName: req.file.originalname,
    });
  }
}
```

11. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/audio.ts | source continues without omission

```
    }).catch(() => {});
  }

  if (isFreeUse && usageField && usageUserId) {
    await incrementUsage(usageUserId, usageField);
    await incrementUsage(usageUserId, "totalDownloads");
    res.setHeader("X-GK-Free-Remaining", String(freeRemaining));
  }

  if (!(await consumeExportOrReject())) return;

  // MP3 fallback for free users; WAV for paid users.
  const mp3Wanted = !await hasUnlimitedSplits(req);
  if (mp3Wanted) {
    const mp3Id = randomUUID();
    const mp3InPath = `tmp/gk_vr_in_${mp3Id}.wav`;
    const mp3OutPath = `tmp/gk_vr_out_${mp3Id}.mp3`;
    await writeFile(mp3InPath, wavBuffer);
    try {
      await execFileAsync("ffmpeg", [
        "-y", "-i", mp3InPath, "-acodec", "libmp3lame", "-b:a", "320k", "-ar", "44100", mp3OutPath,
      ], { maxBuffer: 50 * 1024 * 1024, timeout: 120_000 });
      const mp3Buf = await readFile(mp3OutPath);
      res.setHeader("Content-Type", "audio/mpeg");
      res.setHeader("Content-Disposition", `attachment; filename="gravelking_instrumental.mp3"`);
      res.setHeader("X-GK-Mode", "voice_remove");
      res.setHeader("X-GK-Routing", routing);
      res.setHeader("X-GK-Parity", kernelParity);
      res.setHeader("X-GK-Format", "mp3");
      res.setHeader("X-GK-Separator", model);
      res.setHeader("X-GK-Model", model);
      res.setHeader("X-GK-Protocol", protocol);
      res.setHeader("X-GK-Stack", stack);
      streamBuffer(res, mp3Buf);
    } finally {
      await unlink(mp3InPath).catch(() => {});
      await unlink(mp3OutPath).catch(() => {});
    }
  } else {
    res.setHeader("Content-Type", "audio/wav");
    res.setHeader("Content-Disposition", `attachment; filename="gravelking_instrumental.wav"`);
    res.setHeader("X-GK-Mode", "voice_remove");
    res.setHeader("X-GK-Routing", routing);
    res.setHeader("X-GK-Parity", kernelParity);
    res.setHeader("X-GK-Efficiency", "1.0000");
    res.setHeader("X-GK-Decay-Rate", "0.0000");
    res.setHeader("X-GK-Sample-Count", String(wavBuffer.length / 2));
    res.setHeader("X-GK-Separator", model);
    res.setHeader("X-GK-Model", model);
    res.setHeader("X-GK-Protocol", protocol);
    res.setHeader("X-GK-Stack", stack);
    streamBuffer(res, wavBuffer);
  }
  return;
} catch (err: any) {
  if (fsJobId) completeAudioJob(fsJobId, { status: "ERROR", errorMessage: err.message });
  void logToolError("Singing Booth", "VOICE_REMOVE", err);
  await unlink(filePath).catch(() => {});
  res.status(500).json({ success: false, error: err.message });
  return;
}
}

// — Stem splitting – Morris Law Kernel V4 (fast local) —————
if (mode === "stem_split") {
  recordActivity((req.cookies as Record<string, string>)?["gk_session"], "Singing Booth");
  // Pre-compress: files > 15 MB are re-encoded to 22050 Hz stereo so Cloud
  // Run can accept them (it rejects > 20 MB). Also makes Replicate faster.
  try {
    const { size } = await stat(filePath);
    if (size > 15 * 1024 * 1024) {
      const compPath = `tmp/gk_comp_${randomUUID()}.wav`;
      await execFileAsync("ffmpeg", [
        "-y", "-i", filePath, "-vn",
        "-acodec", "pcm_s16le", "-ar", "22050", "-ac", "2",
        compPath,
      ], { timeout: 60_000 });
      await unlink(filePath).catch(() => {});
      filePath = compPath;
      req.log.info({ originalBytes: size }, "stem_split: pre-compressed to 22050 Hz");
    }
  } catch {
    // Non-fatal – continue with original file
  }

  try {
    const { channels } = await getAudioInfo(filePath);
    type StemResultShape = Awaited<ReturnType<typeof mlkStemSplit>>;
    let stemResult: StemResultShape;
    let stemSeparatorLabel = "MLK_v3";
    let stemModelLabel: string = MLK_KERNEL;
    let stemProtocolLabel: string = MLK_PROTOCOL;
    // Honest engine reporting: "remote" only when real neural separation ran.
    let stemRouting: "local" | "remote" = "local";

    // Tries Replicate Demucs (cloud GPU), falls back to local DSP band-split.
    // Returns the result and the routing/label metadata to apply.
    async function runReplicateThenDsp(label: string): Promise<{
      result: StemResultShape;
      routing: "local" | "remote";
      separator: string;
    }> {
      if (isReplicateConfigured()) {
        try {
          const r = await replicateStemSplit(filePath, multiplier, primaryStemsOnly);
          req.log.info({ separator: "replicate_demucs_htdemucs" }, `${label} via Replicate Demucs`);
          return { result: r, routing: "remote", separator: "Replicate_Demucs_htdemucs" };
        } catch (repErr: any) {
          req.log.warn({ err: repErr?.message }, `${label} Replicate failed; falling back to DSP`);
        }
      }
      const r = await mlkStemSplit(filePath, ext, channels, multiplier, primaryStemsOnly);
      req.log.info({ separator: "MLK_v3_DSP" }, `${label} via DSP`);
    }
  }
}
```

11. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/audio.ts | source continues without omission

```
        return { result: r, routing: "local", separator: "MLK_v3_DSP" };
    }

    if (isDemucsConfigured()) {
        // Primary: Cloud Run Demucs htdemucs - real neural 4-stem separation.
        try {
            stemResult = await demucsStemSplit(filePath, multiplier, primaryStemsOnly);
            stemRouting = "remote";
            stemSeparatorLabel = "Demucs_htdemucs";
            stemModelLabel = stemResult.model;
            stemProtocolLabel = stemResult.protocol;
            req.log.info({ separator: "cloudrun_demucs_htdemucs" }, "stem_split via Cloud Run Demucs htdemucs");
        } catch (demucsErr: any) {
            req.log.warn(
                { err: demucsErr?.message },
                "Cloud Run Demucs stem_split failed; trying Replicate then DSP",
            );
            const { result, routing, separator } = await runReplicateThenDsp("stem_split fallback");
            stemResult = result;
            stemRouting = routing;
            stemSeparatorLabel = separator;
            stemModelLabel = stemResult.model;
            stemProtocolLabel = stemResult.protocol;
        }
    } else {
        // No Cloud Run configured - try Replicate Demucs, then DSP.
        const { result, routing, separator } = await runReplicateThenDsp("stem_split");
        stemResult = result;
        stemRouting = routing;
        stemSeparatorLabel = separator;
        stemModelLabel = stemResult.model;
        stemProtocolLabel = stemResult.protocol;
    }

    const event: TelemetryEvent = {
        routing: stemRouting,
        parity: stemResult.kernelParity,
        efficiency: "1.0000",
        decayRate: "0.0000",
        sampleCount: String(stemResult.zipBuffer.length),
        timestamp: new Date().toISOString(),
    };
    telemetryBus.emit("run", event);

    if (req.dbUser) {
        db.insert(processRunsTable).values({
            userId: req.dbUser.id,
            routing: stemRouting,
            parity: stemResult.kernelParity,
            efficiency: 1,
            decayRate: 0,
            sampleCount: stemResult.zipBuffer.length,
            fileName: req.file.originalName,
        }).catch(() => {});
    }

    if (isFreeUse && usageField && usageUserId) {
        await incrementUsage(usageUserId, usageField);
        await incrementUsage(usageUserId, "totalDownloads");
        res.setHeader("X-GK-Free-Remaining", String(freeRemaining));
    }

    await unlink(filePath).catch(() => {});

    if (!(await consumeExportOrReject())) return;

    res.setHeader("Content-Type", "application/zip");
    res.setHeader("Content-Disposition", `attachment; filename="gravelking_stems.zip"`);
    res.setHeader("X-GK-Mode", "stem_split");
    res.setHeader("X-GK-Routing", stemRouting);
    res.setHeader("X-GK-Parity", stemResult.kernelParity);
    res.setHeader("X-GK-Stems", stemResult.stems.join(","));
    res.setHeader("X-GK-Separator", stemSeparatorLabel);
    res.setHeader("X-GK-Model", stemModelLabel);
    res.setHeader("X-GK-Protocol", stemProtocolLabel);
    res.setHeader("X-GK-Stack", stemResult.stack);
    streamBuffer(res, stemResult.zipBuffer);
    return;
} catch (err: any) {
    req.log.error({ err: err?.message ?? String(err) }, "stem_split failed");
    void logToolError("Singing Booth", "STEM_SPLIT", err);
    await unlink(filePath).catch(() => {});
    res.status(500).json({
        success: false,
        error: err?.message ?? err?.toString() ?? "Stem split processing failed. Try a shorter file or try again.",
    });
    return;
}

// — Studio-only gate for standard mode —————
if (!await hasStudio(req)) {
    await unlink(filePath).catch(() => {});
    res.status(403).json({
        success: false,
        error: "GravelKing Standard processing requires a GravelKing King subscription.",
        code: "STUDIO_REQUIRED",
    });
    return;
}

// — Standard mode: MLK V4 (Morris Law Kernel V4) via ffmpeg (local only, no external routing) ———
try {
    const { wavBuffer, parity, efficiency, decayRate, sampleCount } =
        await processWithMLKV3Ffmpeg(filePath, ext, multiplier);

    // Cleanup uploaded file after processing
    await unlink(filePath).catch(() => {});

    const event: TelemetryEvent = {
        routing: "local",
        parity,
        efficiency,
```

11. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/audio.ts | source continues without omission

```
        decayRate,
        sampleCount: String(sampleCount),
        timestamp: new Date().toISOString(),
    });
    telemetryBus.emit("run", event);

    if (req.dbUser) {
        db.insert(processRunsTable).values({
            userId: req.dbUser.id,
            routing: "local",
            parity,
            efficiency: parseFloat(efficiency),
            decayRate: parseFloat(decayRate),
            sampleCount,
            fileName: req.file.originalname,
        }).catch(() => {});
    }

    if (!(await consumeExportOrReject())) return;

    res.setHeader("Content-Type", "audio/wav");
    res.setHeader("Content-Disposition", "attachment; filename="gravelking_processed.wav");
    res.setHeader("X-GK-Routing", "local");
    res.setHeader("X-GK-Parity", parity);
    res.setHeader("X-GK-Efficiency", efficiency);
    res.setHeader("X-GK-Decay-Rate", decayRate);
    res.setHeader("X-GK-Sample-Count", String(sampleCount));
    res.setHeader("X-GK-Kernel", "MLK_v3");
    streamBuffer(res, wavBuffer);
} catch (err: any) {
    await unlink(filePath).catch(() => {});
    res.status(500).json({ success: false, error: err.message });
}
}
);

// — Vocal transcription —
// POST /api/audio/transcribe — accepts audio, returns { segments, lines, fullText }.
// Provider: Gemini Vertex AI (GCP SERVICE ACCOUNT). No Replicate fallback —
// when Gemini is unavailable the client falls back to manual tap-to-time.
// For known songs, prefer lrclib synced lyrics on the client before calling
// this endpoint.
audioRouter.post(
    "/audio/transcribe",
    audioRateLimit,
    upload.single("audio"),
    async (req: Request, res: Response) => {
        const pastedLyrics = typeof req.body?.lyrics === "string" ? req.body.lyrics : "";
        if (!req.file && pastedLyrics.trim()) {
            res.json({
                segments: [],
                lines: parsePastedLyrics(pastedLyrics, Number(req.body?.durationMs) || 0),
                fullText: pastedLyrics.trim(),
                fallback: "pasted-lyrics",
            });
            return;
        }
        if (!req.file) {
            res.status(400).json({ error: "No audio file uploaded." });
            return;
        }
        const filePath = req.file.path;
        try {
            const result = await transcribeWithGemini(filePath);
            const lines = transcriptToTimedLines(result.segments);
            if (lines.length > 0) {
                res.json({ ...result, lines });
                return;
            }
            if (pastedLyrics.trim()) {
                const durationMs = Math.round((await probeFileDuration(filePath)) * 1000);
                res.json({
                    ...result,
                    lines: parsePastedLyrics(pastedLyrics, durationMs),
                    fullText: pastedLyrics.trim(),
                    fallback: "pasted-lyrics",
                });
                return;
            }
            res.json({ ...result, lines });
        } catch (err: any) {
            const raw = String(err?.message ?? "");
            const busy = /quota|limit|429|thrott|/.test(raw);
            req.log.warn({ err: raw }, "Transcription unavailable; client falls back to tap-to-time");
            if (pastedLyrics.trim()) {
                const durationMs = Math.round((await probeFileDuration(filePath)) * 1000);
                res.json({
                    segments: [],
                    lines: parsePastedLyrics(pastedLyrics, durationMs),
                    fullText: pastedLyrics.trim(),
                    fallback: "pasted-lyrics",
                });
                return;
            }
            res.status(502).json({
                error: busy
                    ? "Auto-transcribe is busy right now. Use Tap-to-Time below to sync your lyrics."
                    : "Couldn't auto-transcribe this track. Use Tap-to-Time below to sync your lyrics.",
                fallback: "tap-timing",
            });
        } finally {
            await unlink(filePath).catch(() => {});
        }
    },
);

export default audioRouter;
```

12. VIDEO EXTRACTION & INGESTION

artifacts/api-server/src/routes/convert.ts | 167 lines | 5,755 bytes | sha256 4b95149a495e214b7e23941f7370a42c82bd8bf3513fddfbaafa118016684c4a

```
import { Router, type Request, type Response } from "express";
import multer from "multer";
import { execFile } from "child_process";
import { promisify } from "util";
import { unlink, readFile } from "fs/promises";
import { randomUUID } from "crypto";
import { sanitizeExt } from "../lib/audioGuards";
import { streamBuffer } from "../lib/streamResponse";
import { getUsageUser } from "../lib/usage";
import { checkExportQuota, consumeExport, exportLimitPayload } from "../lib/exportQuota";
import { resolveTier } from "../lib/entitlement";
import type { User } from "@workspace/db";

const execFileAsync = promisify(execFile);

const upload = multer({
  storage: multer.diskStorage({
    destination: "/tmp",
    filename: (_req, file, cb) => cb(null, `gk_conv_${randomUUID()}.${sanitizeExt(file.originalname)}`),
  }),
  // The converter also accepts video containers and must honor the public
  // 250 MB upload contract before ffmpeg extracts their first audio stream.
  limits: { fileSize: 250 * 1024 * 1024 },
});

const convertRouter = Router();

type OutputFormat = "mp3" | "wav" | "flac" | "m4a" | "ogg";

// Default codec + per-format args; bitrate/sample rate are overridden below
// when the client supplies them.
const DEFAULT_RATE = 44100;
const VALID_RATES = [22050, 44100, 48000];
const DEFAULT_BITRATE_K: Record<OutputFormat, number> = {
  mp3: 320, wav: 1411, flac: 1411, m4a: 256, ogg: 192,
};
// Bitrate applies only to lossy formats (lossless WAV/FLAC are bitrate-free).
const LOSSLESS: Record<OutputFormat, boolean> = {
  mp3: false, wav: true, flac: true, m4a: false, ogg: false,
};

const FORMAT_CODEC: Record<OutputFormat, string[]> = {
  mp3: ["-acodec", "libmp3lame"],
  wav: ["-acodec", "pcm_s16le"],
  flac: ["-acodec", "flac"],
  m4a: ["-acodec", "aac"],
  ogg: ["-acodec", "libvorbis"],
};

function buildFormatArgs(
  format: OutputFormat,
  samplerate: number,
  bitrateK: number | null,
): string[] {
  const args = [...FORMAT_CODEC[format], "-ar", String(samplerate)];
  if (bitrateK !== null && !LOSSLESS[format]) {
    // OGG falls back to its own default in bitrate mode; q:a default omitted.
    if (format === "ogg") args.push("-b:a", `${bitrateK}k`);
    else args.push("-b:a", `${bitrateK}k`);
  }
  return args;
}

const MIME: Record<OutputFormat, string> = {
  mp3: "audio/mpeg",
  wav: "audio/wav",
  flac: "audio/flac",
  m4a: "audio/mp4",
  ogg: "audio/ogg",
};

/**
 * POST /api/convert
 * Accepts any audio or video file, extracts/converts the audio stream,
 * and returns the result in the requested format.
 *
 * Body (multipart/form-data):
 * file - the source file (audio or video)
 * format - output format: mp3 | wav | flac | m4a | ogg (default: mp3)
 */
convertRouter.post(
  "/convert",
  upload.single("file"),
  async (req: Request, res: Response) => {
    if (!req.file) {
      res.status(400).json({ error: "No file uploaded." });
      return;
    }

    const inputPath = req.file.path;
    const rawFormat = ((req.body.format as string) ?? "mp3").toLowerCase() as OutputFormat;
    const format: OutputFormat = FORMAT_CODEC[rawFormat] ? rawFormat : "mp3";
    const tier = await resolveTier(req);
    if (tier === "free") {
      await unlink(inputPath).catch(() => {});
      res.status(402).json({
        success: false,
        code: "PRO_REQUIRED",
        feature: "converter",
        error: "Converter is available on Pro and King plans.",
        fallback: { action: "subscribe", url: "/pricing" },
      });
      return;
    }

    // Sample rate (Hz) - whitelisted values, default 44100.
    const rawRate = Number(String(req.body.samplerate ?? "").trim());
    const samplerate = VALID_RATES.includes(rawRate) ? rawRate : DEFAULT_RATE;

    // Bitrate (kbps) - numeric, clamped to a sane range, ignored for lossless.
    const rawBitrate = Number(String(req.body.bitrateK ?? "").trim());
    const bitrateK = Number.isFinite(rawBitrate) && rawBitrate > 0
      ? Math.round(Math.min(320, Math.max(64, rawBitrate)))
  }
```

12. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/routes/convert.ts | source continues without omission

```
      : null;

// MP3 is unlimited for paid plans. Only WAV consumes the tier-aware quota.
let exportUser: User | null = null;
if (format === "wav") {
  exportUser = await getUser(req, res);
  const quota = checkExportQuota(exportUser);
  if (!quota.allowed) {
    await unlink(inputPath).catch(() => {});
    res.status(429).json(exportLimitPayload(quota));
    return;
  }
}

const outId = randomUUID();
const outPath = `/tmp/gk_conv_out_${outId}.${format}`;

try {
  // -vn: strip video stream; ffmpeg handles any audio/video container as input
  await execFileAsync("ffmpeg", [
    "-y",
    "-i", inputPath,
    "-vn",
    ...buildFormatArgs(format, samplerate, bitrateK),
    outPath,
  ], { timeout: 180_000, maxBuffer: 200 * 1024 * 1024 });

  const buf = await readFile(outPath);
  const baseName = req.file.originalname.replace(/\.([^.]*)$/, "");

  // Consume only after a successful conversion.
  if (exportUser) {
    const quota = await consumeExport(exportUser);
    if (!quota.allowed) {
      res.status(429).json(exportLimitPayload(quota));
      return;
    }
  }

  res.setHeader("Content-Type", MIME[format]);
  res.setHeader("Content-Disposition", `attachment; filename="${baseName}.${format}"`);
  res.setHeader("X-GK-Format", format);
  streamBuffer(res, buf);
} catch (err: unknown) {
  const msg = err instanceof Error ? err.message : "Conversion failed";
  res.status(500).json({ error: msg });
} finally {
  await unlink(inputPath).catch(() => {});
  await unlink(outPath).catch(() => {});
}
});

export default convertRouter;
```

13. VIDEO EXTRACTION & INGESTION

artifacts/api-server/src/middlewares/validateAssetIngestion.ts | 366 lines | 14,600 bytes | sha256 c294a43c3978b39455216ea9d8dd2e6b49e0262fce472028a5bf6b4bebf3954

```
import { Request, Response, NextFunction } from "express";
import { execFile } from "child_process";
import { promisify } from "util";
import { randomUUID } from "crypto";
import { writeFile, unlink } from "fs/promises";
import { logger } from "../lib/logger";
import {
  scanCommercialFingerprint,
  type CommercialFingerprintMatch,
} from "../lib/commercialFingerprint";

const execFileAsync = promisify(execFile);

/**
 * Known copyrighted strings / public-blacklist patterns.
 * In production these would be loaded from a managed database or external
 * content-registry API (e.g. ICE, Blokur, Audible Magic). This array provides
 * the structural hook and representative examples.
 */
const COPYRIGHTED_STRINGS = [
  "beatles", "madonna", "drake", "taylor swift", "kanye", "eminem",
  "beyonce", "coldplay", "rihanna", "adele", "bruno mars", "ed sheeran",
  "billie eilish", "the weeknd", "metallica", "nirvana", "queen",
  "michael jackson", "prince", "elvis", "frank sinatra", "bob dylan",
];

/** ISRC / ISWC / UPC-ish collision patterns. */
const ISRC_ISWC_RE = /\b([A-Z]{2}-?[A-Z0-9]{3})-?\d{2}-?\d{5}[ISWC\s*T-?\d{9,11}[US-S1Z-799-700001]\b/i;

/** Broad label/artist name collision patterns that look like commercial metadata. */
const COMMERCIAL_LABEL_RE = /\b(sony|universal|warner|atlantic|columbia|interscope|def jam|roc nation|capitol|republic|emi|virgin|cbs|bmg|abkco)\b/i;

export interface IngestionValidationResult {
  passed: boolean;
  code?: string;
  reasons: string[];
  metadata: {
    blacklistedHits: string[];
    isrcIswcHits: string[];
    labelHits: string[];
    spectralUniqueness: number | null;
    commercialFingerprint: {
      provider: "acrcloud" | "local-signature";
      status: "not_run" | "no_match" | "local_no_match" | "match" | "unavailable";
      matches: CommercialFingerprintMatch[];
      scope?: "global_commercial" | "local_catalog";
    };
  };
  authorAssertion: boolean;
}

function normalizeMeta(value: unknown): string {
  if (value === null || value === undefined) return "";
  return String(value).toLowerCase().trim();
}

function scanMetadata(body: Record<string, unknown>, file?: Express.Multer.File): { blacklistedHits: string[]; isrcIswcHits: string[]; labelHits: string[] } {
  const haystackParts: string[] = [];

  // Include free-form body fields. Dedicated registry-ID fields are excluded:
  // they are intentionally supplied for certificate binding and are not
  // evidence that a title, prompt, or filename copied commercial metadata.
  const registryIdFields = new Set(["ipiNumber", "iswc", "isrc"]);
  for (const [key, v] of Object.entries(body)) {
    if (registryIdFields.has(key)) continue;
    haystackParts.push(normalizeMeta(v));
  }

  // Include file metadata if present
  if (file) {
    haystackParts.push(normalizeMeta(file.originalname));
  }

  const haystack = haystackParts.join(" ");
  const blacklistedHits: string[] = [];
  for (const term of COPYRIGHTED_STRINGS) {
    if (haystack.includes(term)) blacklistedHits.push(term);
  }

  // ISRC_ISWC_RE is intentionally non-global for one-off checks. Calling
  // exec() repeatedly on a non-global regex never advances lastIndex and used
  // to grow this array until the process OOM'd whenever a match existed.
  const isrcIswcHits = haystack.match(
    new RegExp(ISRC_ISWC_RE.source, `${ISRC_ISWC_RE.flags.replace(/g/g, "")}g`)
  ) ?? [];

  const labelHits: string[] = [];
  let m: RegExpExecArray | null;
  const labelRe = new RegExp(COMMERCIAL_LABEL_RE.source, "gi");
  while ((m = labelRe.exec(haystack)) !== null) {
    labelHits.push(m[0]);
  }

  return { blacklistedHits, isrcIswcHits, labelHits };
}

/**
 * Lightweight spectral uniqueness check.
 * Uses ffmpeg to extract a 512-bin FFT magnitude summary from the first 30s
 * and computes a perceptual-rhythm signature (energy band ratios). Returns a
 * 0-1 score; very low scores indicate silence / near-silence / pure tones that
 * are often copied test signals or fraudulent uploads.
 */
async function computeSpectralUniqueness(audioBuffer: Buffer, ext: string): Promise<number | null> {
  const id = randomUUID();
  const tmpPath = `/tmp/gk_ingest_probe_${id}.${ext} || "wav"`;
  try {
    await writeFile(tmpPath, audioBuffer);

    // Extract a 1D energy summary: mean magnitude per FFT bin across first 30s.
    const { stdout } = await execFileAsync(
      "ffmpeg",
      [
```

13. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/middlewares/validateAssetIngestion.ts | source continues without omission

```

    "-y", "-i", tmpPath,
    "-t", "30",
    "-ac", "2", "-ar", "44100",
    "-af", "astats=metadata=1,amovie=/tmp/nonexistent,join",
    "-f", "null", "-.",
  ],
  { timeout: 30_000 }
).catch(() => ({ stdout: "" }));

// Fallback: use a robust entropy-based signature via ebur128 + silencedetect.
// ffmpeg always writes loudness stats to stderr, not stdout.
const loudnessResult = await execFileAsync(
  "ffmpeg",
  [
    "-y", "-i", tmpPath,
    "-t", "30",
    "-af", "ebur128=peak=true",
    "-f", "null", "-.",
  ],
  { timeout: 30_000 }
).catch((e: { stderr?: string; stdout?: string }) => ({ stdout: e.stdout ?? "", stderr: e.stderr ?? "" }));
const loudnessOutput = loudnessResult.stderr ?? loudnessResult.stdout ?? "";

// ebur128 emits per-frame lines like "I: -70.0 LUFS" while the file plays,
// then a final Summary block with the true integrated value. Always parse
// from the Summary block; per-frame values are still converging.
const summaryIdx = loudnessOutput.indexOf("Summary:");
const loudnessSummary = summaryIdx >= 0 ? loudnessOutput.slice(summaryIdx) : loudnessOutput;

const integratedMatch = loudnessSummary.match(/I:\s*([-\d.]+)\s*LUFS/);
const peakMatch = loudnessSummary.match(/Peak:\s*([-\d.]+)\s*dBFS/);
const lufs = integratedMatch ? parseFloat(integratedMatch[1]) : -70;
const peakDb = peakMatch ? parseFloat(peakMatch[1]) : -70;

// Score: penalize silence, extreme clipping, and single-tone signals.
if (lufs < -55) return 0.05; // near silence
if (peakDb > -0.5) return 0.15; // heavily clipped / artificial maxed signal

// Heuristic: a normal stereo track has a decent LUFS range; map -55..-14 to 0..1
const score = Math.min(1, Math.max(0, (lufs + 55) / 41));
return Number(score.toFixed(3));
} catch (err) {
  logger.warn({ err }, "spectral uniqueness probe failed; allowing ingestion");
  return null;
} finally {
  await unlink(tmpPath).catch(() => {});
}
}

/**
 * Middleware factory. Attach before any route that produces cryptographic hashes
 * or LSB steganographic watermarks. Checks metadata, spectral uniqueness, and
 * requires an explicit author ownership assertion.
 */
export function validateAssetIngestion(
  options: {
    requireAudio?: boolean;
    commercialFingerprint?: boolean;
    allowUncertifiedFallback?: boolean;
  } = {},
) {
  return async (req: Request, res: Response, next: NextFunction): Promise<void> => {
    const uploadedFiles: Express.Multer.File[] = req.file
      ? [req.file]
      : Array.isArray(req.files)
        ? req.files
        : req.files
          ? Object.values(req.files).flat()
          : [];
    const audioFiles = uploadedFiles.filter(
      (candidate) =>
        candidate.mimetype?.startsWith("audio/") ||
        candidate.fieldname === "audio" ||
        candidate.fieldname === "audio_full" ||
        candidate.fieldname === "audio_preview" ||
        candidate.fieldname === "tracks",
    );
    const file = audioFiles[0];
    const body = (req.body as Record<string, unknown>) ?? {};

    const result: IngestionValidationResult = {
      passed: false,
      reasons: [],
      metadata: {
        blacklistedHits: [],
        isrcIswcHits: [],
        labelHits: [],
        spectralUniqueness: null,
        commercialFingerprint: {
          provider: "acricloud",
          status: "not_run",
          matches: [],
        },
      },
    },
    authorAssertion: false,
  };

  // 1. Terms of Ownership Assertion
  const authorAssertion = normalizeMeta(body.author_assertion) === "true" || body.author_assertion === true;
  result.authorAssertion = authorAssertion;
  if (!authorAssertion) {
    result.reasons.push("Missing signed author_assertion warranty");
  }

  // 2. Metadata Check
  const meta = scanMetadata(body, file as Express.Multer.File | undefined);
  result.metadata.blacklistedHits = meta.blacklistedHits;
  result.metadata.isrcIswcHits = meta.isrcIswcHits;
  result.metadata.labelHits = meta.labelHits;

  if (meta.blacklistedHits.length > 0) {
    result.reasons.push(`Blacklisted copyright strings detected: ${meta.blacklistedHits.join(", ")}`);
  }
}
```

13. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/middlewares/validateAssetIngestion.ts | source continues without omission

```

    if (meta.isrcIswcHits.length > 0) {
      result.reasons.push(`ISRC/ISWC collision patterns detected: ${meta.isrcIswcHits.join(", ")}`);
    }
    if (meta.labelHits.length > 0) {
      result.reasons.push(`Commercial label metadata detected: ${meta.labelHits.join(", ")}`);
    }

    // 3. Local signal screening followed by commercial catalog recognition.
    // Every uploaded audio component is checked; a multi-track studio mix
    // cannot hide an external match in a later file.
    if (audioFiles.length && options.requireAudio !== false) {
      const uniquenessScores: number[] = [];
      for (const audioFile of audioFiles) {
        const audioBuffer = (audioFile as Express.Multer.File & { buffer?: Buffer }).buffer;
        let tempPath: string | null = null;
        try {
          let scanPath = audioFile.path;
          let buf = audioBuffer;
          if (!Buffer.isBuffer(buf) && scanPath) {
            const ( readFile ) = await import("node:fs/promises");
            buf = await readFile(scanPath);
          } else if (Buffer.isBuffer(buf) && !scanPath) {
            tempPath = `/tmp/gk_ingest_${randomUUID()}.audio`;
            await writeFile(tempPath, buf);
            scanPath = tempPath;
          }
          if (!Buffer.isBuffer(buf) || !scanPath) continue;

          const ext = (audioFile.originalname?.split(".").pop() ?? "wav").toLowerCase();
          const uniqueness = await computeSpectralUniqueness(buf, ext);
          if (uniqueness !== null) uniquenessScores.push(uniqueness);
          if (uniqueness !== null && uniqueness < 0.1) {
            result.reasons.push(
              `Audio spectral uniqueness too low (${uniqueness}); possible silence, test tone, or copied signal`,
            );
          }

          // Avoid provider cost when local validation has already rejected the
          // asset, but never stamp a locally-valid external upload without the
          // commercial recognition result.
          if (result.reasons.length === 0 && options.commercialFingerprint === true) {
            const commercial = await scanCommercialFingerprint(scanPath);
            result.metadata.commercialFingerprint.status = commercial.status;
            result.metadata.commercialFingerprint.provider = commercial.provider;
            if ("scope" in commercial) result.metadata.commercialFingerprint.scope = commercial.scope;
            if (commercial.status === "unavailable") {
              logger.warn(
                { reason: commercial.reason, filename: audioFile.originalname },
                "commercial fingerprint scan temporarily unavailable",
              );
              (req as Request & { ingestionValidation?: IngestionValidationResult }).ingestionValidation = result;
              if (options.allowUncertifiedFallback === true) {
                // Provider availability controls only whether a stamp can be
                // emitted. The underlying audio tool must still run.
                next();
                return;
              }
              res.status(503).json({
                success: false,
                code: "SCAN_TEMPORARILY_UNAVAILABLE",
                error: "Scan temporarily unavailable",
                message: "Your audio was not stamped. Please try again later.",
              });
              return;
            }
            if (commercial.status === "match") {
              result.metadata.commercialFingerprint.matches.push(...commercial.matches);
              const identified = commercial.matches
                .slice(0, 3)
                .map((match) => {
                  const artist = match.artists?.join(", ");
                  return [match.title, artist].filter(Boolean).join(" - ");
                })
                .filter(Boolean);
              result.reasons.push(
                `Commercial fingerprint match detected${identified.length ? `: ${identified.join(", ")}` : ""}`,
              );
            }
          }
        } catch (err) {
          logger.warn({ err, filename: audioFile.originalname }, "could not scan uploaded audio");
          (req as Request & { ingestionValidation?: IngestionValidationResult }).ingestionValidation = result;
          result.metadata.commercialFingerprint.status = "unavailable";
          if (options.allowUncertifiedFallback === true) {
            next();
            return;
          }
          res.status(503).json({
            success: false,
            code: "SCAN_TEMPORARILY_UNAVAILABLE",
            error: "Scan temporarily unavailable",
            message: "Your audio was not stamped. Please try again later.",
          });
          return;
        } finally {
          if (tempPath) await unlink(tempPath).catch(() => {});
        }
      }
      result.metadata.spectralUniqueness =
        uniquenessScores.length > 0 ? Math.min(...uniquenessScores) : null;
    }

    result.passed = result.reasons.length === 0;

    // Attach result so downstream routes can include it in audit logs / certs
    (req as Request & { ingestionValidation?: IngestionValidationResult }).ingestionValidation = result;

    if (!result.passed) {
      if (
        options.allowUncertifiedFallback === true &&
        result.metadata.commercialFingerprint.status === "match"
      ) {
        // A catalog match suppresses the stamp, not the mastering operation.

```

13. VIDEO EXTRACTION & INGESTION (continued)

artifacts/api-server/src/middlewares/validateAssetIngestion.ts | source continues without omission

```
        next();
        return;
    }
    logger.warn({ result }, "ingestion validation blocked asset");
    res.status(422).json({
      success: false,
      code: "ERR_COPYRIGHT_WARRANT_REQUIRED",
      error: "Asset ingestion failed copyright/ownership validation.",
      reasons: result.reasons,
      metadata: result.metadata,
    });
    return;
  }
  next();
};
}
```

export default validateAssetIngestion;

```
declare global {
  namespace Express {
    interface Request {
      ingestionValidation?: IngestionValidationResult;
    }
  }
}
```

14. VAULT / LIBRARY ORDERING

artifacts/api-server/src/routes/tracks.ts | 961 lines | 35,657 bytes | sha256 fedd8723d60237e1596b88ad19c3eee21efb99bf2ae7ad74506bee90aaeb7f7c

```
import { Router, type Request, type Response } from "express";
import { db, tracksTable, purchasedTracksTable, usersTable } from "@workspace/db";
import { eq, and, inArray, desc, asc } from "drizzle-orm";
import { resolveTier } from "../lib/entitlementment";
import { ObjectStorageService, saveObjectWithFallback, getObjectFileWithFallback } from "../lib/objectStorage";
import { sanitizeExt } from "../lib/audioGuards";
import { getUncachableStripeClient } from "../stripeClient";
import { storage } from "../storage";
import multer from "multer";
import { randomUUID } from "node:crypto";
import { validateAssetIngestion } from "../middlewares/validateAssetIngestion";
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import { readFileSync, writeFileSync, unlinkSync } from "node:fs";
import { tmpdir } from "node:os";
import { join } from "node:path";
import { isAdminAutomationAuthenticated } from "../lib/adminAuth";
import { recordAnalyticsEvent } from "../analytics";
import { CREDIT_COSTS, spendCredits } from "../lib/credits";
```

```
const execFileAsync = promisify(execFile);
```

```
const MAX_PREVIEW_SECONDS = 30;
```

```
/** Returns duration in seconds for an in-memory audio buffer, using ffprobe. */
async function getAudioDurationSeconds(buffer: Buffer, ext: string): Promise<number> {
  const tmpPath = join(tmpdir(), `gkp-preview-${randomUUID()}${ext}`);
  try {
    writeFileSync(tmpPath, buffer);
    const { stdout } = await execFileAsync("ffprobe", [
      "-v", "quiet",
      "-print_format", "json",
      "-show_format",
      tmpPath,
    ]);
    const parsed = JSON.parse(stdout) as { format?: { duration?: string } };
    return parseFloat(parsed.format?.duration ?? "0");
  } finally {
    try { unlinkSync(tmpPath); } catch { /* ignore */ }
  }
}
```

```
const router = Router();
const objectStorageService = new ObjectStorageService();
```

```
/**
 * Columns safe to return to public callers – deliberately excludes audioFullKey.
 * audioFullKey must never appear in unauthenticated API responses; it is only
 * read server-side inside the gated /api/tracks/:id/download handler.
 */
```

```
const publicTrackCols = {
  id: tracksTable.id,
  title: tracksTable.title,
  artistName: tracksTable.artistName,
  audioPreviewKey: tracksTable.audioPreviewKey,
  coverArtKey: tracksTable.coverArtKey,
  price: tracksTable.price,
  status: tracksTable.status,
  stripeProductId: tracksTable.stripeProductId,
  stripePriceId: tracksTable.stripePriceId,
  submittedByUserId: tracksTable.submittedByUserId,
  createdAt: tracksTable.createdAt,
  updatedAt: tracksTable.updatedAt,
};
```

```
/**
 * Library-only columns: everything public PLUS the generated lyrics text.
 * Lyrics are only returned to the track's owner via /api/library – never on
 * the unauthenticated /tracks listings.
 */
```

```
const libraryTrackCols = {
  ...publicTrackCols,
  lyricsText: tracksTable.lyricsText,
  isPinned: purchasedTracksTable.isPinned,
};
```

```
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 150 * 1024 * 1024 },
});
```

```
const SEVEN_DAYS_MS = 7 * 24 * 60 * 60 * 1000;
```

```
async function saveFileToBucket(buffer: Buffer, key: string, contentType: string): Promise<void> {
  const bucketId = process.env.DEFAULT_OBJECT_STORAGE_BUCKET_ID ?? "";
  // Falls back to the owner-project bucket when the Replit-managed bucket
  // rejects the write (platform-side 403 IAM breakage).
  await saveObjectWithFallback(bucketId, key, buffer, { contentType });
}
```

```
/** Derive gk session-based userId. Returns null if no session cookie present. */
async function resolveSessionUser(req: Request): Promise<{ sessionId: string; } | null> {
  // Clerk-signed-in users FIRST – they never carry a gk session cookie, so
  // without this check every vault/download route 401'd for logged-in users
  // (same bug class as the subscription-status fix).
  if (req.dbUser) {
    return { sessionId: "", userId: req.dbUser.id };
  }
  const sessionId = (req.cookies as Record<string, string>)?.gk_session;
  if (!sessionId) return null;
  const user = await storage.getUserBySession(sessionId);
  if (!user) return null;
  return { sessionId, userId: user.id };
}
```

```
/** GET /api/tracks – list all accepted tracks, newest first. audioFullKey excluded. */
router.get("/tracks", async (_req: Request, res: Response) => {
  try {
    const rows = await db
      .select(publicTrackCols)
      .from(tracksTable)
      .where(and(
        eq(tracksTable.status, "accepted"),
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```
        eq(tracksTable.takenDown, false),
      ))
      .orderBy(desc(tracksTable.createdAt));
    res.json({ tracks: rows });
  } catch (_err) {
    res.status(500).json({ error: "Failed to list tracks" });
  }
});

/** GET /api/tracks/artist/:artist - list accepted tracks by artist, newest first. audioFullKey excluded. */
router.get("/tracks/artist/:artist", async (req: Request, res: Response) => {
  const artist = req.params.artist as string;
  try {
    const rows = await db
      .select(publicTrackCols)
      .from(tracksTable)
      .where(and(
        eq(tracksTable.artistName, artist),
        eq(tracksTable.status, "accepted"),
        eq(tracksTable.takenDown, false),
      ))
      .orderBy(desc(tracksTable.createdAt));
    res.json({ tracks: rows });
  } catch (_err) {
    res.status(500).json({ error: "Failed to list tracks" });
  }
});

/**
 * GET /api/tracks/submit-eligibility
 *
 * Returns the session user's eligibility to submit a track. Must be defined
 * before /tracks/:id routes to avoid param capture.
 *
 * Response shape:
 * { eligible: true, tier }
 * { eligible: false, tier: "free", reason: "free_tier" }
 * { eligible: false, tier: "pro"|"king", reason: "cooldown",
 *   cooldownDaysLeft: number, nextSubmissionDate: string }
 */
router.get("/tracks/submit-eligibility", async (req: Request, res: Response) => {
  // Dev privilege: an admin-authenticated request can always submit, and its
  // submissions go live instantly (see POST /tracks/submit).
  const { isAdminAuthenticated } = await import("../lib/adminAuth");
  if (isAdminAuthenticated(req)) {
    res.json({ eligible: true, tier: "admin" });
    return;
  }

  const tier = await resolveTier(req);
  if (tier === "free") {
    res.json({ eligible: false, tier: "free", reason: "free_tier" });
    return;
  }

  // node_auditor - unlimited, never on cooldown
  if (tier === "node_auditor") {
    res.json({ eligible: true, tier: "node_auditor" });
    return;
  }

  // Pro / King - check 7-day cooldown
  let userId: string | null = null;
  if (req.dbUser) {
    userId = req.dbUser.id;
  } else {
    const sessionId = (req.cookies as Record<string, string>)?.gk_session;
    if (sessionId) {
      const user = await storage.getUserBySession(sessionId);
      if (user) userId = user.id;
    }
  }
  if (userId) {
    const [dbUser] = await db.select().from(usersTable).where(eq(usersTable.id, userId));
    const last = dbUser?.lastSubmissionDate;
    const now = new Date();
    if (last && now.getTime() - new Date(last).getTime() < SEVEN_DAYS_MS) {
      const msRemaining = SEVEN_DAYS_MS - (now.getTime() - new Date(last).getTime());
      const daysLeft = Math.ceil(msRemaining / (24 * 60 * 60 * 1000));
      const nextDate = new Date(new Date(last).getTime() + SEVEN_DAYS_MS).toISOString();
      res.json({ eligible: false, tier, reason: "cooldown", cooldownDaysLeft: daysLeft, nextSubmissionDate: nextDate });
      return;
    }
  }

  res.json({ eligible: true, tier });
});

/** GET /api/library - tracks the user has purchased (gk_session-scoped, no auth wall) */
router.get("/library", async (req: Request, res: Response) => {
  const session = await resolveSessionUser(req);
  if (!session) {
    res.json({ tracks: [] });
    return;
  }

  try {
    const purchased = await db
      .select()
      .from(purchasedTracksTable)
      .where(eq(purchasedTracksTable.userId, session.userId));

    const trackIds = purchased.map(r => r.trackId);
    if (trackIds.length === 0) {
      res.json({ tracks: [] });
      return;
    }

    const tracks = await db
      .select(libraryTrackCols)
      .from(tracksTable)
      .innerJoin(purchasedTracksTable, and(
        eq(purchasedTracksTable.trackId, tracksTable.id),
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```
        eq(purchasedTracksTable.userId, session.userId),
      ))
      .where(inArray(tracksTable.id, trackIds))
      .orderBy(desc(purchasedTracksTable.isPinned), desc(tracksTable.createdAt));

    res.json({ tracks });
  } catch (_err) {
    res.status(500).json({ error: "Failed to load library" });
  }
});

async function resolveLibraryOwner(req: Request): Promise<string | null> {
  const session = await resolveSessionUser(req);
  return session?.userId ?? null;
}

/** Update the current user's library pin state. */
router.patch("/library/:id/pin", async (req: Request, res: Response): Promise<void> => {
  const userId = await resolveLibraryOwner(req);
  if (!userId) { res.status(401).json({ error: "Sign in required" }); return; }
  const trackId = String(req.params.id);
  const pinned = Boolean((req.body as { pinned?: unknown }).pinned);
  const [row] = await db.update(purchasedTracksTable)
    .set({ isPinned: pinned })
    .where(and(eq(purchasedTracksTable.userId, userId), eq(purchasedTracksTable.trackId, trackId)))
    .returning({ isPinned: purchasedTracksTable.isPinned });
  if (!row) { res.status(404).json({ error: "Track not found in your library" }); return; }
  res.json(row);
});

/** Remove a track from the current user's library without deleting shared audio. */
router.delete("/library/:id", async (req: Request, res: Response): Promise<void> => {
  const userId = await resolveLibraryOwner(req);
  if (!userId) { res.status(401).json({ error: "Sign in required" }); return; }
  const trackId = String(req.params.id);
  const deleted = await db.delete(purchasedTracksTable)
    .where(and(eq(purchasedTracksTable.userId, userId), eq(purchasedTracksTable.trackId, trackId)))
    .returning({ id: purchasedTracksTable.id });
  if (!deleted.length) { res.status(404).json({ error: "Track not found in your library" }); return; }
  res.json({ ok: true });
});

/** Replace a library owner's cover art. */
router.patch(
  "/library/:id/cover",
  upload.single("cover_art"),
  async (req: Request, res: Response): Promise<void> => {
    const userId = await resolveLibraryOwner(req);
    if (!userId) { res.status(401).json({ error: "Sign in required" }); return; }
    const trackId = String(req.params.id);
    const file = req.file;
    if ((file || !file.mimetype.startsWith("image/")) {
      res.status(400).json({ error: "Choose an image file" }); return;
    }
    const [owned] = await db.select({ id: tracksTable.id })
      .from(tracksTable)
      .innerJoin(purchasedTracksTable, and(
        eq(purchasedTracksTable.trackId, tracksTable.id),
        eq(purchasedTracksTable.userId, userId),
      ))
      .where(eq(tracksTable.id, trackId));
    if (!owned) { res.status(404).json({ error: "Track not found in your library" }); return; }
    const key = `tracks/${trackId}/cover_art_user_${randomUUID()}.${sanitizeExt(file.originalname)}`;
    await objectStorageService.savePublicObject(key, file.buffer, file.mimetype);
    const [updated] = await db.update(tracksTable).set({ coverArtKey: key, updatedAt: new Date() })
      .where(eq(tracksTable.id, trackId)).returning({ coverArtKey: tracksTable.coverArtKey });
    res.json(updated);
  },
);

/**
 * POST /api/tracks/submit - submit a track via multipart file upload.
 * Requires any paid subscription tier (Pro, King, Node Auditor).
 * Rate-limited to once per 7 days - except node_auditor which is unlimited.
 */
router.post(
  "/tracks/submit",
  upload.fields([
    { name: "audio_full", maxCount: 1 },
    { name: "audio_preview", maxCount: 1 },
    { name: "cover_art", maxCount: 1 },
  ]),
  validateAssetIngestion({ requireAudio: true }),
  async (req: Request, res: Response) => {
    // Dev privileges: an admin-authenticated request (gk_admin cookie or
    // x-admin-key header, both derived from ADMIN_KEY) bypasses the paid-tier
    // gate, the 7-day cooldown, and moderation - the track goes live instantly.
    const { isAdminAuthenticated } = await import("../lib/adminAuth");
    const isAdmin = isAdminAuthenticated(req);

    let tier = "admin";
    if (!isAdmin) {
      tier = await resolveTier(req);
      if (tier === "free") {
        res.status(403).json({ error: "A Pro subscription is required to submit tracks. Subscribe at /pricing." });
        return;
      }
    }

    const body = req.body as Record<string, string>;
    const title = body.title?.trim();
    const artistName = body.artistName?.trim();

    const files = req.files as Record<string, Express.Multer.File[]> | undefined;
    const audioFull = files?.audio_full?.[0];
    const audioPreview = files?.audio_preview?.[0];
    const coverArt = files?.cover_art?.[0];

    if (!title || !artistName || !audioFull || !audioPreview || !coverArt) {
      res.status(400).json({ error: "Missing required fields or files (audio_full, audio_preview, cover_art)" });
      return;
    }
  }
}
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```
// Enforce preview ≤30 s server-side using fprobe so this cannot be bypassed via the API.
const previewDuration = await getAudioDurationSeconds(
  audioPreview.buffer,
  `.${sanitizeExt(audioPreview.originalname)}`,
);
if (previewDuration > MAX_PREVIEW_SECONDS) {
  res.status(400).json({
    error: `Preview clip must be ${MAX_PREVIEW_SECONDS} seconds or less (yours is ${Math.round(previewDuration)}s).`,
  });
  return;
}

const now = new Date();

// Resolve the submitting user. OIDC auth takes priority; fall back to gk_session.
let user: Awaited<ReturnType<typeof storage.getOrCreateUser>> | null = null;
let userId: string | null = null;
if (req.dbUser) {
  userId = req.dbUser.id;
  user = req.dbUser;
} else {
  const sessionId = (req.cookies as Record<string, string>)?.gk_session;
  if (sessionId) {
    user = await storage.getOrCreateUser(sessionId);
    userId = user.id;
  }
}
const submittedByUserId = userId ?? "admin";

// Node Auditor has unlimited submissions; Pro/King tiers are rate-limited.
// to once per 7 days. Admins and node_auditor are exempt.
if (!isAdmin && tier !== "node_auditor" && user) {
  const [dbUser] = await db.select().from(usersTable).where(eq(usersTable.id, user.id));
  const last = dbUser?.lastSubmissionDate;
  if (last && (now.getTime() - new Date(last).getTime()) < SEVEN_DAYS_MS) {
    const daysLeft = Math.ceil((SEVEN_DAYS_MS - (now.getTime() - new Date(last).getTime())) / (24 * 60 * 60 * 1000));
    res.status(429).json({ error: `Track submissions are limited to once per 7 days. Try again in ${daysLeft} day(s).` });
    return;
  }
}

try {
  const id = randomUUID();
  // Full audio is stored under private/ so it is never reachable via the
  // public-objects route regardless of PUBLIC_OBJECT_SEARCH_PATHS config.
  const audioFullKey = `private/tracks/${id}/audio_full.${sanitizeExt(audioFull.originalname)}`;
  const audioPreviewKey = `tracks/${id}/audio_preview.${sanitizeExt(audioPreview.originalname)}`;
  const coverArtKey = `tracks/${id}/cover_art.${sanitizeExt(coverArt.originalname)}`;

  await Promise.all([
    // Full (paid) audio: bucket root under private/ – read directly by the gated download route.
    saveFileToBucket(audioFull.buffer, audioFullKey, audioFull.mimetype),
    // Preview + cover are PUBLIC assets: they must be written under the public
    // search path so the /storage/public-objects route (searchPublicObject) finds them.
    objectStorageService.savePublicObject(audioPreviewKey, audioPreview.buffer, audioPreview.mimetype),
    objectStorageService.savePublicObject(coverArtKey, coverArt.buffer, coverArt.mimetype),
  ]);

  const [track] = await db
    .insert(tracksTable)
    .values({
      title,
      artistName,
      audioFullKey,
      audioPreviewKey,
      coverArtKey,
      // Dev uploads go live immediately; everyone else waits for moderation.
      status: isAdmin ? "accepted" : "pending",
      submittedByUserId,
      price: 9.99,
    })
    .returning();

  // Only consume a 7-day submission slot for rate-limited tiers.
  if (!isAdmin && tier !== "node_auditor" && user) {
    await db
      .update(usersTable)
      .set({ lastSubmissionDate: now })
      .where(eq(usersTable.id, user.id));
  }

  res.json({ track });
} catch (err) {
  res.status(500).json({ error: "Failed to submit track" });
}
};

/**
 * Resolve the buyer identity for track purchases. A signed-in user
 * (req.dbUser – Clerk/OIDC or session-derived) always wins over the anonymous
 * gk_session row so their purchase lands in THEIR library; /library resolves
 * identity the same way. Exported for tests.
 */
export async function resolveTrackBuyer(req: Request) {
  const sessionId = (req.cookies as Record<string, string>)?.gk_session ?? randomUUID();
  const sessionUser = await storage.getOrCreateUser(sessionId);
  return { buyer: req.dbUser ?? sessionUser, sessionUser };
}

/**
 * POST /api/tracks/:id/payment-intent – embedded-checkout variant of /checkout.
 * Returns a PaymentIntent client secret so the buyer pays in-app (card /
 * Apple Pay / Google Pay) without a redirect. Fulfillment still comes from
 * the verified payment_intent.succeeded webhook and /tracks/confirm-purchase.
 * Identity matches /library: signed-in (Clerk/OIDC) users first, gk_session
 * fallback for anonymous visitors – so a signed-in buyer's purchase lands in
 * THEIR library, not a fresh anonymous session row.
 */
router.post("/tracks/:id/payment-intent", async (req: Request, res: Response) => {
  const { buyer: user, sessionUser } = await resolveTrackBuyer(req);

  const trackId = req.params.id as string;
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```
const [track] = await db.select().from(tracksTable).where(eq(tracksTable.id, trackId));
if (!track) {
  res.status(404).json({ error: "Track not found" });
  return;
}
if (track.status !== "accepted") {
  res.status(403).json({ error: "Track not yet available for purchase" });
  return;
}

const stripe = await getUncachableStripeClient();

const intent = await stripe.paymentIntents.create({
  amount: 999,
  currency: "usd",
  automatic_payment_methods: { enabled: true },
  description: `${track.artistName} - ${track.title}`,
  metadata: { type: "track", track_id: track.id, user_id: user.id },
});
if (!intent.client_secret) {
  res.status(502).json({ error: "Stripe did not return a payment secret" });
  return;
}

// Persist gk_session so purchases are attributable to this browser session.
res.cookie("gk_session", sessionUser.sessionId ?? sessionUser.id, {
  httpOnly: true,
  sameSite: "lax",
  maxAge: 365 * 24 * 60 * 60 * 1000,
  path: "/",
});

void recordAnalyticsEvent({
  type: "checkout_started",
  visitorId: (req.cookies as Record<string, string>)?.gk_vid ?? null,
  sessionId: user.sessionId ?? user.id,
  path: "/label",
  metadata: { trackId: track.id, embedded: "true" },
}).catch(() => {});

res.json({ clientSecret: intent.client_secret, paymentIntentId: intent.id });
});

/**
 * POST /api/tracks/:id/checkout - Stripe one-time checkout for a track.
 * Identity is gk_session-scoped; no OIDC login required.
 * Origin for success/cancel URLs is derived from REPLIT_DOMAINS (never from request headers).
 * DEPRECATED: hosted-checkout fallback kept for older app versions; the web
 * client uses /payment-intent above.
 */
router.post("/tracks/:id/checkout", async (req: Request, res: Response) => {
  const sessionId = (req.cookies as Record<string, string>)?.gk_session ?? randomUUID();
  const user = await storage.getOrCreateUser(sessionId);

  const trackId = req.params.id as string;
  const [track] = await db.select().from(tracksTable).where(eq(tracksTable.id, trackId));
  if (!track) {
    res.status(404).json({ error: "Track not found" });
    return;
  }

  // Only tracks approved by admins can be sold via checkout.
  if (track.status !== "accepted") {
    res.status(403).json({ error: "Track not yet available for purchase" });
    return;
  }

  const stripe = await getUncachableStripeClient();

  let priceId = track.stripePriceId;
  if (!priceId) {
    const product = await stripe.products.create({
      name: `${track.artistName} - ${track.title}`,
      metadata: { trackId: track.id, type: "track" },
    });
    const price = await stripe.prices.create({
      product: product.id,
      unit_amount: 999,
      currency: "usd",
    });
    priceId = price.id;
    await db
      .update(tracksTable)
      .set({ stripeProductId: product.id, stripePriceId: price.id })
      .where(eq(tracksTable.id, trackId));
  }

  // Derive origin from server-side config only - never from req.headers.origin or
  // x-forwarded-* which an attacker-controlled page could spoof.
  const domain = process.env.REPLIT_DOMAINS?.split(",")[0]?.trim() ?? "localhost:80";
  const origin = `https://${domain}`;

  const session = await stripe.checkout.sessions.create({
    line_items: [{ price: priceId, quantity: 1 }],
    mode: "payment",
    success_url: `${origin}/library?cs={CHECKOUT_SESSION_ID}`,
    cancel_url: `${origin}/label/${encodeURIComponent(track.artistName)}`,
    metadata: { track_id: track.id, user_id: user.id, type: "track" },
  });

  // Persist gk_session so purchases are attributable to this browser session.
  res.cookie("gk_session", user.sessionId ?? user.id, {
    httpOnly: true,
    sameSite: "lax",
    maxAge: 365 * 24 * 60 * 60 * 1000,
    path: "/",
  });

  res.json({ url: session.url });
});

/**
 * POST /api/tracks/confirm-purchase - called by the frontend after payment.
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```
* Verifies the Stripe checkout session (hosted redirect flow) or payment
* intent (embedded flow) server-side and records the purchase idempotently.
* Complements the webhook handler for cases where the webhook fires late.
*/
router.post("/tracks/confirm-purchase", async (req: Request, res: Response) => {
  const { checkoutSessionId, paymentIntentId } = req.body as {
    checkoutSessionId?: string;
    paymentIntentId?: string;
  };
  if (!checkoutSessionId && !paymentIntentId) {
    res.status(400).json({ error: "checkoutSessionId or paymentIntentId required" });
    return;
  }

  try {
    const stripe = await getUncachableStripeClient();

    let userId: string;
    let trackId: string;
    let refId: string;

    if (paymentIntentId) {
      const intent = await stripe.paymentIntents.retrieve(paymentIntentId);
      const meta = intent.metadata ?? {};
      if (meta.type !== "track" || !meta.track_id || !meta.user_id || intent.status !== "succeeded") {
        res.status(400).json({ error: "Payment is not a completed track purchase" });
        return;
      }
      userId = meta.user_id;
      trackId = meta.track_id;
      refId = intent.id;
    } else {
      const session = await stripe.checkout.sessions.retrieve(checkoutSessionId as string);
      const meta = session.metadata ?? {};
      if (
        meta.type !== "track" ||
        !meta.track_id ||
        !meta.user_id ||
        !meta.user_id ||
        !(session.payment_status === "paid" || session.status === "complete")
      ) {
        res.status(400).json({ error: "Session is not a completed track purchase" });
        return;
      }
      userId = meta.user_id;
      trackId = meta.track_id;
      refId = session.id;
    }

    await db
      .insert(purchasedTracksTable)
      .values({ userId, trackId, stripeCheckoutSessionId: refId })
      .onConflictDoNothing();

    res.json({ ok: true });
  } catch (_err) {
    res.status(500).json({ error: "Could not confirm purchase" });
  }
});

/**
 * GET /api/tracks/:id/stream - in-app playback for owned tracks.
 * Same ownership gate as /download but does NOT consume the rolling export
 * quota (playback is not an export) and serves inline (no attachment header).
 * Prefers the MP3 sibling to keep playback bandwidth sane; falls back to WAV.
 */
router.get("/tracks/:id/stream", async (req: Request, res: Response) => {
  const session = await resolveSessionUser(req);
  if (!session) {
    res.status(401).json({ error: "Sign in to play this track" });
    return;
  }

  const trackId = req.params.id as string;
  const [track] = await db.select().from(tracksTable).where(eq(tracksTable.id, trackId));
  // No status gate here: playback is ownership-gated below (purchased_tracks).
  // Private/generated tracks MUST stay playable in their owner's library even
  // though they never appear on the public label page.
  if (!track) {
    res.status(404).json({ error: "Track not found" });
    return;
  }

  const [purchase] = await db
    .select()
    .from(purchasedTracksTable)
    .where(
      and(
        eq(purchasedTracksTable.userId, session.userId),
        eq(purchasedTracksTable.trackId, trackId),
      ),
    );
  if (!purchase) {
    res.status(403).json({ error: "This track is not in your library" });
    return;
  }

  const bucketId = process.env.DEFAULT_OBJECT_STORAGE_BUCKET_ID ?? "";
  let audioKey = track.audioFullKey;
  let contentType = "audio/wav";
  let file = null;
  const mp3Key = track.audioFullKey.replace(/\.(\^.\/)+$/, ".mp3");
  if (mp3Key !== track.audioFullKey) {
    const mp3File = await getObjectFileWithFallback(bucketId, mp3Key).catch(() => null);
    if (mp3File) {
      audioKey = mp3Key;
      contentType = "audio/mpeg";
      file = mp3File;
    }
  }
  if (!file) {
    try {
      file = await getObjectFileWithFallback(bucketId, audioKey);
    } catch (err) {
      req.log?.error?.({ err }, "track stream: storage backends unavailable");
      res.status(503).json({ error: "Storage backend unavailable - try again shortly" });
    }
  }
});
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```
        return;
      }
    }
  }
  if (!file) {
    res.status(404).json({ error: "Track audio not found in storage" });
    return;
  }

  res.setHeader("Content-Type", contentType);
  res.setHeader("Cache-Control", "private, max-age=3600");

  file.createReadStream()
    .on("error", (err) => {
      if (!res.headersSent) {
        res.status(500).json({ error: "Playback failed" });
      } else {
        res.destroy(err);
      }
    })
    .pipe(res);
});

/** GET /api/tracks/:id/download - stream purchased track directly to the user's device */
router.get("/tracks/:id/download", async (req: Request, res: Response) => {
  const session = await resolveSessionUser(req);
  const adminBypass = isAdminAutomationAuthenticated(req);
  if (!session && !adminBypass) {
    res.status(401).json({ error: "No session - purchase the track first" });
    return;
  }

  const trackId = req.params.id as string;
  const [track] = await db.select().from(tracksTable).where(eq(tracksTable.id, trackId));
  if (!track) {
    res.status(404).json({ error: "Track not found" });
    return;
  }

  // No status gate: download rights come from the purchased tracks row below.
  // Private/generated tracks stay downloadable by their owner (export quota
  // still applies); label status only controls the PUBLIC page + checkout.

  if (!adminBypass) {
    const [purchase] = await db
      .select()
      .from(purchasedTracksTable)
      .where(and(
        eq(purchasedTracksTable.userId, session!.userId),
        eq(purchasedTracksTable.trackId, trackId),
      ));

    if (!purchase) {
      res.status(403).json({ error: "Purchase this track to download it" });
      return;
    }
  }

  // Stream the file directly - no redirect, no signed URL, file lands on device.
  const bucketId = process.env.DEFAULT_T_OBJECT_STORAGE_BUCKET_ID ?? "";

  // Optional ?format=mp3 - serve the 320 kbps MP3 sibling (audio_full.mp3)
  // when it exists; older tracks without one silently fall back to WAV.
  let audioKey = track.audioFullKey;
  let contentType = "audio/wav";
  // Resolve against the primary bucket first, then the owner-project fallback
  // bucket (tracks vaulted during a primary-bucket outage live there).
  let file = null;
  if (req.query.format === "mp3") {
    const mp3Key = track.audioFullKey.replace(/\.[^./]+$/, ".mp3");
    if (mp3Key !== track.audioFullKey) {
      const mp3File = await getObjectFileWithFallback(bucketId, mp3Key).catch(() => null);
      if (mp3File) {
        audioKey = mp3Key;
        contentType = "audio/mpeg";
        file = mp3File;
      }
    }
  }
  if (!file) {
    try {
      file = await getObjectFileWithFallback(bucketId, audioKey);
    } catch (err) {
      // Storage incident (neither backend checkable) - report it as such,
      // never as a misleading "not found".
      req.log?.error?.({ err }, "track download: storage backends unavailable");
      res.status(503).json({ error: "Storage backend unavailable - try again shortly" });
      return;
    }
  }
  if (!file) {
    res.status(404).json({ error: "Track audio not found in storage" });
    return;
  }

  // Older tracks may predate the stored MP3 sibling. Produce the format the
  // user explicitly requested instead of silently returning a WAV file.
  let generatedMp3: Buffer | null = null;
  if (req.query.format === "mp3" && contentType !== "audio/mpeg") {
    const sourcePath = join(tmpdir(), `gkp-download-${randomUUID()}.source`);
    const mp3Path = join(tmpdir(), `gkp-download-${randomUUID()}.mp3`);
    try {
      const [sourceBuffer] = await file.download();
      writeFileSync(sourcePath, sourceBuffer);
      await execFileAsync("ffmpeg", [
        "-y", "-i", sourcePath,
        "-acodec", "libmp3lame", "-b:a", "320k", "-ar", "44100",
        mp3Path,
      ], { maxBuffer: 50 * 1024 * 1024, timeout: 120_000 });
      generatedMp3 = readFileSync(mp3Path);
      audioKey = track.audioFullKey.replace(/\.[^./]+$/, ".mp3");
      contentType = "audio/mpeg";
    } catch (err) {
      req.log?.error?.({ err, trackId }, "track download: MP3 conversion failed");
    }
  }
});
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```

    res.status(500).json({ error: "Could not prepare this track as MP3" });
    return;
  } finally {
    try { unlinkSync(sourcePath); } catch { /* ignore */ }
    try { unlinkSync(mp3Path); } catch { /* ignore */ }
  }
}

// Charge only after the requested file is confirmed available. Developer
// and admin automation requests remain operational bypasses.
const [dlUser] = session
  ? await db.select().from(usersTable).where(eq(usersTable.id, session.userId))
  : [];
if (dlUser && !adminBypass && !dlUser.isDeveloper) {
  const requestedMp3 = req.query.format === "mp3";
  const creditCost = requestedMp3 ? CREDIT_COSTS.exportMp3 : CREDIT_COSTS.exportWav;
  const spend = await spendCredits(
    dlUser.id,
    creditCost,
    requestedMp3 ? "export_mp3" : "export_wav",
    `track_export:${trackId}:${requestedMp3 ? "mp3" : "wav"}:${randomUUID()}`,
  );
  if (!spend.ok) {
    res.status(402).json({
      error: `${requestedMp3 ? "MP3" : "WAV"} download costs ${creditCost} credits. You have ${spend.balance}.`,
      code: "INSUFFICIENT_CREDITS",
      creditsRequired: creditCost,
      creditsBalance: spend.balance,
      purchaseUrl: "/pricing#credits",
    });
    return;
  }
  res.setHeader("X-GK-Credits-Balance", String(spend.balance));
}

const safeTitle = track.title.replace(/[^a-zA-Z0-9 _-]/g, "").trim() || "track";
const ext = audioKey.split(".").pop() ?? "wav";
const filename = `${safeTitle}.${ext}`;

res.setHeader("Content-Type", contentType);
res.setHeader("Content-Disposition", `attachment; filename=${filename}`);

if (generatedMp3) {
  res.end(generatedMp3);
  return;
}

file.createReadStream()
  .on("error", (err) => {
    if (!res.headersSent) {
      res.status(500).json({ error: "Download failed" });
    } else {
      res.destroy(err);
    }
  })
  .pipe(res);
});

/** GET /api/admin/tracks - admin: list all tracks for moderation */
router.get("/admin/tracks", async (req: Request, res: Response) => {
  const { requireAdmin } = await import("../lib/adminAuth");
  if (!await requireAdmin(req, res)) return;
  try {
    const rows = await db
      .select()
      .from(tracksTable)
      .orderBy(desc(tracksTable.createdAt));
    res.json({ tracks: rows });
  } catch (_err) {
    res.status(500).json({ error: "Failed to list tracks" });
  }
});

/** GET /api/admin/tracks/pending - admin: list only pending tracks */
router.get("/admin/tracks/pending", async (req: Request, res: Response) => {
  const { requireAdmin } = await import("../lib/adminAuth");
  if (!await requireAdmin(req, res)) return;
  try {
    const rows = await db
      .select()
      .from(tracksTable)
      .where(eq(tracksTable.status, "pending"))
      .orderBy(desc(tracksTable.createdAt));
    res.json({ tracks: rows });
  } catch (_err) {
    res.status(500).json({ error: "Failed to list pending tracks" });
  }
});

/** POST /api/admin/tracks/:id/approve - admin approve */
router.post("/admin/tracks/:id/approve", async (req: Request, res: Response) => {
  const { requireAdmin } = await import("../lib/adminAuth");
  if (!await requireAdmin(req, res)) return;

  const trackId = req.params.id as string;
  try {
    const [track] = await db
      .update(tracksTable)
      .set({ status: "accepted" })
      .where(eq(tracksTable.id, trackId))
      .returning();
    if (!track) {
      res.status(404).json({ error: "Track not found" });
      return;
    }
    res.json({ track });
  } catch (_err) {
    res.status(500).json({ error: "Failed to approve track" });
  }
});

/** POST /api/admin/tracks/:id/reject - admin reject */
router.post("/admin/tracks/:id/reject", async (req: Request, res: Response) => {
```

14. VAULT / LIBRARY ORDERING (continued)

artifacts/api-server/src/routes/tracks.ts | source continues without omission

```
const { requireAdmin } = await import("../lib/adminAuth");
if (!await requireAdmin(req, res)) return;

const trackId = req.params.id as string;
try {
  const [track] = await db
    .update(tracksTable)
    .set({ status: "rejected" })
    .where(eq(tracksTable.id, trackId))
    .returning();
  if (!track) {
    res.status(404).json({ error: "Track not found" });
    return;
  }
  res.json({ track });
} catch (_err) {
  res.status(500).json({ error: "Failed to reject track" });
}
});

/** POST /api/admin/tracks/seed - admin-only: seed label tracks into production DB */
router.post("/admin/tracks/seed", async (req: Request, res: Response) => {
  const { requireAdmin } = await import("../lib/adminAuth");
  if (!await requireAdmin(req, res)) return;

  const seedData = [
    {
      title: "You Used to Think I Was Superman",
      artistName: "Tgk, The Gravelking",
      audioFullKey: "private/releases/tgk-the-gravelking/you-used-to-think-i-was-superman-full.wav",
      audioPreviewKey: "releases/tgk-the-gravelking/you-used-to-think-i-was-superman-preview.wav",
      coverArtKey: "releases/tgk-the-gravelking/you-used-to-think-i-was-superman-cover.jpg",
      price: 9.99,
      status: "accepted" as const,
    },
  ],
  ];

  const inserted: string[] = [];
  const skipped: string[] = [];

  for (const t of seedData) {
    const existing = await db
      .select({ id: tracksTable.id })
      .from(tracksTable)
      .where(and(eq(tracksTable.title, t.title), eq(tracksTable.artistName, t.artistName)));
    if (existing.length > 0) {
      skipped.push(t.title);
      continue;
    }
    await db.insert(tracksTable).values(t);
    inserted.push(t.title);
  }

  res.json({ inserted, skipped, count: inserted.length });
});

export default router;
```

15. VAULT / LIBRARY ORDERING

artifacts/gravelkingpro/src/pages/library.tsx | 684 lines | 29,872 bytes | sha256 d8622e836c9e883d1650977ce8719df9dd66924f78a5d6a3dcb81f71e2da4eae

```
import { useEffect, useRef, useState } from "react";
import { Layout } from "@components/layout";
import { Card, CardContent } from "@components/ui/card";
import { Button } from "@components/ui/button";
import { Badge } from "@components/ui/badge";
import { useToast } from "@hooks/use-toast";
import { downloadBlob } from "@lib/download";
import { ExportQuotaBadge } from "@components/export-quota-badge";
import { useExportQuota } from "@hooks/use-export-quota";
import { CreditTopUpDialog, type CreditTopUpRequest } from "@components/credit-top-up-dialog";
import {
  Library, Download, Music, ArrowLeft, Loader2, FileText, Mic,
  CheckCircle2, Clock, Play, Pause, Wand2, ChevronLeft, ChevronRight,
  Pin, Trash2, ImagePlus,
} from "lucide-react";
import { Link, useSearch, useLocation } from "wouter";

interface Track {
  id: string;
  title: string;
  artistName: string;
  coverArtKey: string;
  audioPreviewKey: string;
  price: number;
  lyricsText?: string | null;
  createdAt?: string;
  isPinned?: boolean;
}

interface SongDraft {
  draftId: string;
  genre?: string;
  storyPrompt?: string;
  aiDraft: string;
  authorshipScore: number;
  isCopyrightEligible: boolean;
  is_certified?: boolean;
  lineCount: number;
  createdAt: string;
}

interface AudioJob {
  jobId: string;
  fileName: string;
  mode: string;
  tier: string;
  status: string;
  model?: string;
  createdAt: string;
  completedAt?: string;
}

type Tab = "tracks" | "songs" | "stems";

function formatDate(iso: string): string {
  try {
    return new Date(iso).toLocaleDateString("en-US", { month: "short", day: "numeric", year: "numeric" });
  } catch {
    return "";
  }
}

function fmtTime(s: number): string {
  if (!Number.isFinite(s)) return "0:00";
  const m = Math.floor(s / 60);
  const sec = Math.floor(s % 60);
  return `${m}:${String(sec).padStart(2, "0")}`;
}

/**
 * Selected-track player: cover, transport, seek bar, lyrics, downloads, and
 * a per-track Master action. Playback streams from /api/tracks/:id/stream -
 * the authenticated full-quality route that does NOT consume the export quota.
 */
function TrackPlayer({
  track, onBack, onPrev, onNext, hasPrev, hasNext,
  downloading, onDownload,
}): {
  track: Track;
  onBack: () => void;
  onPrev: () => void;
  onNext: () => void;
  hasPrev: boolean;
  hasNext: boolean;
  downloading: string | null;
  onDownload: (trackId: string, format: "wav" | "mp3") => void;
} {
  const audioRef = useRef<HTMLAudioElement>(null);
  const [playing, setPlaying] = useState(false);
  const [time, setTime] = useState(0);
  const [duration, setDuration] = useState(0);
  const [audioError, setAudioError] = useState<string | null>(null);

  // Reset playback state when the selected track changes.
  useEffect(() => {
    setPlaying(false);
    setTime(0);
    setDuration(0);
    setAudioError(null);
  }, [track.id]);

  const togglePlay = () => {
    const el = audioRef.current;
    if (!el) return;
    if (playing) {
      el.pause();
    } else {
      // iOS: play must launch inside the gesture.
      void el.play().catch(() => {
        setAudioError("Playback failed - the track may still be uploading. Try again shortly.");
      });
    }
  };
};
```

15. VAULT / LIBRARY ORDERING (continued)

artifacts/gravelkingpro/src/pages/library.tsx | source continues without omission

```
const seek = (e: React.ChangeEvent) => {
  const el = audioRef.current;
  if (!el) return;
  const t = Number(e.target.value);
  el.currentTime = t;
  setTime(t);
};

return (
  <div className="space-y-5" data-testid="track-player">
    /* Navigation row */
    <div className="flex items-center justify-between">
      <Button variant="ghost" size="sm" className="gap-1 text-muted-foreground" onClick={onBack} data-testid="button-player-back">
        <ArrowLeft className="w-4 h-4" /> All tracks
      </Button>
      <div className="flex items-center gap-1">
        <Button variant="ghost" size="sm" disabled={!hasPrev} onClick={onPrev} data-testid="button-player-prev">
          <ChevronLeft className="w-4 h-4" />
        </Button>
        <Button variant="ghost" size="sm" disabled={!hasNext} onClick={onNext} data-testid="button-player-next">
          <ChevronRight className="w-4 h-4" />
        </Button>
      </div>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-[280px_1fr] gap-6">
      /* Cover + transport */
      <div className="space-y-4">
        <div className="relative aspect-square rounded-2xl overflow-hidden border border-border/40 bg-secondary/20">
          <img
            src={` /api/storage/public-objects/${track.coverArtKey}`}
            alt={track.title}
            className="w-full h-full object-cover"
            onError={(e) => { (e.target as HTMLImageElement).style.display = "none"; }}
          />
          <button
            onClick={togglePlay}
            className="absolute inset-0 flex items-center justify-center bg-black/30 opacity-0 hover:opacity-100 transition-opacity"
            aria-label={playing ? "Pause" : "Play"}
            data-testid="button-player-cover-toggle"
          >
            {playing
              ? <Pause className="w-14 h-14 text-white drop-shadow-lg" />
              : <Play className="w-14 h-14 text-white drop-shadow-lg" />}
          </button>
        </div>

        <div>
          <h2 className="text-lg font-bold truncate" data-testid="text-player-title">{track.title}</h2>
          <p className="text-sm text-muted-foreground truncate">{track.artistName}</p>
        </div>

        /* Transport */
        <div className="space-y-2">
          <div className="flex items-center gap-3">
            <Button
              size="sm"
              onClick={togglePlay}
              className="bg-amber-500 hover:bg-amber-600 text-black font-bold rounded-full w-10 h-10 p-0"
              data-testid="button-player-play"
            >
              {playing ? <Pause className="w-4 h-4" /> : <Play className="w-4 h-4 ml-0.5" />}
            </Button>
            <input
              type="range"
              min={0}
              max={duration || 0}
              step={0.1}
              value={time}
              onChange={seek}
              className="flex-1 accent-amber-500"
              aria-label="Seek"
            />
          </div>
          <div className="flex justify-between text-[11px] text-muted-foreground font-mono">
            <span>{fmtTime(time)}</span>
            <span>{fmtTime(duration)}</span>
          </div>
          {audioError && <p className="text-xs text-rose-400">{audioError}</p>}
        </div>

        <audio
          ref={audioRef}
          src={` /api/tracks/${track.id}/stream`}
          preload="metadata"
          onPlay={() => setPlaying(true)}
          onPause={() => setPlaying(false)}
          onEnded={() => setPlaying(false)}
          onTimeUpdate={(e) => setTime((e.target as HTMLAudioElement).currentTime)}
          onLoadedMetadata={(e) => setDuration((e.target as HTMLAudioElement).duration)}
          onError={() => setAudioError("Could not load audio for this track.")}
        />

        /* Actions */
        <div className="space-y-2">
          <Link href={` /mastering?gkTrack=${encodeURIComponent(track.id)}&gkTitle=${encodeURIComponent(track.title)}`}>
            <Button className="w-full bg-sky-500 hover:bg-sky-600 text-black font-bold" data-testid="button-player-master">
              <Wand2 className="w-4 h-4 mr-2" /> Master this track
            </Button>
          </Link>
          <div className="grid grid-cols-1 gap-2">
            <Button
              variant="outline"
              className="gap-1.5"
              disabled={downloading === `${track.id}:mp3`}
              onClick={() => onDownload(track.id, "mp3")}
              data-testid="button-player-download-mp3"
            >
              {downloading === `${track.id}:mp3` ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Download className="w-3.5 h-3.5" />}
              Download MP3 (50 Credits)
            </Button>
          </div>
        </div>
      </div>
    </div>
  </div>
);
```

15. VAULT / LIBRARY ORDERING (continued)

artifacts/gravelkingpro/src/pages/library.tsx | source continues without omission

```

        variant="outline"
        className="gap-1.5"
        disabled={downloading === `${track.id}.wav`}
        onClick={() => onDownload(track.id, "wav")}
        data-testid="button-player-download-wav"
      >
        {downloading === `${track.id}.wav` ? <Loader2 className="w-3.5 h-3.5 animate-spin" /> : <Download className="w-3.5 h-3.5" />}
        Download WAV (100 Credits)
      </Button>
    </div>
    <p className="text-[10px] text-muted-foreground/60 text-center">
      Playback is free. Download credits are deducted only after the file is available.
    </p>
  </div>
</div>

/* Lyrics pane */
<div className="rounded-2xl border border-border/40 bg-card/40 p-5 min-h-[300px]">
  <div className="flex items-center gap-2 mb-3">
    <FileText className="w-4 h-4 text-amber-500" />
    <span className="text-sm font-semibold">Lyrics</span>
    {track.lyricsText?.startsWith("[AI-written lyrics]") && (
      <Badge variant="outline" className="text-[10px] border-violet-500/40 text-violet-400 ml-1">
        AI-written
      </Badge>
    )}
  </div>
  {track.lyricsText ? (
    <pre className="text-sm font-mono text-foreground/85 whitespace-pre-wrap leading-relaxed max-h-[480px] overflow-y-auto" data-testid="text-player-lyrics">
      {track.lyricsText.startsWith("[AI-written lyrics]\n")
        ? track.lyricsText.slice("[AI-written lyrics]\n".length)
        : track.lyricsText}
    </pre>
  ) : (
    <p className="text-sm text-muted-foreground italic">
      No lyrics on file for this track – it's an instrumental or an uploaded purchase.
    </p>
  )}
</div>
</div>
);
}

export default function LibraryPage() {
  const { toast } = useToast();
  const [tab, setTab] = useState<Tab>("tracks");
  const [tracks, setTracks] = useState<Track[]>([]);
  const [songs, setSongs] = useState<SongDraft[]>([]);
  const [jobs, setJobs] = useState<AudioJob[]>([]);
  const [loading, setLoading] = useState(true);
  const [studioLoading, setStudioLoading] = useState(true);
  const [confirming, setConfirming] = useState(false);
  const [downloading, setDownloading] = useState<string | null>(null);
  const [busyTrack, setBusyTrack] = useState<string | null>(null);
  const [selectedId, setSelectedId] = useState<string | null>(null);
  const [topUpRequest, setTopUpRequest] = useState<CreditTopUpRequest | null>(null);
  const search = useSearch();
  const [, navigate] = useLocation();
  const { quota, refresh: refreshQuota } = useExportQuota();

  const loadLibrary = async (): Promise<Track[]> => {
    setLoading(true);
    try {
      const r = await fetch("/api/library", { credentials: "include" });
      const d = (await r.json()) as { tracks?: Track[] };
      const list = d.tracks || [];
      setTracks(list);
      return list;
    } catch {
      setTracks([]);
      return [];
    } finally {
      setLoading(false);
    }
  };

  const loadStudio = async () => {
    setStudioLoading(true);
    try {
      const r = await fetch("/api/library/studio", { credentials: "include" });
      const d = (await r.json()) as { songs?: SongDraft[]; jobs?: AudioJob[] };
      setSongs(d.songs || []);
      setJobs(d.jobs || []);
    } catch {
      setSongs([]);
      setJobs([]);
    } finally {
      setStudioLoading(false);
    }
  };

  useEffect(() => {
    const params = new URLSearchParams(search);
    const cs = params.get("cs");
    // ?track=<id> – deep link straight into the player (e.g. right after
    // generation in the Songwriting Studio).
    const wantTrack = params.get("track");
    const finish = async () => {
      const list = await loadLibrary();
      if (wantTrack && list.some((t) => t.id === wantTrack)) {
        setSelectedId(wantTrack);
      }
    };
    if (cs) {
      setConfirming(true);
      fetch("/api/tracks/confirm-purchase", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        credentials: "include",
        body: JSON.stringify({ checkoutSessionId: cs }),
      })
        .then(() => {
```

15. VAULT / LIBRARY ORDERING (continued)

artifacts/gravelkingpro/src/pages/library.tsx | source continues without omission

```
        toast({ title: "Purchase complete!", description: "Your track is now in your library." });
      })
      .catch(() => {})
      .finally(() => {
        setConfirming(false);
        void finish();
      });
    } else {
      void finish();
    }
    loadStudio();
    // eslint-disable-next-line react-hooks/exhaustive-deps
  }, []);

const download = async (trackId: string, format: "wav" | "mp3" = "wav") => {
  setDownloading(`${trackId}:${format}`);
  try {
    const r = await fetch(`/api/tracks/${trackId}/download${format === "mp3" ? "?format=mp3" : ""}`, { credentials: "include" });
    if (!r.ok) {
      const data = (await r.json()).catch(() => ({})) as {
        error?: string;
        code?: string;
        creditsRequired?: number;
        creditsBalance?: number;
      };
      if (r.status === 402 && data.code === "INSUFFICIENT_CREDITS") {
        setTopUpRequest({
          format: format === "mp3" ? "MP3" : "WAV",
          required: data.creditsRequired ?? (format === "mp3" ? 50 : 100),
          balance: data.creditsBalance ?? 0,
        });
        return;
      }
      toast({ title: "Download failed", description: data.error || "Please try again.", variant: "destructive" });
      return;
    }
    const blob = await r.blob();
    const disposition = r.headers.get("Content-Disposition") ?? "";
    const nameMatch = disposition.match(/filename="([~]+)"/);
    const filename = nameMatch?.[1] ?? "track.wav";
    downloadBlob(blob, filename);
    void refreshQuota();
  } catch {
    toast({ title: "Error", description: "Could not start download.", variant: "destructive" });
  } finally {
    setDownloading(null);
  }
}

const togglePin = async (track: Track) => {
  setBusyTrack(track.id);
  try {
    const r = await fetch(`/api/library/${track.id}/pin`, {
      method: "PATCH", credentials: "include", headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ pinned: !track.isPinned }),
    });
    if (!r.ok) throw new Error();
    await loadLibrary();
    toast({ title: track.isPinned ? "Unpinned" : "Pinned", description: `${track.title} updated in your library.` });
  } catch { toast({ title: "Could not update pin", variant: "destructive" }); }
  finally { setBusyTrack(null); }
};

const deleteTrack = async (track: Track) => {
  if (!window.confirm(`Remove "${track.title}" from your library?`)) return;
  setBusyTrack(track.id);
  try {
    const r = await fetch(`/api/library/${track.id}`, { method: "DELETE", credentials: "include" });
    if (!r.ok) throw new Error();
    if (selectedId === track.id) selectTrack(null);
    await loadLibrary();
    toast({ title: "Removed from library", description: `${track.title} is no longer in your library.` });
  } catch { toast({ title: "Could not remove track", variant: "destructive" }); }
  finally { setBusyTrack(null); }
};

const changeCover = async (track: Track, file: File) => {
  setBusyTrack(track.id);
  try {
    const form = new FormData();
    form.append("cover_art", file);
    const r = await fetch(`/api/library/${track.id}/cover`, { method: "PATCH", credentials: "include", body: form });
    if (!r.ok) throw new Error();
    await loadLibrary();
    toast({ title: "Cover updated", description: "Your new cover is now saved." });
  } catch { toast({ title: "Could not update cover", variant: "destructive" }); }
  finally { setBusyTrack(null); }
};

const selectedIdx = tracks.findIndex((t) => t.id === selectedId);
const selected = selectedIdx >= 0 ? tracks[selectedIdx] : null;

const selectTrack = (id: string | null) => {
  setSelectedId(id);
  // Keep the URL shareable/back-navigable without a full reload.
  navigate(id ? `/library?track=${encodeURIComponent(id)}` : "/library", { replace: true });
};

const TabButton = ({ id, label, icon, count }: { id: Tab; label: string; icon: React.ReactNode; count: number }) => (
  <button
    onClick={() => setTab(id)}
    className={`flex items-center gap-2 px-4 py-2 rounded-lg text-sm font-medium transition-colors ${
      tab === id ? "bg-amber-500/20 text-amber-400" : "text-muted-foreground hover:text-foreground hover:bg-white/5"
    }`}
  >
    {icon}
    {label}
    <span className={`text-xs px-1.5 py-0.5 rounded-full ${tab === id ? "bg-amber-500/30" : "bg-muted/40"}`}>{count}</span>
  </button>
);

return (
  <Layout>
```

15. VAULT / LIBRARY ORDERING (continued)

artifacts/gravelkingpro/src/pages/library.tsx | source continues without omission

```
<div className="max-w-6xl mx-auto py-8 space-y-6">
  <div className="flex items-center gap-2">
    <Link href="/label">
      <Button variant="ghost" size="sm" className="gap-1 text-muted-foreground">
        <ArrowLeft className="w-4 h-4" /> Back to Label
      </Button>
    </Link>
  </div>
  <div className="flex flex-wrap items-center gap-2">
    <Library className="w-6 h-6 text-amber-500" />
    <h1 className="text-2xl font-bold">My Library</h1>
    <ExportQuotaBadge quota={quota} className="ml-auto" />
  </div>

  {/* Tabs (hidden while the player is open – the player has its own back nav) */}
  {!selected && (
    <div className="flex flex-wrap gap-2 border-b border-border/40 pb-3">
      <TabButton id="tracks" label="My Tracks" icon={<Music className="w-4 h-4" />} count={tracks.length} />
      <TabButton id="songs" label="Lyrics & Songs" icon={<FileText className="w-4 h-4" />} count={songs.length} />
      <TabButton id="stems" label="Audio Stems" icon={<Mic className="w-4 h-4" />} count={jobs.length} />
    </div>
  )}

  {/* — Player (selected track) — */}
  {selected && (
    <TrackPlayer
      track={selected}
      onBack={() => selectTrack(null)}
      onPrev={() => { if (selectedIdx > 0) selectTrack(tracks[selectedIdx - 1]!.id); }}
      onNext={() => { if (selectedIdx < tracks.length - 1) selectTrack(tracks[selectedIdx + 1]!.id); }}
      hasPrev={selectedIdx > 0}
      hasNext={selectedIdx >= 0 && selectedIdx < tracks.length - 1}
      downloading={downloading}
      onDownload={download}
    />
  )}

  {/* — Track grid — */}
  {!selected && tab === "tracks" && (
    <
      {(loading || confirming) && (
        <div className="flex items-center justify-center py-16">
          <Loader2 className="w-6 h-6 animate-spin text-muted-foreground" />
          {confirming && <span className="ml-2 text-sm text-muted-foreground">Confirming your purchase...</span>}
        </div>
      )}

      {!loading && !confirming && tracks.length === 0 && (
        <div className="text-center py-16">
          <Music className="w-12 h-12 text-muted-foreground mx-auto mb-3" />
          <h3 className="text-lg font-medium">No tracks yet</h3>
          <p className="text-sm text-muted-foreground mt-1">Generate a song in the Songwriting Studio or purchase from the Label.</p>
          <div className="flex items-center justify-center gap-2 mt-4">
            <Link href="/songwriting">
              <Button variant="outline">Songwriting Studio</Button>
            </Link>
            <Link href="/label">
              <Button variant="outline">Browse Label</Button>
            </Link>
          </div>
        </div>
      )}

      {!loading && !confirming && tracks.length > 0 && (
        <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4">
          {tracks.map((t) => (
            <Card
              key={t.id}
              className="border-border/40 bg-card/40 overflow-hidden cursor-pointer hover:border-amber-500/40 transition-colors group"
              onClick={() => selectTrack(t.id)}
              data-testid={`card-track-${t.id}`}
            >
              <div className="relative aspect-square bg-secondary/20 overflow-hidden">
                <img src={` /api/storage/public-objects/${t.coverArtKey}`} alt={t.title} className="w-full h-full object-cover" />
                <div className="absolute inset-0 flex items-center justify-center bg-black/30 opacity-0 group-hover:opacity-100 transition-opacity">
                  <Play className="w-10 h-10 text-white drop-shadow-lg" />
                </div>
                <div className="absolute top-2 right-2 flex gap-1" onClick={(e) => e.stopPropagation()}>
                  <label className="inline-flex items-center justify-center w-8 h-8 rounded-full bg-black/60 text-white hover:bg-black/80 cursor-pointer" title="Change cover">
                    <ImagePlus className="w-4 h-4" />
                    <input type="file" accept="image/*" className="hidden" onChange={(e) => { const file = e.target.files?.[0]; if (file) void changeCover(t, file); e.currentTarget.value = ""; }} />
                  </label>
                  <button className={`w-8 h-8 rounded-full bg-black/60 hover:bg-black/80 ${t.isPinned ? "text-amber-400" : "text-white"}`} title={t.isPinned ? "Unpin" : "Pin"} onClick={() => void togglePin(t)} disabled={busyTrack === t.id}>
                    <Pin className="w-4 h-4 mx-auto" />
                  </button>
                  <button className="w-8 h-8 rounded-full bg-black/60 text-white hover:bg-rose-600" title="Remove from library" onClick={() => void deleteTrack(t)} disabled={busyTrack === t.id}>
                    <Trash2 className="w-4 h-4 mx-auto" />
                  </button>
                </div>
              </div>
              <CardContent className="pt-4 pb-3">
                <div className="font-semibold text-sm truncate">{t.title}</div>
                <div className="text-xs text-muted-foreground truncate">{t.artistName}</div>
                <div className="flex items-center justify-between mt-2">
                  {t.lyricsText
                    ? <Badge variant="outline" className="text-[10px] border-emerald-500/30 text-emerald-400">Lyrics</Badge>
                    : <span />}
                  {t.isPinned && <span className="text-[11px] text-amber-400 flex items-center gap-1"><Pin className="w-3 h-3" /> Pinned</span>}
                  <span className="text-[11px] text-amber-400 font-medium">Open player </span>
                </div>
                <div className="grid grid-cols-1 gap-2 mt-3" onClick={(event) => event.stopPropagation()}>
                  <Button
                    size="sm"
                    variant="outline"
                    disabled={downloading === `${t.id}:mp3`}
                    onClick={() => void download(t.id, "mp3")}
                    data-testid={`button-card-download-mp3-${t.id}`}
                  >
                    {downloading === `${t.id}:mp3` ? <Loader2 className="w-3.5 h-3.5 mr-1.5 animate-spin" /> : <Download className="w-3.5 h-3.5 mr-1.5" />}
                    Download MP3 (50 Credits)
                  </Button>
                  <Button
                    size="sm"

```

15. VAULT / LIBRARY ORDERING (continued)

artifacts/gravelkingpro/src/pages/library.tsx | source continues without omission

```

      variant="outline"
      disabled={downloading === `${t.id}:wav`}
      onClick={() => void download(t.id, "wav")}
      data-testid={`button-card-download-wav-${t.id}`}
    >
      {downloading === `${t.id}:wav` ? <Loader2 className="w-3.5 h-3.5 mr-1.5 animate-spin" /> : <Download className="w-3.5 h-3.5 mr-1.5" />}
      Download WAV (100 Credits)
    </Button>
  </div>
</CardContent>
</Card>
  )}
</div>
  )}
</>
  )}
  <CreditTopUpDialog request={topUpRequest} onOpenChange={open} => { if (open) setTopUpRequest(null); } />

  /* — Lyrics & Songs — */
  {!selected && tab === "songs" && (
    <
      {studioLoading && (
        <div className="flex items-center justify-center py-16">
          <Loader2 className="w-6 h-6 animate-spin text-muted-foreground" />
        </div>
      )}
      {!studioLoading && songs.length === 0 && (
        <div className="text-center py-16">
          <FileText className="w-12 h-12 text-muted-foreground mx-auto mb-3" />
          <h3 className="text-lg font-medium">No saved lyrics yet</h3>
          <p className="text-sm text-muted-foreground mt-1">Songs you save in the Songwriting Studio appear here.</p>
          <Link href="/songwriting">
            <Button className="mt-4" variant="outline">Open Songwriting Studio</Button>
          </Link>
        </div>
      )}
      {!studioLoading && songs.length > 0 && (
        <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
          {songs.map(s) => (
            <Card key={s.draftId} className="border-border/40 bg-card/40">
              <CardContent className="pt-4 pb-4 space-y-2">
                <div className="flex items-center justify-between gap-2">
                  <span className="text-sm font-semibold truncate">{s.genre || "Song"}</span>
                  {s.is certified ? (
                    <span className="flex items-center gap-1 text-xs text-emerald-400 shrink-0">
                      <CheckCircle2 className="w-3.5 h-3.5" /> Certified
                    </span>
                  ) : (
                    <span className="text-xs text-rose-400/80 shrink-0">AI Draft</span>
                  )}
                </div>
                {s.storyPrompt && (
                  <p className="text-xs text-muted-foreground line-clamp-2 italic">&ldquo;{s.storyPrompt}&rdquo;</p>
                )}
                <p className="text-xs text-muted-foreground/80 line-clamp-3 whitespace-pre-wrap">{s.aiDraft.slice(0, 180)}...</p>
                <div className="flex items-center justify-between pt-1 text-xs text-muted-foreground">
                  <span>{s.lineCount} lines · {s.authorshipScore}% human</span>
                  <span className="flex items-center gap-1"><Clock className="w-3 h-3" />{formatDate(s.createdAt)}</span>
                </div>
              </CardContent>
            </Card>
          )}
        </div>
      )}
    </>
  )}

  /* — Audio Stems — */
  {!selected && tab === "stems" && (
    <
      {studioLoading && (
        <div className="flex items-center justify-center py-16">
          <Loader2 className="w-6 h-6 animate-spin text-muted-foreground" />
        </div>
      )}
      {!studioLoading && jobs.length === 0 && (
        <div className="text-center py-16">
          <Mic className="w-12 h-12 text-muted-foreground mx-auto mb-3" />
          <h3 className="text-lg font-medium">No processed audio yet</h3>
          <p className="text-sm text-muted-foreground mt-1">Foundry masters and Vocal Booth sessions appear here after processing.</p>
          <Link href="/mastering">
            <Button className="mt-4" variant="outline">Master a Track</Button>
          </Link>
        </div>
      )}
      {!studioLoading && jobs.length > 0 && (
        <div className="space-y-3">
          {jobs.map(j) => (
            <Card key={j.jobId} className="border-border/40 bg-card/40">
              <CardContent className="py-3 flex items-center gap-3">
                <Mic className="w-5 h-5 text-amber-500/70 shrink-0" />
                <div className="min-w-0 flex-1">
                  <div className="text-sm font-medium truncate">{j.fileName}</div>
                  <div className="text-xs text-muted-foreground">
                    {j.mode.replace(/_/g, " ")} · {j.tier} · {formatDate(j.createdAt)}
                  </div>
                </div>
                <span
                  className={`text-xs px-2 py-1 rounded-full shrink-0 ${
                    j.status.includes("COMPLETE")
                      ? "bg-emerald-500/15 text-emerald-400"
                      : j.status.includes("ERROR") || j.status.includes("FAIL")
                      ? "bg-rose-500/15 text-rose-400"
                      : "bg-amber-500/15 text-amber-400"
                  }`}
                >
                  {j.status.replace(/_/g, " ")}
                </span>
              </CardContent>
            </Card>
          )}
        </div>
      )}
    </>
  )}

```

15. VAULT / LIBRARY ORDERING (continued)
artifacts/gravelkingpro/src/pages/library.tsx | source continues without omission

```
        </>
      })
    </div>
  </Layout>
};
```

16. VAULT / LIBRARY ORDERING

artifacts/gravelkingpro/src/lib/stage-catalog.ts | 149 lines | 5,407 bytes | sha256 f1e500183bef5f653583e7c4cc8bc940fe5562ff571f40228620e7a49063082f

```
import type { LyricTiming } from "@components/prompter/LyricTimer";

export type SerializedAudioBuffer = {
  sampleRate: number;
  length: number;
  numberOfChannels: number;
  channels: Float32Array[];
};

export type StageCatalogAudio = Blob | SerializedAudioBuffer;

export type StageCatalogLine = LyricTiming & { text: string };

export type StageCatalogMixer = {
  instrumentalLevel: number;
  guideLevel: number;
  instrumentalMute: boolean;
  guideMute: boolean;
  instrumentalSolo: boolean;
  guideSolo: boolean;
};

export type StageCatalogEntry = {
  id: string;
  createdAt: string;
  title: string;
  instrumental: StageCatalogAudio;
  guideVocal: StageCatalogAudio | null;
  lines: StageCatalogLine[];
  /** Explicit JSON timing payload for clients that need the LRC-style array. */
  syncedLyrics?: StageCatalogLine[];
  /** Guide-vocal alignment relative to the backing track, in seconds. */
  guideOffsetSeconds?: number;
  /** Decoded instrumental length in seconds, captured at save time for library metadata. */
  durationSeconds?: number;
  mixer?: StageCatalogMixer;
};

const DB_NAME = "gkp-stage-catalog";
const DB_VERSION = 1;
const STORE_NAME = "workshops";

function openStageCatalogOnce(): Promise<IDBDatabase> {
  return new Promise((resolve, reject) => {
    if (typeof indexedDB === "undefined") {
      reject(new Error("Stage Vault storage is not available in this browser."));
      return;
    }
    const request = indexedDB.open(DB_NAME, DB_VERSION);
    request.onupgradeneeded = () => {
      if (!request.result.objectStoreNames.contains(STORE_NAME)) {
        request.result.createObjectStore(STORE_NAME, { keyPath: "id" });
      }
    };
    request.onsuccess = () => {
      const database = request.result;
      database.onversionchange = () => database.close();
      resolve(database);
    };
    request.onerror = () => reject(request.error ?? new Error("Could not open Stage Vault."));
  });
}

function isRecoverableIndexedDbError(cause: unknown): boolean {
  if (!(cause instanceof DOMException)) return false;
  return ["AbortError", "InvalidStateError", "NotFoundError", "TransactionInactiveError", "UnknownError"]
    .includes(cause.name);
}

async function withStageCatalog<T>({
  operation: (database: IDBDatabase) => Promise<T>,
}): Promise<T> {
  let lastError: unknown;
  for (let attempt = 0; attempt < 2; attempt += 1) {
    let database: IDBDatabase | null = null;
    try {
      database = await openStageCatalogOnce();
      return await operation(database);
    } catch (cause) {
      lastError = cause;
      if (attempt > 0 || !isRecoverableIndexedDbError(cause)) throw cause;
      await new Promise((resolve) => setTimeout(resolve, 50));
    } finally {
      database?.close();
    }
  }
  throw lastError;
}

function deserializeLegacyAudioBuffer(context: BaseAudioContext, serialized: SerializedAudioBuffer): AudioBuffer {
  const buffer = context.createBuffer(
    Math.max(1, serialized.numberOfChannels),
    serialized.length,
    serialized.sampleRate,
  );
  serialized.channels.forEach((channel, index) => {
    if (index < buffer.numberOfChannels) buffer.getChannelData(index).set(channel);
  });
  return buffer;
}

function isLegacyAudioBuffer(audio: StageCatalogAudio): audio is SerializedAudioBuffer {
  return "channels" in audio && Array.isArray(audio.channels);
}

export async function decodeStageCatalogAudio(
  context: BaseAudioContext,
  audio: StageCatalogAudio,
): Promise<AudioBuffer> {
  if (isLegacyAudioBuffer(audio)) return deserializeLegacyAudioBuffer(context, audio);
  return context.decodeAudioData(await audio.arrayBuffer());
}
```

16. VAULT / LIBRARY ORDERING (continued)

artifacts/gravelkingpro/src/lib/stage-catalog.ts | source continues without omission

```
export async function saveStageCatalogEntry(entry: StageCatalogEntry): Promise<void> {
  await withStageCatalog((database) => new Promise<void>((resolve, reject) => {
    const transaction = database.transaction(STORE_NAME, "readwrite");
    transaction.objectStore(STORE_NAME).put(entry);
    transaction.oncomplete = () => resolve();
    transaction.onerror = () => reject(transaction.error ?? new Error("Could not save the Stage Vault entry."));
    transaction.onabort = () => reject(transaction.error ?? new DOMException("Stage Vault save was interrupted.", "AbortError"));
  }));
}

export async function listStageCatalogEntries(): Promise<StageCatalogEntry[]> {
  return withStageCatalog((database) => new Promise<StageCatalogEntry[]>((resolve, reject) => {
    const transaction = database.transaction(STORE_NAME, "readonly");
    const request = transaction.objectStore(STORE_NAME).getAll();
    request.onsuccess = () => {
      const entries = (request.result as StageCatalogEntry[]).sort((left, right) => (
        right.createdAt.localeCompare(left.createdAt)
      ));
      resolve(entries);
    };
    request.onerror = () => reject(request.error ?? new Error("Could not read Stage Vault."));
    transaction.onabort = () => reject(transaction.error ?? new DOMException("Stage Vault read was interrupted.", "AbortError"));
  }));
}

export function stageCatalogLinesToPrompterLines(lines: StageCatalogLine[]) {
  return lines
    .slice()
    .sort((left, right) => left.timeSeconds - right.timeSeconds)
    .map((line, index, all) => ({
      id: `stage-${line.lineIndex}-${index}`,
      text: line.text,
      startTimeMs: Math.max(0, line.timeSeconds * 1000),
      endTimeMs: (all[index + 1]??.timeSeconds ?? line.timeSeconds + 3.5) * 1000,
    }));
}
```

17. CREDIT LEDGER & STRIPE HOOKS

artifacts/api-server/src/lib/credits.ts | 195 lines | 7,502 bytes | sha256 db597d33f9f2ec9a18c397745aecb7a370405eeb121f42c88fd4bdf5619be233

```
import { and, desc, eq, gte, sql } from "drizzle-orm";
import type { Request } from "express";
import { randomUUID } from "node:crypto";
import { db, creditTransactionsTable, type User, usersTable } from "@workspace/db";
import { storage } from "../storage";
import { logger } from "../logger";

export const CREDIT_COSTS = {
  song: 20,
  master: 75,
  exportMp3: 50,
  exportWav: 100,
  certificate: 0,
} as const;

export const CREDIT_PACKS = [
  { id: "starter", name: "Starter", credits: 500, bonusCredits: 250, amountCents: 1000, description: "500 credits + 250 bonus credits" },
  { id: "artist", name: "Artist", credits: 1250, bonusCredits: 500, amountCents: 2000, description: "1,250 credits + 500 bonus credits" },
  { id: "studio", name: "Studio", credits: 2500, bonusCredits: 1000, amountCents: 3000, description: "2,500 credits + 1,000 bonus credits" },
] as const;

// Subscription credits are a hard reset, not a rollover balance. The webhook
// resets the wallet to this exact amount at each paid billing renewal.
export const MONTHLY_CREDITS = {
  pro: 800,
  king: 2500,
} as const;

export type CreditSpendKind = "song" | "master" | "export_mp3" | "export_wav" | "certificate";

export async function resolveCreditUser(req: Request): Promise<User | null> {
  if (req.dbUser) return req.dbUser;
  const sessionId = (req.cookies as Record<string, string>)?._gk_session;
  return sessionId ? storage.getUserBySession(sessionId) : null;
}

export async function getCreditsBalance(userId: string): Promise<number> {
  const [row] = await db
    .select({ creditsBalance: usersTable.creditsBalance })
    .from(usersTable)
    .where(eq(usersTable.id, userId));
  return row?.creditsBalance ?? 0;
}

export async function getCreditTransactionHistory(userId: string, page: number, pageSize: number) {
  const safePage = Math.max(1, Math.floor(page));
  const safePageSize = Math.min(50, Math.max(1, Math.floor(pageSize)));
  const rows = await db
    .select({
      id: creditTransactionsTable.id,
      delta: creditTransactionsTable.delta,
      kind: creditTransactionsTable.kind,
      reference: creditTransactionsTable.reference,
      createdAt: creditTransactionsTable.createdAt,
    })
    .from(creditTransactionsTable)
    .where(eq(creditTransactionsTable.userId, userId))
    .orderBy(desc(creditTransactionsTable.createdAt), desc(creditTransactionsTable.id))
    .limit(safePageSize + 1)
    .offset((safePage - 1) * safePageSize);
  const hasMore = rows.length > safePageSize;
  return {
    data: rows.slice(0, safePageSize),
    page: safePage,
    pageSize: safePageSize,
    hasMore,
  };
}

/**
 * Atomically spends credits only when the balance is sufficient, and records
 * the spend in the same transaction. A reference makes a retried request a
 * no-op instead of a second charge.
 */
export async function spendCredits(
  userId: string,
  amount: number,
  kind: CreditSpendKind,
  reference = `${kind}:${randomUUID()}`,
): Promise<{ ok: true; balance: number } | { ok: false; balance: number }> {
  if (!Number.isInteger(amount) || amount <= 0) throw new Error("Credit spend amount must be a positive integer.");
  return db.transaction(async (tx) => {
    const existing = await tx
      .select({ delta: creditTransactionsTable.delta })
      .from(creditTransactionsTable)
      .where(eq(creditTransactionsTable.reference, reference));
    if (existing.length > 0) {
      const [current] = await tx
        .select({ creditsBalance: usersTable.creditsBalance })
        .from(usersTable)
        .where(eq(usersTable.id, userId));
      logger.info(
        `[LEDGER] User ${userId} balance unchanged at ${current?.creditsBalance ?? 0} (idempotent ${kind})`,
      );
      return { ok: true as const, balance: current?.creditsBalance ?? 0 };
    }

    const updated = await tx
      .update(usersTable)
      .set({ creditsBalance: sql`${usersTable.creditsBalance} - ${amount}` })
      .where(and(eq(usersTable.id, userId), gte(usersTable.creditsBalance, amount)))
      .returning({ creditsBalance: usersTable.creditsBalance });
    if (updated.length === 0) {
      const [current] = await tx
        .select({ creditsBalance: usersTable.creditsBalance })
        .from(usersTable)
        .where(eq(usersTable.id, userId));
      logger.info(
        `[LEDGER] User ${userId} balance: ${current?.creditsBalance ?? 0} -> deducted ${amount} -> remaining ${current?.creditsBalance ?? 0} (insufficient ${kind})`,
      );
      return { ok: false as const, balance: current?.creditsBalance ?? 0 };
    }
  })
}
```

17. CREDIT LEDGER & STRIPE HOOKS (continued)

artifacts/api-server/src/lib/credits.ts | source continues without omission

```

    await tx.insert(creditTransactionsTable).values({
      userId,
      delta: -amount,
      kind: `spend_${kind}`,
      reference,
    });
    const remaining = updated[0].creditsBalance;
    logger.info(
      `[LEDGER] User ${userId} balance: ${remaining + amount} -> deducted ${amount} -> remaining ${remaining} (${kind})`,
    );
    return { ok: true as const, balance: remaining };
  });
}

/**
 * Grants credits idempotently. Stripe payment intents and subscription
 * invoices use stable references, so duplicate webhook delivery is harmless.
 */
export async function grantCredits(
  userId: string,
  amount: number,
  kind: string,
  reference: string,
  stripePaymentIntentId?: string,
): Promise<{ granted: boolean; balance: number }> {
  if (!Number.isInteger(amount) || amount <= 0) throw new Error("Credit grant amount must be a positive integer.");
  return db.transaction(async (tx) => {
    const inserted = await tx
      .insert(creditTransactionsTable)
      .values({
        userId,
        delta: amount,
        kind,
        reference,
        stripePaymentIntentId: stripePaymentIntentId ?? null,
      })
      .onConflictDoNothing()
      .returning({ id: creditTransactionsTable.id });

    if (inserted.length === 0) {
      const [current] = await tx
        .select({ creditsBalance: usersTable.creditsBalance })
        .from(usersTable)
        .where(eq(usersTable.id, userId));
      return { granted: false, balance: current?.creditsBalance ?? 0 };
    }

    const [updated] = await tx
      .update(usersTable)
      .set({ creditsBalance: sql`${usersTable.creditsBalance} + ${amount}` })
      .where(eq(usersTable.id, userId))
      .returning({ creditsBalance: usersTable.creditsBalance });
    return { granted: true, balance: updated?.creditsBalance ?? 0 };
  });
}

/** Replace a subscription wallet at renewal; unused credits never roll over. */
export async function resetCredits(
  userId: string,
  amount: number,
  kind: string,
  reference: string,
): Promise<{ reset: boolean; balance: number }> {
  if (!Number.isInteger(amount) || amount <= 0) throw new Error("Credit reset amount must be a positive integer.");
  return db.transaction(async (tx) => {
    const inserted = await tx
      .insert(creditTransactionsTable)
      .values({ userId, delta: amount, kind, reference })
      .onConflictDoNothing()
      .returning({ id: creditTransactionsTable.id });

    if (inserted.length === 0) {
      const [current] = await tx.select({ creditsBalance: usersTable.creditsBalance }).from(usersTable).where(eq(usersTable.id, userId));
      return { reset: false, balance: current?.creditsBalance ?? 0 };
    }

    const [updated] = await tx
      .update(usersTable)
      .set({ creditsBalance: amount })
      .where(eq(usersTable.id, userId))
      .returning({ creditsBalance: usersTable.creditsBalance });
    return { reset: true, balance: updated?.creditsBalance ?? amount };
  });
}
```

18. CREDIT LEDGER & STRIPE HOOKS

artifacts/api-server/src/webhookHandlers.ts | 417 lines | 18,438 bytes | sha256 d4c340c8f70fe613d2e03834a15b90881f7c35fb8533029831bbf2beebd9ae88

```
import type Stripe from 'stripe';
import { getStripeSync, getUncachableStripeClient } from './stripeClient';
import {
  db,
  purchasedTracksTable,
  usersTable,
  promotersTable,
  referralAttributionsTable,
  commissionsTable,
  promoReferralsTable,
  ipCertStubsTable,
} from '@workspace/db';
import { and, eq, sql } from 'drizzle-orm';
import type { Request } from 'express';
import { recordAnalyticsEvent } from './analytics';
import { grantCredits, resetCredits, MONTHLY_CREDITS } from './lib/credits';

/**
 * Loads the managed webhook signing secrets from stripe_managed_webhooks.
 * Exposed as a type so tests can inject a deterministic implementation.
 */
export type SecretsLoader = () => Promise<string[]>;

async function defaultSecretsLoader(): Promise<string[]> {
  // Read ALL secrets ordered newest-first so stale rows are tried last.
  // This is the same table stripe-replit-sync populates via findOrCreateManagedWebhook.
  const result = await db.execute(
    sql`SELECT secret FROM stripe_managed_webhooks ORDER BY created DESC`,
  ) as unknown as { rows: { secret: string }[] };
  return (result.rows ?? []).map((r) => r.secret).filter(Boolean);
}

/**
 * Fulfill a verified credits PaymentIntent without making another Stripe call.
 * Keeping this small, deterministic operation separate lets integration tests
 * replay the same webhook payload without live payment credentials.
 */
export async function fulfillCreditPurchasePaymentIntent(
  intent: Pick<Stripe.PaymentIntent, "id" | "metadata">,
): Promise<{ granted: boolean; balance: number }> {
  const meta = intent.metadata ?? {};
  const credits = Number(meta.credits);
  if (meta.kind !== "credits_purchase" || !meta.user_id || !Number.isInteger(credits) || credits <= 0) {
    throw new Error('Invalid credits purchase metadata for ${intent.id}');
  }
  return grantCredits(
    meta.user_id,
    credits,
    "stripe_purchase",
    `stripe-payment-intent:${intent.id}`,
    intent.id,
  );
}

export class WebhookHandlers {
  /**
   * @param payload Raw request body Buffer (must NOT be parsed by express.json()).
   * @param signature Stripe-Signature header value.
   * @param req Optional Express request for structured logging.
   * @param _loadSecrets Override the signing-secret source (production: DB; tests: injected).
   */
  static async processWebhook(
    payload: Buffer,
    signatures: string,
    req?: Request,
    loadSecrets?: SecretsLoader,
  ): Promise<void> {
    if (!Buffer.isBuffer(payload)) {
      throw new Error(
        'STRIPE WEBHOOK ERROR: Payload must be a Buffer. ' +
        'Received type: ' + typeof payload + '. ' +
        'This usually means express.json() parsed the body before reaching this handler. ' +
        'FIX: Ensure webhook route is registered BEFORE app.use(express.json()).'
      );
    }

    const loadSecrets = _loadSecrets ?? defaultSecretsLoader;

    // Intercept checkout.session.completed for track one-time purchases.
    // We retrieve the signing secret from the stripe_managed_webhooks table so we
    // can construct a verified Stripe event ourselves before stripe-replit-sync
    // processes the webhook (which only mirrors Stripe data into stripe.* tables).
    try {
      const secrets = await loadSecrets();
      let event: Stripe.Event | null = null;
      if (secrets.length > 0) {
        const stripe = await getUncachableStripeClient();
        for (const signingSecret of secrets) {
          try {
            event = stripe.webhooks.constructEvent(payload, signature, signingSecret);
            break;
          } catch {
            // Signed by a different endpoint (e.g. a stale row from a
            // previously connected Stripe account) - try the next secret.
          }
        }
      }
    }

    if (event) {
      if (
        event.type === 'customer.subscription.created' ||
        event.type === 'customer.subscription.updated'
      ) {
        const subscription = event.data.object as Stripe.Subscription;
        const customerId = typeof subscription.customer === 'string'
          ? subscription.customer
          : subscription.customer.id;
        const userId = subscription.metadata?.userId;
        const [user] = userId
          ? await db.select({ id: usersTable.id }).from(usersTable).where(eq(usersTable.id, userId))
          : await db.select({ id: usersTable.id }).from(usersTable).where(eq(usersTable.stripeCustomerId, customerId));
        if (user) {
          const plan = subscription.metadata?.plan || null;

```

18. CREDIT LEDGER & STRIPE HOOKS (continued)

artifacts/api-server/src/webhookHandlers.ts | source continues without omission

```
const entitled = ['active', 'trialing', 'past_due', 'unpaid'].includes(subscription.status);
await db.update(usersTable).set({
  stripeCustomerId: customerId,
  stripeSubscriptionId: subscription.id,
  isPro: entitled,
  subscriptionTier: entitled ? plan : null,
  ...(subscription.status === 'trialing' ? { trialUsed: true } : {}),
}).where(eq(usersTable.id, user.id));
}
}
if (event.type === 'customer.subscription.deleted') {
const subscription = event.data.object as Stripe.Subscription;
await db.update(usersTable).set({
  isPro: false,
  subscriptionTier: null,
  stripeSubscriptionId: null,
}).where(eq(usersTable.stripeSubscriptionId, subscription.id));
}
if (event.type === 'payment_intent.succeeded' || event.type === 'setup_intent.succeeded') {
const intent = event.data.object as Stripe.PaymentIntent | Stripe.SetupIntent;
const userId = intent.metadata?.userId;
if (userId) {
  await db.update(usersTable).set({ isPro: true }).where(eq(usersTable.id, userId));
}
}
// — Embedded one-time purchases (Payment Element flow) —————
// Track sales and $1.99 certificate unlocks now confirm in-app via a
// PaymentIntent, so fulfillment must listen to payment intent.succeeded
// as well as checkout.session.completed (hosted fallback). Same
// metadata contract and the same idempotency guards.
if (event.type === 'payment_intent.succeeded') {
const intent = event.data.object as Stripe.PaymentIntent;
const meta = intent.metadata ?? {};
if (meta.kind === 'credits_purchase' && meta.user_id && meta.credits) {
const credits = Number(meta.credits);
const grant = await fulfillCreditPurchasePaymentIntent(intent);
void recordAnalyticsEvent({
  type: "purchase_completed",
  sessionId: meta.user_id,
  path: "/pricing",
  metadata: {
    kind: "credits",
    credits,
    paymentIntentId: intent.id,
    duplicate: !grant.granted,
  },
}).catch(() => {});
return;
}
if (meta.type === 'track' && meta.track_id && meta.user_id) {
await db
  .insert(purchasedTracksTable)
  .values({
    userId: meta.user_id,
    trackId: meta.track_id,
    stripeCheckoutSessionId: intent.id,
  })
  .onConflictDoNothing();
void recordAnalyticsEvent({
  type: "purchase_completed",
  sessionId: meta.user_id,
  path: "/label",
  metadata: { kind: "track", trackId: meta.track_id, paymentIntentId: intent.id },
}).catch(() => {});
// Track purchases are fully handled here.
return;
}
if (meta.kind === 'cert_unlock' && meta.cert_id && meta.user_id) {
await db
  .update(ipCertStubsTable)
  .set({
    unlockedAt: sql`NOW()`,
    unlockSource: 'purchase',
    stripeSessionId: intent.id,
  })
  .where(
    sql`${ipCertStubsTable.certId} = ${meta.cert_id}
      AND ${ipCertStubsTable.ownerUserId} = ${meta.user_id}
      AND ${ipCertStubsTable.unlockedAt} IS NULL`,
  );
void recordAnalyticsEvent({
  type: "purchase_completed",
  sessionId: meta.user_id,
  path: "/mastering",
  metadata: { kind: "cert_unlock", certId: meta.cert_id, paymentIntentId: intent.id },
}).catch(() => {});
// Cert unlocks are fully handled here.
return;
}
}
if (event.type === 'checkout.session.completed') {
const session = event.data.object;
const userId = session.client_reference_id ?? session.metadata?.userId;
if (userId && session.mode === "subscription") {
await db.update(usersTable).set({ trialUsed: true }).where(eq(usersTable.id, userId));
void recordAnalyticsEvent({
  type: "subscription_activated",
  sessionId: userId,
  path: "/pricing",
  metadata: { plan: session.metadata?.plan ?? "" },
}).catch(() => {});
}
}
if (event.type === 'checkout.session.completed') {
const session = event.data.object;
const meta = session.metadata ?? {};
// Specified metadata contract: track_id and user_id (snake_case).
if (
  meta.track_id &&
  meta.user_id &&
  (session.payment_status === 'paid' || session.status === 'complete')
) {
  // Idempotent insert – duplicate webhook deliveries are safe.
```

18. CREDIT LEDGER & STRIPE HOOKS (continued)

artifacts/api-server/src/webhookHandlers.ts | source continues without omission

```

    await db
      .insert(purchasedTracksTable)
      .values({
        userId: meta.user_id,
        trackId: meta.track_id,
        stripeCheckoutSessionId: session.id,
      })
      .onConflictDoNothing();

    // Track purchases are fully handled here.
    // Do NOT forward to stripe-replit-sync (which mirrors subscription data).
    return;
  }

  // — Per-certificate $1.99 unlock —————
  // Idempotent: the conditional UPDATE only fires while unlocked_at is
  // NULL, so duplicate webhook deliveries are no-ops. Owner-scoped so a
  // paid session can never unlock someone else's certificate.
  if (
    meta.kind === 'cert_unlock' &&
    meta.cert_id &&
    meta.user_id &&
    (session.payment_status === 'paid' || session.status === 'complete')
  ) {
    await db
      .update(ipCertStubsTable)
      .set({
        unlockedAt: sql`NOW()`,
        unlockSource: 'purchase',
        stripeSessionId: session.id,
      })
      .where(
        sql`${ipCertStubsTable.certId} = ${meta.cert_id}
          AND ${ipCertStubsTable.ownerUserId} = ${meta.user_id}
          AND ${ipCertStubsTable.unlockedAt} IS NULL`,
      );
    // Cert unlocks are fully handled here – nothing for stripe-replit-sync.
    return;
  }

  // — Referral commission accrual —————
  // Commission only on actually-paid invoices ($0 trial invoices are
  // skipped by the amount_paid > 0 check). Attribution comes from our
  // own DB (written server-side at checkout), never from the client.
  if (event.type === 'invoice.paid') {
    const invoice = event.data.object;
    const customerId = typeof invoice.customer === 'string' ? invoice.customer : invoice.customer?.id;
    if (customerId && invoice.amount_paid > 0 && invoice.id) {
      const [subscriber] = await db
        .select({
          id: usersTable.id,
          subscriptionTier: usersTable.subscriptionTier,
        })
        .from(usersTable)
        .where(eq(usersTable.stripeCustomerId, customerId));
      const tier = subscriber?.subscriptionTier;
      const monthlyCredits = tier === 'pro' || tier === 'weekly'
        ? MONTHLY_CREDITS.pro
        : tier === 'king' || tier === 'monthly'
        ? MONTHLY_CREDITS.king
        : 0;
      if (subscriber && monthlyCredits > 0) {
        await resetCredits(
          subscriber.id,
          monthlyCredits,
          'subscription_reset',
          `stripe-invoice:${invoice.id}`,
        );
      }
    }
  }
  if (customerId && invoice.amount_paid > 0 && invoice.id) {
    const [user] = await db
      .select({ id: usersTable.id })
      .from(usersTable)
      .where(eq(usersTable.stripeCustomerId, customerId));
    if (user) {
      // Nina's campaign converts only after money is actually paid.
      // The update is idempotent: later paid invoices do not create
      // additional conversions for the same referred account.
      await db
        .update(promoReferralsTable)
        .set({
          status: "converted",
          convertedAt: new Date(),
          stripeInvoiceId: invoice.id,
        })
        .where(
          and(
            eq(promoReferralsTable.referredUserId, user.id),
            eq(promoReferralsTable.promoCodeUsed, "NINA"),
            eq(promoReferralsTable.status, "applied"),
          ),
        );
      const [attribution] = await db
        .select()
        .from(referralAttributionsTable)
        .where(eq(referralAttributionsTable.userId, user.id));
      if (attribution) {
        const [promoter] = await db
          .select()
          .from(promotersTable)
          .where(eq(promotersTable.id, attribution.promoterId));
        // Fraud basics: promoter must still be approved, and
        // self-referral never accrues (defense in depth – checkout
        // already refuses to write such an attribution).
        if (promoter && promoter.status === 'approved' && promoter.userId !== user.id) {
          const commissionCents = Math.floor((invoice.amount_paid * promoter.commissionRate) / 100);
          if (commissionCents > 0) {
            // Idempotent – one commission per Stripe invoice.
            await db
              .insert(commissionsTable)
              .values({
                promoterId: promoter.id,
                userId: user.id,
                stripeInvoiceId: invoice.id,
              })
              .onConflictDoNothing();
          }
        }
      }
    }
  }
}
```

18. CREDIT LEDGER & STRIPE HOOKS (continued)

artifacts/api-server/src/webhookHandlers.ts | source continues without omission

```
        invoiceAmountCents: invoice.amount_paid,
        commissionCents,
        originalCommissionCents: commissionCents,
        currency: invoice.currency ?? 'usd',
      })
      .onConflictDoNothing();
    }
  }
}

// Refunded charge - adjust the matching commission. charge.refunded
// fires for BOTH partial and full refunds; charge.amount_refunded is
// the CUMULATIVE refunded total, so repeated deliveries and multiple
// partial refunds are naturally idempotent (we always recompute the
// net commission from the original accrual and the cumulative total).
// Full refund - status 'reversed' (even if already marked paid, so
// the admin can see the overpayment); partial refund - commission
// reduced proportionally, status unchanged.
if (event.type === 'charge.refunded') {
  const charge = event.data.object as Stripe.Charge & { invoice?: string | { id: string } | null };
  const invoiceId = typeof charge.invoice === 'string' ? charge.invoice : charge.invoice?.id;
  if (invoiceId) {
    const [commission] = await db
      .select()
      .from(commissionsTable)
      .where(eq(commissionsTable.stripeInvoiceId, invoiceId));
    if (commission) {
      const refundedCents = Math.min(
        Math.max(charge.amount_refunded ?? 0, 0),
        commission.invoiceAmountCents,
      );
      // Original accrual basis (rows created before this column existed
      // have 0 - fall back to the current commission).
      const original = commission.originalCommissionCents || commission.commissionCents;
      const fullyRefunded = refundedCents >= commission.invoiceAmountCents;
      const netCommission = fullyRefunded
        ? 0
        : Math.floor(
            (original * (commission.invoiceAmountCents - refundedCents)) /
              commission.invoiceAmountCents,
          );
      await db
        .update(commissionsTable)
        .set({
          refundedCents,
          commissionCents: netCommission,
          ...(fullyRefunded && { status: 'reversed' }),
        })
        .where(eq(commissionsTable.stripeInvoiceId, invoiceId));
    }
  }
}
} catch (err) {
  // Log so the error is observable, but never block stripe-replit-sync processing
  // if our track handler fails.
  const log = req?.log ?? { error: console.error };
  log.error({ err }, 'Custom webhook handler failed');
}

const sync = await getStripeSync();
try {
  await sync.processWebhook(payload, signature);
} catch (err: any) {
  // stripe-replit-sync cannot insert invoice.upcoming events: Stripe sends
  // them with a temporary null ID that violates the invoices NOT NULL
  // constraint. These are pre-billing notices only - skipping is safe and
  // stops Stripe from retrying endlessly (which floods logs with 400s).
  if (err?.code === "23502" && err?.table === "invoices" && err?.column === "id") {
    return;
  }
  throw err;
}
}
```

19. CREDIT LEDGER & STRIPE HOOKS

artifacts/api-server/src/app.ts | 156 lines | 6,172 bytes | sha256 4bf94a5f6ff81b1beda4c566ff00eda4efb0a7906cf0e48f7430eac4a3aecda3

```
import express, { type Express, type Request, type Response, type NextFunction } from "express";
import cors from "cors";
import cookieParser from "cookie-parser";
import pinoHttp from "pino-http";
import { clerkMiddleware } from "@clerk/express";
import { publishableKeyFromHost } from "@clerk/shared/keys";
import router from "./routes";
import { recoverStaleMasterJobs } from "./routes/master";
import { logger } from "./lib/logger";
import { loadAuthUser } from "./middlewares/authMiddleware";
import { maintenanceModeMiddleware } from "./middlewares/maintenanceMode";
import { WebhookHandlers } from "./webhookHandlers";
import {
  CLERK_PROXY_PATH,
  clerkProxyMiddleware,
  getClerkProxyHost,
} from "./middlewares/clerkProxyMiddleware";

// Log crashes before the process dies – helps diagnose production "Processing failed" with no log entry.
process.on("uncaughtException", (err) => logger.fatal({ err }, "uncaughtException"));
process.on("unhandledRejection", (reason) => logger.fatal({ reason }, "unhandledRejection"));

const app: Express = express();

// In-process mastering cannot resume once its /tmp input disappears at a
// restart. Recover stale records early so clients can retry instead of polling
// "running" forever. A not-yet-published schema must not prevent boot.
void recoverStaleMasterJobs().catch((err) => {
  logger.warn({ err }, "unable to recover stale mastering jobs");
});

// Stamp request arrival so routes can report true server-side handler time
// (health.ts serverMs, master.ts X-GK-Timing-* headers).
app.use((_req: Request, res: Response, next: NextFunction) => {
  res.locals.requestStartMs = performance.now();
  next();
});

app.use(
  pinoHttp({
    logger,
    serializers: {
      req(req) {
        return { id: req.id, method: req.method, url: req.url?.split("?")[0] };
      },
      res(res) {
        return { statusCode: res.statusCode };
      },
    },
  }),
);

// Clerk proxy MUST be mounted before body parsers – it streams raw bytes.
app.use(CLERK_PROXY_PATH, clerkProxyMiddleware());

// Webhook route MUST be registered before express.json() so the raw Buffer body is preserved
app.post(
  "/api/stripe/webhook",
  express.raw({ type: "application/json" }),
  async (req, res) => {
    const signature = req.headers["stripe-signature"];
    if (!signature) {
      res.status(400).json({ error: "Missing stripe-signature header" });
      return;
    }

    const sig = Array.isArray(signature) ? signature[0] : signature;

    try {
      await WebhookHandlers.processWebhook(req.body as Buffer, sig, req);
      res.status(200).json({ received: true });
    } catch (err: unknown) {
      const message = err instanceof Error ? err.message : "Unknown error";
      logger.error({ err }, "Webhook processing error");
      res.status(400).json({ error: message });
    }
  },
);

// Build an exact-match allowlist from REPLIT_DOMAINS (comma-separated in prod).
// Also include the Replit dev domain and Expo web preview domain for development.
// In development any localhost / 127.0.0.1 origin is also permitted.
const trustedOrigins: Set<string> = new Set(
  [
    ...(process.env.REPLIT_DOMAINS ?? "").split(",").map(d => d.trim()).filter(Boolean).map(d => `https://${d}`),
    process.env.REPLIT_DEV_DOMAIN ? `https://${process.env.REPLIT_DEV_DOMAIN}` : null,
    // Expo web preview: <id>.expo.<host> – derived from REPLIT_EXPO_DEV_DOMAIN
    process.env.REPLIT_EXPO_DEV_DOMAIN ? `https://${process.env.REPLIT_EXPO_DEV_DOMAIN}` : null,
  ].filter((d): d is string => typeof d === "string"),
);

app.use(
  cors({
    credentials: true,
    origin(requestOrigin, callback) {
      // Same-origin and server-to-server requests have no Origin header – allow.
      if (!requestOrigin) return callback(null, true);
      // Exact match against the allowlist.
      if (trustedOrigins.has(requestOrigin)) return callback(null, true);
      // Permit localhost in development only.
      if (
        process.env.NODE_ENV !== "production" &&
        /^https?:\/\/(localhost|127\.0\.0\.1)(:\d+)?$/i.test(requestOrigin)
      ) {
        return callback(null, true);
      }
      callback(new Error(`CORS: origin not allowed: ${requestOrigin}`));
    },
  }),
);
app.use(cookieParser());
// JAX enterprise sessions may intentionally retransmit a multi-file codebase
// and accumulated diagnostics on every turn. Keep this aligned with the GKA
```

19. CREDIT LEDGER & STRIPE HOOKS (continued)

artifacts/api-server/src/app.ts | source continues without omission

```
// proxy's 2 MB request ceiling while preserving an explicit finite limit.
app.use(express.json({ limit: "2mb" }));
app.use(express.urlencoded({ extended: true }));

// Clerk session validation – resolves the publishable key from the request host
// so the same server can handle multiple custom domains.
app.use(
  clerkMiddleware((req) => ({
    publishableKey: publishableKeyFromHost(
      getClerkProxyHost(req) ?? "",
      process.env.CLERK_PUBLISHABLE_KEY,
    ),
  })),
);

// Load authenticated user into req.dbUser (optional – does not reject requests).
app.use(loadAuthUser);
app.use(maintenanceModeMiddleware);

app.use("/", api", router);

// Keep unknown API paths as real HTTP 404s. The frontend has its own static
// route handling, but API clients must never receive an empty 200 response.
app.use("/api", (_req: Request, res: Response) => {
  res.status(404).json({ success: false, error: "API route not found" });
});

// Global JSON error handler – MUST be last. Catches any error passed to next(err)
// (multer LIMIT_FILE_SIZE, CORS failures, unhandled route errors, etc.) and returns
// JSON instead of Express's default HTML page, which the client can't parse and
// shows as a generic "Processing failed" with no useful message.
// eslint-disable-next-line @typescript-eslint/no-unused-vars
app.use((err: Error, _req: Request, res: Response, _next: NextFunction): void => {
  const status = (err as { status?: number; statusCode?: number }).status
    ?? (err as { status?: number; statusCode?: number }).statusCode
    ?? 500;
  logger.error({ err }, "unhandled express error");
  if (!res.headersSent) {
    res.status(status).json({ success: false, error: err.message ?? "Internal server error" });
  }
});

export default app;
```